



Universidad de Jaén

Escuela Politécnica Superior de Linares

De-bleeding en tiempo real de micrófonos cercanos mediante IA: diseño e implementación de un VST3 multicanal

Estudiante: Enrique Santiago Martínez

Grado: Ingeniería de Tecnologías de Telecomunicación

Dirección del TFG: Pablo Cabañas Molero

Departamento: Ingeniería de Telecomunicación

Dirección del TFG: Pedro Vera Candeas

Departamento: Ingeniería de Telecomunicación

Fecha: 01/09/2026



RESUMEN

En las grabaciones musicales realizadas con varios instrumentos de forma simultánea es habitual situar un micrófono cerca de cada fuente sonora para registrar las señales en pistas independientes. Sin embargo, el aislamiento entre los micrófonos nunca es completo, por lo que cada uno de ellos capta, además del instrumento principal, parte del sonido procedente del resto. Este fenómeno, conocido como microphone bleeding, reduce la independencia entre las pistas y dificulta las tareas posteriores de mezcla, edición y procesamiento.

El objetivo de este Trabajo Fin de Grado es diseñar e implementar un complemento de audio VST3 multicanal capaz de aplicar técnicas de inteligencia artificial para reducir dicha contaminación dentro de una estación de trabajo de audio digital. El sistema parte de un modelo basado en Hybrid Demucs previamente entrenado para la separación de señales de micrófonos cercanos. La aportación principal del trabajo se centra en adaptar e integrar este modelo en un entorno de procesamiento de audio mediante C++, JUCE y LibTorch, asegurando también que funcione en tiempo real con baja latencia.

Para hacer posible su ejecución desde el complemento, el modelo se exporta mediante TorchScript y se emplea Metal Performance Shaders para acelerar la inferencia en equipos Apple. La aplicación desarrollada permite recibir entre uno y doce canales reales, completar internamente la entrada esperada por el modelo y devolver las señales procesadas a pistas independientes dentro de REAPER. También se implementan una ventana histórica de 8192 muestras y un procedimiento de solapamiento-suma para mantener la continuidad entre los fragmentos de audio procesados.

En la configuración principal evaluada, con una frecuencia de muestreo de 48 kHz y bloques de 4096 muestras, la inferencia acelerada presenta tiempos aproximados de entre 58 y 61 ms en el equipo de pruebas, inferiores a los 85,3 ms correspondientes a la duración de cada bloque. Estos resultados muestran la viabilidad del prototipo para mantener un procesamiento continuado, aunque su funcionamiento implica una latencia asociada al tamaño de la ventana y al propio modelo.

Finalmente, se desarrolla un ReaScript que automatiza la creación del bus multicanal, la inserción del complemento y el encaminamiento de sus salidas. El resultado es un prototipo experimental que acerca un modelo de separación de fuentes a un flujo de trabajo real de producción musical.

ABSTRACT

In music recordings in which several instruments are played simultaneously, it is common practice to position a microphone close to each sound source so that the signals can be recorded on separate tracks. However, complete acoustic isolation between microphones cannot be achieved. As a result, each microphone captures not only its intended instrument but also part of the sound produced by the other sources. This phenomenon, known as microphone bleeding, reduces the independence of the tracks and complicates subsequent mixing, editing and audio-processing tasks.

The aim of this Bachelor's dissertation is to design and implement a multichannel VST3 audio plug-in capable of applying artificial intelligence techniques to reduce this unwanted interference within a digital audio workstation. The system is based on a previously trained Hybrid Demucs model designed to separate close-miked signals. The main contribution of this work lies in adapting and integrating the model into an audio-processing environment using C++, JUCE and LibTorch, whilst maintaining real-time operation with low latency.

To allow the model to run from within the plug-in, it is exported using TorchScript, while Metal Performance Shaders is used to accelerate inference on Apple hardware. The developed system can receive between one and twelve real input channels, internally pad the remaining channels to match the input structure required by the model and return the processed signals to separate tracks in REAPER. An 8192 sample history window and an overlap-add procedure are also implemented to preserve continuity between consecutive processed audio segments.

For the main configuration evaluated, using a sample rate of 48 kHz and blocks of 4096 samples, accelerated inference required approximately 58-61 ms on the test system. This was below the 85.3 ms duration of each block. The measurements therefore indicate that the prototype can maintain continuous processing, although its operation introduces latency as a result of the analysis window and the computational requirements of the model.

Finally, a ReaScript was developed to automate the creation of the multichannel bus, the insertion of the plug-in and the routing of its outputs. The resulting experimental prototype integrates an artificial-intelligence-based source-separation model into a practical music production workflow.

ÍNDICE

RESUMEN.....	III
ABSTRACT	IV
ÍNDICE DE TABLAS.....	IX
ÍNDICE DE FIGURAS	X
1. INTRODUCCIÓN	1
1.1. Contexto de la grabación multicanal	1
1.2. Problema del microphone bleeding.....	2
1.3. Antecedentes y punto de partida	4
1.4. Motivación tecnológica	6
1.5. Enfoque y contribución propia	8
1.6. Organización de la memoria.....	10
2. OBJETIVOS, ALCANCE Y REQUISITOS DEL PROYECTO.....	12
2.1. Objetivo general	12
2.2. Objetivos específicos.....	12
2.3. Alcance del proyecto.....	13
2.4. Requisitos de diseño y funcionamiento	14
2.4.1. Requisitos funcionales	14
2.4.2. Requisitos técnicos.....	15
2.4.3. Requisitos temporales.....	16
2.5. Limitaciones del trabajo	17
3. FUNDAMENTOS TEÓRICOS Y TECNOLÓGICOS	18

3.1. Audio digital y procesamiento multicanal.....	19
3.1.1. Representación digital del audio.....	19
3.1.2. Señales multicanal y procesamiento por bloques.....	20
3.2. El problema del bleeding y la separación de fuentes	23
3.2.1. Bleeding en grabaciones multicanal	24
3.2.2. Separación de fuentes y de-bleeding.....	25
3.3. Separación de fuentes mediante aprendizaje profundo.....	27
3.4. Arquitectura Hybrid Demucs	28
3.4.1. Fundamentos de Demucs	28
3.4.2. Procesamiento híbrido.....	29
3.4.3. Modelo heredado: entradas, salidas y procesamiento por segmentos	31
3.5. Procesamiento de audio en tiempo real	33
3.5.1. Procesamiento mediante bloques	33
3.5.2. Latencia, continuidad y plazo de procesamiento.....	34
3.5.3. Diferencia entre bloques y segmentos.....	34
3.6. Complementos de audio y formato VST3.....	35
3.6.1. DAW y complementos de audio.....	36
3.6.2. Formato VST3.....	37
3.6.3. JUCE y REAPER	38
3.7. Ejecución de modelos de PyTorch desde C++	39
3.7.1. PyTorch y TorchScript	39
3.7.2. LibTorch.....	40
3.7.3. Intercambio de datos entre búferes y tensores	41
4. ESTADO DEL ARTE	43
4.1. Métodos convencionales para la prevención y reducción del bleeding	43
4.1.1. Actuaciones durante la captación	43
4.1.2. Control de las relaciones de fase y tiempo	44
4.1.3. Puertas de ruido y expansores.....	45
4.1.4. Ecualización y filtrado	46
4.1.5. Edición manual y tratamiento espectral	46
4.2. Evolución de los modelos de separación de fuentes mediante aprendizaje profundo	48
4.2.1. Métodos basados en espectrogramas	48

4.2.2. Métodos basados en la forma de onda	49
4.2.3. Modelos híbridos	50
4.2.4. Aplicación al de-bleeding de grabaciones multicanal	52
4.3. Herramientas existentes para la reducción del bleeding y la separación de fuentes.....	53
4.4. Sistema heredado	56
4.4.1. Sistema de partida y funcionamiento previo	56
4.4.2. Limitaciones y transición hacia la integración propuesta	57
4.5. Síntesis comparativa y posicionamiento del presente trabajo.....	59
5. MATERIALES Y MÉTODOS	61
5.1. Materiales y entorno de desarrollo	61
5.1.1. Equipo y sistema operativo	62
5.1.2. Herramientas y bibliotecas utilizadas	62
5.1.3. Modelo heredado y material de audio	63
5.1.4. Projucer: reacción del esqueleto del plugin	64
5.1.5. Exportación de modelos con TorchScript.....	67
5.2. Diseño e implementación del complemento VST3	69
5.2.1. Arquitectura general del sistema	69
5.2.2. Proyecto de Xcode: integración de LibTorch	71
5.2.3. Carga del modelo con LibTorch	73
5.2.4. Configuración multicanal del complemento	76
5.2.5. Recepción y acumulación de bloques de audio	78
5.2.6. Ejecución del modelo y aceleración mediante MPS	82
5.2.7. Reconstrucción de la salida y solapamiento-suma	84
5.2.8. Interfaz gráfica y automatización del enrutamiento	88
5.3. Metodología de evaluación	91
6. RESULTADOS Y DISCUSIÓN	94
6.1. Exportación y validación del modelo en TorchScript	94
6.2. Integración y funcionamiento del complemento VST3	99
6.3. Rendimiento temporal	101

6.4. Resultados de la separación multicanal	103
6.5. Limitaciones y valoración global	104
7. CONCLUSIONES Y LÍNEAS FUTURAS	106
7.1. Cumplimiento de los objetivos	106
7.2. Conclusiones principales	108
7.3. Líneas de trabajo futuras.....	110
ANEXOS	112
Anexo A. Configuración del entorno y dependencias.....	112
CONFIGURACIÓN DEL ENTORNO	112
INSTALACIÓN Y CONFIGURACIÓN DE LAS DEPENDENCIAS	113
ORGANIZACIÓN DE LOS ARCHIVOS.....	114
Anexo B. Configuración multicanal y enrutamiento en REAPER	115
INSTALACIÓN Y RECONOCIMIENTO DEL COMPLEMENTO.....	116
CONFIGURACIÓN DEL DISPOSITIVO Y DEL PROYECTO	117
CONFIGURACIÓN MULTICANAL Y ENRUTAMIENTO	118
Anexo C. Guía de los scripts y archivos complementarios.....	118
ARCHIVOS PRINCIPALES	118
GENERACIÓN DE LOS MÓDULOS TORCHSCRIPT	119
EJECUCIÓN DE LA PRUEBA OFFLINE DESDE C++	120
Anexo D. Resultados y mediciones adicionales.....	120
MEDICIONES TEMPORALES EN CPU Y MPS.....	121
COMPROBACIÓN DEL TAMAÑO DE VENTANA	122
VALIDACIÓN DEL ENRUTAMIENTO MULTICANAL.....	123
ALCANCE DE LAS MEDICIONES.....	123
BIBLIOGRAFÍA	124

ÍNDICE DE TABLAS

TABLA 1 DIFERENCIACIÓN ENTRE LOS ELEMENTOS HEREDADOS Y LA CONTRIBUCIÓN PROPIA DEL PRESENTE TFG. FUENTE: ELABORACIÓN PROPIA.....	9
TABLA 2 DIFERENCIAS PRINCIPALES ENTRE LOS BLOQUES ENTREGADOS POR EL ANFITRIÓN Y LOS SEGMENTOS UTILIZADOS POR EL MODELO. FUENTE: ELABORACIÓN PROPIA.	35
TABLA 3 COMPARACIÓN DE LOS PRINCIPALES MÉTODOS CONVENCIONALES EMPLEADOS PARA PREVENIR O REDUCIR EL BLEEDING. FUENTE: ELABORACIÓN PROPIA A PARTIR DE [1], [4] Y [5].	47
TABLA 4 COMPARACIÓN DE ALGUNOS MODELOS REPRESENTATIVOS DE SEPARACIÓN MUSICAL MEDIANTE APRENDIZAJE PROFUNDO. FUENTE: ELABORACIÓN PROPIA A PARTIR DE [7], [8], [21] Y [22].....	51
TABLA 5 COMPARACIÓN DE HERRAMIENTAS EXISTENTES PARA LA REDUCCIÓN DEL BLEEDING Y LA SEPARACIÓN DE FUENTES. FUENTE: ELABORACIÓN PROPIA A PARTIR DE [23], [25], [26] Y [27].	55
TABLA 6 COMPARACIÓN ENTRE LAS ALTERNATIVAS REVISADAS Y EL PLANTEAMIENTO DEL PRESENTE TFG. FUENTE: ELABORACIÓN PROPIA A PARTIR DE LOS APARTADOS ANTERIORES.....	60
TABLA 7 PRINCIPALES HERRAMIENTAS UTILIZADAS DURANTE EL DESARROLLO. FUENTE: ELABORACIÓN PROPIA.	63
TABLA 8 ARCHIVOS PRINCIPALES DE UN PROYECTO DE COMPLEMENTO DE AUDIO GENERADO MEDIANTE PROJUCER. FUENTE: ELABORACIÓN PROPIA.	65
TABLA 9 PRUEBAS Y CRITERIOS EMPLEADOS PARA EVALUAR EL FUNCIONAMIENTO DEL COMPLEMENTO. FUENTE: ELABORACIÓN PROPIA.....	93
TABLA 10 RESUMEN DE LOS TIEMPOS DE INFERENCIA OBTENIDOS DURANTE EL FUNCIONAMIENTO DEL COMPLEMENTO MEDIANTE MPS. FUENTE: ELABORACIÓN PROPIA.	102

ÍNDICE DE FIGURAS

FIGURA 1 REPRESENTACIÓN SIMPLIFICADA DEL FENÓMENO DE MICROPHONE BLEEDING EN UNA GRABACIÓN CON TRES FUENTES SONORAS Y TRES MICRÓFONOS. LAS FLECHAS CONTINUAS MUESTRAN LAS CAPTACIONES PRINCIPALES Y LAS DISCONTINUAS, LAS CAPTACIONES NO DESEADAS. ELABORACIÓN PROPIA.....	3
FIGURA 2 ORGANIZACIÓN DE LOS CANALES Y LAS MUESTRAS DENTRO DE UN BLOQUE DE AUDIO MULTICANAL. FUENTE: ELABORACIÓN PROPIA.....	22
FIGURA 3 ESQUEMA SIMPLIFICADO DEL FUNCIONAMIENTO DE DEMUCS EN EL DOMINIO TEMPORAL. LAS CONEXIONES DE SALTO TRASLADAN INFORMACIÓN ENTRE LAS ETAPAS CORRESPONDIENTES DEL CODIFICADOR Y DEL DECODIFICADOR. FUENTE: ELABORACIÓN PROPIA A PARTIR DE [7].....	29
FIGURA 4 ESQUEMA SIMPLIFICADO DEL PROCESAMIENTO REALIZADO POR HYBRID DEMUCS MEDIANTE UNA RAMA TEMPORAL Y UNA RAMA ESPECTRAL. FUENTE: ELABORACIÓN PROPIA A PARTIR DE [8].	31
FIGURA 5 INTERCAMBIO DE DATOS ENTRE LOS BÚFERES DE AUDIO DEL COMPLEMENTO Y LOS TENSORES UTILIZADOS POR EL MODELO. FUENTE: ELABORACIÓN PROPIA.	42
FIGURA 6 CARACTERIZACIÓN DE UN BLOQUE DE 512 MUESTRAS RECIBIDO POR EL COMPLEMENTO A 48 KHZ DURANTE UNA LLAMADA A <code>PROCESSBLOCK()</code> . FUENTE: ELABORACIÓN PROPIA.	66
FIGURA 7 FRAGMENTOS PRINCIPALES DEL SCRIPT DE EXPORTACIÓN DE HYBRID DEMUCS MEDIANTE TORCHSCRIPT PARA MPS: (A) CREACIÓN DEL TENSOR DE EJEMPLO; (B) TRAZADO DEL MODELO Y GUARDADO DEL MÓDULO .PT. FUENTE: ELABORACIÓN PROPIA.	68
FIGURA 8 ARQUITECTURA GENERAL DEL SISTEMA FORMADO POR REAPER, EL COMPLEMENTO VST3 DESARROLLADO CON JUCE, LIBTORCH, EL MÓDULO HYBRID DEMUCS Y EL SCRIPT DE AUTOMATIZACIÓN DEL ENRUTAMIENTO. FUENTE: ELABORACIÓN PROPIA.	71
FIGURA 9 OBTENCIÓN Y CONFIGURACIÓN DE LIBTORCH: (A) SELECCIÓN DE LA DISTRIBUCIÓN ESTABLE PARA C++ Y MACOS ARM64 EN LA PÁGINA OFICIAL DE PYTORCH; (B) CONFIGURACIÓN DEL EXPORTADOR DE XCODE EN PROJUCER, INCLUYENDO LAS OPCIONES DE COMPILACIÓN, LA RUTA DE BÚSQUEDA DI Y LAS BIBLIOTECAS EXTERNAS. FUENTE: (A) CAPTURA DE LA PÁGINA OFICIAL DE PYTORCH [19]; (B) ELABORACIÓN PROPIA.	73
FIGURA 10 DEFINICIÓN DE LA CLASE PRINCIPAL DEL COMPLEMENTO Y SUS MIEMBROS AÑADIDOS: LA FUNCIÓN DE CARGA DEL MODELO, EL MÓDULO TORCHSCRIPT, EL DISPOSITIVO DE EJECUCIÓN Y LA VARIABLE UTILIZADA PARA REGISTRAR EL ESTADO DEL MODELO. LA CLASE SE DECLARA EN <code>PLUGINPROCESSOR.H</code> . FUENTE: ELABORACIÓN PROPIA.....	74
FIGURA 11 FRAGMENTO DE LA FUNCIÓN <code>LOADMODEL()</code> UTILIZADO PARA SELECCIONAR EL DISPOSITIVO, CARGAR EL MÓDULO TORCHSCRIPT, ESTABLECER EL MODO DE EVALUACIÓN Y CONTROLAR POSIBLES ERRORES DE LIBTORCH. FUENTE: ELABORACIÓN PROPIA.	75
FIGURA 12 CONFIGURACIÓN MULTICANAL DEL PROCESADOR: (A) DECLARACIÓN DE LOS BUSES PRINCIPALES DE ENTRADA Y SALIDA CON UNA DISTRIBUCIÓN ESTÉREO INICIAL; (B) VALIDACIÓN DE CONFIGURACIONES SIMÉTRICAS COMPRENDIDAS ENTRE UNO Y DOCE CANALES MEDIANTE <code>ISBUSES LAYOUTSUPPORTED()</code> . FUENTE: ELABORACIÓN PROPIA.....	77

FIGURA 13 INICIALIZACIÓN DEL PROCESAMIENTO EN <code>PREPARETOPLAY()</code> , INCLUYENDO LA COMPROBACIÓN DE LA FRECUENCIA DE MUESTREO, LA PROTECCIÓN DEL TAMAÑO DE SOLAPE Y LA RESERVA DE LA VENTANA HISTÓRICA MULTICANAL. FUENTE: ELABORACIÓN PROPIA.	79
FIGURA 14 RECEPCIÓN Y ACTUALIZACIÓN DE LA ENTRADA HISTÓRICA EN <code>PROCESSBLOCK()</code> : (A) OBTENCIÓN DEL TAMAÑO DEL BLOQUE Y DEL NÚMERO DE CANALES REALES; (B) DESPLAZAMIENTO DEL HISTORIAL, INCORPORACIÓN DE LAS NUEVAS MUESTRAS Y RELLENO DE LOS CANALES AUSENTES. FUENTE: ELABORACIÓN PROPIA.	81
FIGURA 15 CONVERSIÓN DE LA MEMORIA CONTINUA DE <code>MODELINPUT</code> EN UN TENSOR DE TIPO <code>FLOAT32</code> CON DIMENSIONES <code>[1, 12, N]</code> Y TRASLADO AL DISPOSITIVO SELECCIONADO. FUENTE: ELABORACIÓN PROPIA. 83	
FIGURA 16 EJECUCIÓN DEL MÓDULO <code>TORCHSCRIPT</code> SIN CÁLCULO DE GRADIENTES, PREPARACIÓN DE LOS ARGUMENTOS MEDIANTE <code>IVALUE</code> Y TRASLADO DEL TENSOR RESULTANTE DESDE EL DISPOSITIVO DE INFERENCIA HASTA CPU. FUENTE: ELABORACIÓN PROPIA.....	84
FIGURA 17 PREPARACIÓN DEL SOLAPAMIENTO-SUMA: (A) INICIALIZACIÓN DE LA COLA ANTERIOR, CREACIÓN DE LAS VENTANAS Y ESTADO INICIAL; (B) GENERACIÓN DE LA VENTANA DE HANNING Y SEPARACIÓN DE SUS MITADES DE ENTRADA Y SALIDA. FUENTE: ELABORACIÓN PROPIA.	85
FIGURA 18 RECONSTRUCCIÓN CONTINUA DE LA SALIDA: (A) SUMA DE LA COLA ANTERIOR CON EL INICIO SUAVIZADO DEL BLOQUE ACTUAL Y COPIA DE LA PARTE DIRECTA; (B) ALMACENAMIENTO DE LA NUEVA COLA PONDERADA PARA LA SIGUIENTE LLAMADA. FUENTE: ELABORACIÓN PROPIA.	87
FIGURA 19 INTERFAZ INFORMATIVA DEL COMPLEMENTO <i>DEMUCS12</i> MOSTRADA DENTRO DE REAPER, CON UN RESUMEN DEL PROCESAMIENTO IMPLEMENTADO Y DE LA CONFIGURACIÓN UTILIZADA POR EL PROTOTIPO. FUENTE: ELABORACIÓN PROPIA.	88
FIGURA 20 AUTOMATIZACIÓN DEL ENRUTAMIENTO MULTICANAL MEDIANTE REASCRIP: (A) COMPROBACIÓN DE LAS PISTAS SELECCIONADAS Y ASIGNACIÓN DE LOS CANALES DE ENTRADA; (B) RESULTADO DE LA EJECUCIÓN, CON EL BUS <i>DEMUCS12</i> , LAS PISTAS DE SALIDA Y EL REGISTRO DE LAS CONEXIONES REALIZADAS. FUENTE: ELABORACIÓN PROPIA.	91
FIGURA 21 RESULTADO DE LA EXPORTACIÓN DE HYBRID DEMUCS PARA CPU: (A) CARGA DEL MODELO Y COMPROBACIÓN DE LA ENTRADA Y SALIDA; (B) FINALIZACIÓN DEL TRAZADO Y ALMACENAMIENTO DEL ARCHIVO <i>DEMUCS_EXPORT_TORCHSCRIPT_CPU.PT</i> . FUENTE: ELABORACIÓN PROPIA.....	95
FIGURA 22 RESULTADO DE LA EXPORTACIÓN DE HYBRID DEMUCS PARA MPS: (A) SELECCIÓN DEL DISPOSITIVO Y COMPROBACIÓN DE LA INFERENCIA; (B) FINALIZACIÓN DEL TRAZADO Y ALMACENAMIENTO DEL ARCHIVO <i>DEMUCS_EXPORT_TORCHSCRIPT_MPS.PT</i> . FUENTE: ELABORACIÓN PROPIA.....	97
FIGURA 23 PRUEBA DE LOS MODELOS DESPUÉS DE SU EXPORTACIÓN: (A) CARGA E INFERENCIA DE LA VERSIÓN PARA CPU; (B) CARGA E INFERENCIA DE LA VERSIÓN PARA MPS. FUENTE: ELABORACIÓN PROPIA.	99
FIGURA 24 COMPROBACIÓN DE LA INTEGRACIÓN DEL COMPLEMENTO: (A) RECONOCIMIENTO DE <i>DEMUCS12</i> DENTRO DE REAPER; (B) SELECCIÓN DE MPS, CARGA DEL MODELO <code>TORCHSCRIPT</code> Y CONFIGURACIÓN INICIAL DEL PROCESAMIENTO. FUENTE: ELABORACIÓN PROPIA.	101
FIGURA 25 TIEMPOS DE INFERENCIA REGISTRADOS DURANTE EL PROCESAMIENTO DE BLOQUES DE 4096 MUESTRAS MEDIANTE MPS. FUENTE: ELABORACIÓN PROPIA.	102

FIGURA 26 ORGANIZACIÓN DE LOS ARCHIVOS DEL PROYECTO: (A) DIRECTORIOS PRINCIPALES; (B) CONTENIDO DE LA CARPETA CORRESPONDIENTE AL MODELO, LOS AUDIOS, EL CHECKPOINT Y LOS MÓDULOS EXPORTADOS. FUENTE: ELABORACIÓN PROPIA.	115
FIGURA 27 CONFIGURACIÓN DE REAPER PARA BUSCAR EL COMPLEMENTO DIRECTAMENTE EN LA CARPETA DE COMPILACIÓN GENERADA POR XCODE. FUENTE: ELABORACIÓN PROPIA.	116
FIGURA 28 CONFIGURACIÓN DEL DISPOSITIVO DE AUDIO DE REAPER A 48 KHZ Y CON UN TAMAÑO DE BLOQUE DE 4096 MUESTRAS. FUENTE: ELABORACIÓN PROPIA.	117
FIGURA 29 COMPARACIÓN DE LOS TIEMPOS DE INFERENCIA REGISTRADOS DURANTE LA EJECUCIÓN DEL COMPLEMENTO EN CPU Y MEDIANTE MPS. EN AMBOS CASOS SE UTILIZA UNA FRECUENCIA DE MUESTREO DE 48 KHZ, BLOQUES DE 4096 MUESTRAS, UNA VENTANA HISTÓRICA DE 8192 MUESTRAS Y SOLAPAMIENTO-SUMA DE 512 MUESTRAS. FUENTE: ELABORACIÓN PROPIA.	122

1. INTRODUCCIÓN

1.1. Contexto de la grabación multicanal

Una grabación musical puede estar formada por una única voz o instrumento, pero también puede involucrar a varios intérpretes tocando de manera simultánea. Esta segunda situación es habitual, por ejemplo, en la grabación de grupos musicales, ensayos o actuaciones en directo. Registrar a los músicos al mismo tiempo permite conservar la interacción natural que se produce entre ellos, aunque también hace que el proceso de captación resulte más complejo.

Para disponer de un mayor control sobre el sonido obtenido, normalmente se utilizan varios micrófonos, situados cerca de las diferentes fuentes sonoras. Cada uno de ellos recoge el sonido y lo convierte en una señal que puede almacenarse en el ordenador. Estas señales se organizan habitualmente en pistas independientes, es decir, en espacios diferenciados dentro del proyecto de grabación. De esta forma, el técnico de sonido puede trabajar posteriormente con cada instrumento por separado y realizar los ajustes necesarios sin modificar de manera intencionada los demás [1].

El conjunto de señales se gestiona mediante una estación de trabajo de audio digital, conocida habitualmente por las siglas DAW, procedentes del término inglés *Digital Audio Workstation*. Se trata de un programa informático diseñado para grabar, organizar, editar y combinar señales de audio. A través de este tipo de software es posible modificar, entre otros aspectos, el volumen o el equilibrio tonal de cada pista y, finalmente, combinar todas ellas para obtener el resultado musical deseado. En este proyecto se utiliza REAPER, una DAW que permite trabajar con varias pistas y aplicar herramientas de procesamiento de audio durante la producción [2].

Cuando se registran y procesan varias señales de audio de manera simultánea se trabaja, de forma general, en un entorno multicanal. En condiciones ideales, cada una de estas señales contendría únicamente el instrumento al que se ha destinado su micrófono. Sin embargo, en una grabación real, el sonido se propaga por el espacio y puede alcanzar también los demás micrófonos presentes. Por este motivo, aunque las señales se almacenen en pistas independientes, su contenido no siempre se encuentra completamente separado. Esta situación da lugar al fenómeno conocido como microphone bleeding, que se presenta y analiza en el siguiente apartado.

1.2. Problema del microphone bleeding

Como se ha explicado anteriormente, en una grabación con varios intérpretes es habitual colocar un micrófono próximo a cada voz o instrumento. Esta disposición busca que cada señal represente principalmente una fuente sonora concreta. Sin embargo, un micrófono no puede identificar de dónde procede el sonido: simplemente responde a las variaciones de presión acústica que llegan hasta él. Por este motivo, cuando varias fuentes suenan simultáneamente en un mismo espacio, el sonido de cada una puede alcanzar más de un micrófono.

La fuente situada más cerca suele registrarse con mayor intensidad y constituye el contenido principal de la pista. No obstante, también pueden quedar registrados, a un nivel inferior, los sonidos producidos por otros instrumentos. Por ejemplo, un micrófono colocado delante de una voz puede captar parcialmente la batería o una guitarra que estén sonando en la misma sala. Esta captación no deseada recibe el nombre de microphone bleeding, aunque en la bibliografía también se utilizan términos como microphone leakage o spill [3].

El fenómeno se produce en el propio entorno acústico, antes de que las señales sean almacenadas o procesadas por el ordenador. Por tanto, no se debe a un error de la estación de trabajo de audio ni implica necesariamente que los micrófonos o el resto del equipo funcionen de forma incorrecta. Tampoco debe confundirse con la diafonía eléctrica, originada por el acoplamiento no deseado entre circuitos o vías eléctricas. Aunque algunos trabajos utilizan la expresión microphone cross-talk para referirse al mismo problema acústico, en esta memoria se empleará principalmente el término microphone bleeding.

La Figura 1 muestra una representación simplificada de esta situación. Las flechas continuas indican la captación principal de cada micrófono, mientras que las discontinuas representan algunas de las rutas no deseadas. Como consecuencia, las pistas registradas contienen una fuente dominante, pero no se encuentran completamente aisladas de las demás.

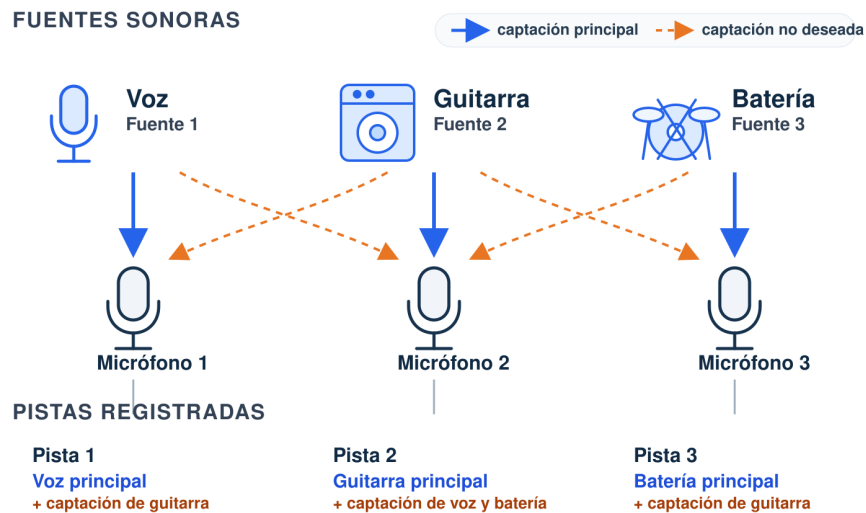


Figura 1 Representación simplificada del fenómeno de microphone bleeding en una grabación con tres fuentes sonoras y tres micrófonos. Las flechas continuas muestran las captaciones principales y las discontinuas, las captaciones no deseadas. Elaboración propia.

La cantidad de bleeding presente no es igual en todas las grabaciones ni en todos los canales. Depende de factores como la distancia entre las fuentes y los micrófonos, el nivel sonoro de cada instrumento, la orientación y directividad de los micrófonos y las características acústicas del recinto. El sonido puede llegar directamente desde otra fuente, pero también después de reflejarse en paredes, techo u otras superficies. La colocación del micrófono cerca del instrumento, técnica conocida como microfonía cercana o close miking, permite que la fuente principal tenga más presencia frente al resto. Sin embargo, incluso utilizando micrófonos direccionales y una colocación cuidadosa, no siempre es posible conseguir un aislamiento completo [4].

La consecuencia más inmediata es la pérdida de independencia entre las pistas. Por ejemplo, reducir el volumen de la pista asignada a una guitarra no garantiza que esta desaparezca de la mezcla, ya que parte de su sonido puede permanecer registrada en los micrófonos de la voz o de la batería. Del mismo modo, al ecualizar, comprimir o modificar una pista no solo se actúa sobre su fuente principal, sino también sobre los sonidos ajenos que contiene. Esta situación dificulta la corrección individual de errores, la sustitución de fragmentos y la aplicación precisa de determinados efectos.

También pueden aparecer problemas de fase cuando una misma fuente es captada por varios micrófonos situados a diferentes distancias. El sonido tarda un tiempo ligeramente distinto en llegar a cada uno de ellos y, al combinar posteriormente las pistas, las señales pueden no quedar perfectamente alineadas. Como resultado, determinadas frecuencias pueden reforzarse mientras que otras se atenúan, alterando el equilibrio tonal de la mezcla.

La separación y colocación de los micrófonos desempeñan, por ello, un papel importante en la reducción de estas interferencias [5].

El microphone bleeding no tiene que considerarse siempre un defecto. En algunos estilos y situaciones puede aportar sensación de espacio, naturalidad o cohesión entre los intérpretes. Sin embargo, cuando se pretende procesar cada instrumento de forma independiente, su presencia limita el control disponible durante la mezcla. En el contexto de este proyecto se considera una interferencia que se desea reducir, procurando conservar la fuente principal de cada canal y evitar la introducción de alteraciones audibles. Esta necesidad constituye el punto de partida de los métodos de reducción y separación de fuentes presentados en los siguientes apartados.

1.3. Antecedentes y punto de partida

La reducción del microphone bleeding se ha abordado tradicionalmente mediante diferentes actuaciones realizadas durante la grabación y la mezcla. Antes de registrar el sonido, pueden emplearse medidas como aumentar la separación entre los intérpretes, colocar barreras acústicas o elegir cuidadosamente la posición y orientación de los micrófonos. Una vez realizada la grabación, también es posible recurrir a herramientas como las puertas de ruido, la edición manual o determinados ajustes de nivel y ecualización. Estas técnicas permiten disminuir la presencia de sonidos no deseados, pero no siempre consiguen eliminarlos por completo, especialmente cuando varios instrumentos suenan simultáneamente y sus contenidos se encuentran mezclados dentro de una misma señal [1], [4].

Esta dificultad favoreció el desarrollo de técnicas de separación de fuentes musicales, conocidas habitualmente como MSS, del inglés Music Source Separation. Su finalidad es estimar por separado las diferentes fuentes que aparecen combinadas en una grabación. Dicho de manera sencilla, se pretende invertir parcialmente el proceso de mezcla: a partir de una señal en la que intervienen varios sonidos, el sistema trata de recuperar la contribución correspondiente a cada uno de ellos [6].

Durante los últimos años, la utilización de redes neuronales profundas ha permitido mejorar considerablemente los resultados obtenidos en esta tarea. Estos sistemas aprenden a reconocer y separar determinados patrones sonoros mediante ejemplos utilizados durante su entrenamiento. Entre las arquitecturas desarrolladas para este propósito se encuentra Demucs, un modelo diseñado para separar fuentes musicales trabajando directamente con las señales de audio [7]. Posteriormente apareció Hybrid Demucs, que combina distintas

formas de representar el sonido con el propósito de aprovechar la información que ofrece cada una de ellas [8]. El funcionamiento detallado de estas arquitecturas se explicará más adelante, dentro de los capítulos dedicados a los fundamentos y al estado del arte.

El antecedente directo del presente proyecto es el Trabajo Fin de Máster realizado por María Dolores Béjar Martos, titulado “Separation of Stems from Close-Microphone Mixtures for Spatialization of Audio Studio Recordings” [9]. En dicho trabajo se desarrolló una adaptación multicanal de Hybrid Demucs orientada específicamente a grabaciones realizadas con microfónica cercana. Para ello, se preparó un conjunto de datos que simulaba diferentes condiciones de grabación en estudio y se entrenó un modelo capaz de reducir el bleeding en configuraciones de hasta doce fuentes simultáneas. Además, el sistema se planteó de forma que no quedara limitado a un conjunto fijo de instrumentos, sino que pudiera actuar sobre diferentes combinaciones de fuentes sonoras.

El trabajo previo incluyó también una primera integración con REAPER destinada a comprobar la aplicación práctica del modelo. En esta solución, las señales se enviaban desde la DAW hasta un programa externo desarrollado en Python mediante un dispositivo de audio virtual denominado BlackHole. El programa procesaba el audio con el modelo y devolvía posteriormente las señales separadas a REAPER. Este prototipo permitió demostrar que el modelo podía incorporarse a un entorno de producción musical, pero el procesamiento seguía realizándose fuera de la propia DAW [9].

Además, las señales se procesaban en segmentos de cinco segundos. Esto provocaba una espera asociada tanto a la duración de cada segmento como al tiempo necesario para ejecutar el modelo. También se observaron pequeñas interrupciones audibles en las uniones entre los bloques procesados. Por tanto, aunque el sistema permitía interactuar con el resultado desde REAPER, su funcionamiento se aproximaba más a un procesamiento por bloques con elevada latencia que a un flujo de audio continuo. Entre las líneas futuras propuestas en aquel trabajo se encontraba precisamente la implementación del modelo como un complemento VST aplicable en tiempo real con baja latencia [9].

A partir de estos resultados se establece el punto de partida del presente TFG. El modelo multicanal previamente entrenado, el conjunto de datos empleado y los resultados obtenidos por María se consideran elementos heredados. En consecuencia, este proyecto no tiene como finalidad volver a diseñar o entrenar la red neuronal, sino estudiar cómo trasladar su funcionamiento a un complemento VST3 que pueda insertarse directamente en REAPER, del mismo modo que cualquier otro efecto de audio.

Para realizar esta integración se plantea desarrollar el sistema como un complemento VST3, formato que permite insertar procesadores de audio directamente en una DAW. El complemento se programará mediante el lenguaje C++ y JUCE, un entorno que facilita el desarrollo de aplicaciones y complementos de audio. LibTorch permitirá cargar y ejecutar

desde C++ el modelo neuronal desarrollado previamente con PyTorch. Esta integración exige adaptar la forma en la que el audio entra y sale del modelo, conservar la correspondencia entre los distintos canales y procesar segmentos más cortos tratando de evitar cortes entre ellos. También resulta necesario evaluar el tiempo empleado en cada operación y comprobar hasta qué punto el sistema puede mantener un procesamiento en tiempo real dentro de la DAW.

De este modo, el presente proyecto constituye una continuación tecnológica del trabajo anterior: se parte de un modelo cuya capacidad de separación ya ha sido estudiada y se centra la investigación en su transformación en un complemento de audio multicanal, así como en el análisis de las limitaciones que aparecen al trabajar con segmentos más cortos y dentro de un flujo de procesamiento continuado. La relevancia de abordar esta evolución se desarrolla en el siguiente apartado.

1.4. Motivación tecnológica

Los resultados del trabajo previo muestran que un modelo basado en aprendizaje profundo puede utilizarse para reducir el microphone bleeding en grabaciones multicanal [9]. Sin embargo, obtener buenos resultados en un entorno experimental no garantiza que la solución pueda utilizarse con facilidad durante una producción musical. Para ello, el modelo debe integrarse en las herramientas empleadas habitualmente por los técnicos de sonido y adaptarse al flujo continuo con el que se procesa el audio. La principal motivación tecnológica de este proyecto consiste, por tanto, en reducir la distancia existente entre un modelo de inteligencia artificial previamente desarrollado y su utilización en tiempo real dentro de un entorno real de producción musical.

Como se ha explicado en el apartado anterior, el prototipo previo necesitaba enviar las señales desde REAPER hasta una aplicación externa en Python y devolver posteriormente el resultado a la DAW. Aunque esta solución permitió comprobar el funcionamiento práctico del modelo, requería configurar varias rutas de audio y mantener diferentes componentes funcionando de forma coordinada. Además, el procesamiento mediante segmentos de cinco segundos introducía una espera elevada y generaba discontinuidades audibles entre algunos de los resultados obtenidos. Su implementación como complemento VST3 permitiría insertar el procesador directamente en REAPER, siguiendo un procedimiento similar al utilizado con otros efectos de audio y manteniendo el flujo de trabajo dentro del propio proyecto de grabación [9], [10].

Para que esta evolución tenga utilidad práctica, el funcionamiento en tiempo real constituye una condición indispensable. En este contexto, trabajar en tiempo real no significa necesariamente eliminar cualquier forma de latencia, sino mantener un procesamiento continuo al mismo ritmo al que avanza el audio, sin producir paradas, cortes o vacíos durante la reproducción. Un sistema podría ofrecer una separación correcta desde el punto de vista del modelo y, aun así, no resultar funcional si su ejecución interrumpiera repetidamente la señal.

Alcanzar este comportamiento supone un reto tecnológico porque una DAW entrega el audio mediante pequeños bloques consecutivos, mientras que el modelo heredado fue concebido para analizar segmentos de mayor duración y requiere una cantidad considerable de operaciones. La integración debe conciliar estas dos formas de funcionamiento, procurando mantener la continuidad del audio sin comprometer de manera significativa la calidad de la separación. Esta relación entre calidad, latencia y coste computacional es especialmente relevante en un sistema basado en aprendizaje profundo.

La naturaleza multicanal del modelo aumenta además la dificultad del problema. En su configuración de mayor tamaño, el sistema puede trabajar con hasta doce señales simultáneas, lo que incrementa la cantidad de información que debe procesarse y exige conservar correctamente la correspondencia entre los diferentes canales. Por tanto, el interés tecnológico del proyecto no reside simplemente en ejecutar una red neuronal, sino en incorporarla a un flujo de audio multicanal continuo y compatible con el funcionamiento de la DAW.

La combinación de JUCE y LibTorch proporciona una base adecuada para llevar a cabo esta integración. JUCE ofrece las herramientas necesarias para desarrollar el complemento y comunicarse con la DAW, mientras que LibTorch permite cargar y ejecutar desde C++ el modelo creado previamente con PyTorch [11], [12]. La unión de ambas tecnologías hace posible trasladar el procesamiento neuronal al interior del complemento y prescindir de la aplicación externa utilizada anteriormente.

Desde un punto de vista práctico, este planteamiento puede facilitar el uso del modelo durante las tareas de edición y mezcla, al integrarlo en un entorno conocido por los profesionales del audio. Desde el ámbito académico, el proyecto permite estudiar la conexión entre el procesamiento digital de audio, la separación de fuentes mediante inteligencia artificial y el desarrollo de complementos. El interés principal se encuentra, por tanto, en transformar un modelo previamente entrenado en una herramienta multicanal integrada y diseñada para funcionar en tiempo real. A partir de esta motivación, el siguiente apartado presenta el enfoque adoptado y delimita la contribución propia del presente TFG.

1.5. Enfoque y contribución propia

A partir de este punto de partida, el trabajo se plantea de forma progresiva. En primer lugar, se desarrolla y comprueba la estructura básica de un complemento VST3 mediante JUCE, verificando su carga en REAPER y estudiando cómo la DAW entrega los bloques de audio, la frecuencia de muestreo y los diferentes canales. Esta etapa permite conocer el entorno en el que posteriormente deberá ejecutarse el modelo y comprobar el recorrido de las señales antes de incorporar el procesamiento neuronal.

A continuación, se prepara el modelo para poder cargarlo y ejecutarlo mediante LibTorch. Para conseguirlo, es necesario adaptar la forma en la que se representan los datos de audio. JUCE almacena las muestras recibidas desde la DAW en búferes, mientras que el modelo necesita trabajar con tensores. Por ello, una parte importante del trabajo consiste en pasar los datos de un formato a otro, respetando sus dimensiones y manteniendo correctamente el orden de los canales de entrada y salida.

Otro punto importante es la adaptación temporal del procesamiento. REAPER entrega el audio al complemento en pequeños bloques consecutivos, pero el modelo necesita segmentos de mayor duración para realizar la separación. Por tanto, es necesario ir reuniendo las muestras hasta completar el segmento requerido, procesarlo y organizar la devolución del resultado sin interrumpir la reproducción. Durante este proceso también deben tenerse en cuenta el tiempo que tarda el modelo, la latencia introducida y la continuidad entre los distintos segmentos procesados.

El funcionamiento en tiempo real no depende únicamente de cuánto tarda el modelo en procesar un segmento aislado. Mientras el complemento ejecuta la separación, REAPER continúa enviando nuevos bloques de audio que deben gestionarse correctamente. Por este motivo, el sistema debe ser capaz de recibir las muestras, organizarlas y entregar el resultado de manera continua. El comportamiento del complemento se estudia dentro de REAPER, ya que comprobar únicamente la ejecución del modelo fuera de la DAW no permitiría conocer su funcionamiento en una situación real de uso.

La Tabla 1 muestra de forma resumida qué elementos proceden del trabajo anterior y cuáles forman parte de la contribución realizada en este TFG.

Aspecto	Punto de partida heredado	Contribución del presente TFG
Modelo de separación	Adaptación multicanal del Hybrid Demucs previamente entrenada para reducir el microphone bleeding.	Preparación e incorporación del modelo en un complemento desarrollado en C++.
Entrenamiento y datos	Conjunto de datos, configuración de	Se reutilizan como puntos de partida sin repetir el

	entrenamiento y parámetros obtenidos en el trabajo previo.	entrenamiento ni crear un nuevo modelo.
Integración con REAPER	Aplicación externa en Python conectada con la DAW mediante BlackHole.	Desarrollo de un complemento VST3 que pueda insertarse directamente en REAPER.
Entorno de ejecución	Ejecución del modelo mediante Pythorch en Python.	Utilización de JUCE para el complemento y de LibTorch para ejecutar el modelo desde C++.
Gestión del audio	Procesamiento externo mediante segmentos de cinco segundos.	Adaptación entre los pequeños bloques entregados por la DAW y los segmentos requeridos del modelo.
Tratamiento multicanal	Modelo preparado para trabajar con configuraciones de hasta doce fuentes.	Gestión de las entradas y salidas del complemento, manteniendo el orden y la correspondencia de los canales.
Funcionamiento temporal	Presencia de una espera elevada y de discontinuidades entre algunos bloques procesados.	Diseño orientado a mantener un flujo continuo y compatible con el funcionamiento en tiempo real.
Evaluación	Estudio de la capacidad de separación y demostración del prototipo externo.	Comprobación de la integración, los tiempos de procesamiento, la latencia, la continuidad del audio y la estabilidad en REAPER.

Tabla 1 Diferenciación entre los elementos heredados y la contribución propia del presente TFG. Fuente: Elaboración propia.

La principal contribución de este TFG se encuentra, por tanto, en todo el sistema desarrollado alrededor del modelo heredado. Esto incluye la creación del complemento VST3, la ejecución del modelo mediante LibTorch, el paso de los datos entre los búferes de JUCE y los tensores utilizados por la red, la gestión de los distintos canales y la adaptación del procesamiento al funcionamiento temporal de la DAW. También forma parte del trabajo el análisis del sistema para conocer su comportamiento y las limitaciones que puedan aparecer durante su utilización.

De este modo, queda definido el alcance del proyecto. No se pretende modificar la arquitectura interna del modelo, volver a entrenarlo ni compararlo con otras redes neuronales. El trabajo se centra en convertir un modelo ya desarrollado y validado en una herramienta integrada directamente en REAPER, capaz de trabajar con audio multicanal de forma continua y en tiempo real. La aportación del TFG tiene, por tanto, un carácter principalmente práctico y

tecnológico, ya que aborda los problemas que aparecen al trasladar el modelo desde un entorno experimental hasta un complemento de audio preparado para utilizarse dentro de una DAW.

1.6. Organización de la memoria

La memoria se organiza en siete capítulos, además de las referencias bibliográficas y los anexos. La estructura seguida permite presentar primero el problema que da origen al trabajo y los conocimientos necesarios para comprenderlo, antes de explicar el desarrollo del complemento, los resultados obtenidos y las conclusiones alcanzadas.

El primer capítulo introduce el problema del microphone bleeding dentro de las grabaciones multicanal. También presenta los antecedentes del proyecto, su motivación tecnológica, el enfoque adoptado y la contribución propia de este TFG.

El segundo capítulo define el objetivo general y los objetivos específicos del trabajo. Además, establece el alcance del proyecto, los requisitos funcionales, técnicos y temporales del complemento y las principales limitaciones consideradas desde el inicio.

El tercer capítulo reúne los fundamentos teóricos y tecnológicos necesarios para comprender el desarrollo. En él se explican los conceptos relacionados con el audio digital, el procesamiento multicanal, el bleeding, la separación de fuentes mediante aprendizaje profundo y la arquitectura Hybrid Demucs. También se presentan el procesamiento de audio en tiempo real, el formato VST3 y las tecnologías utilizadas para ejecutar modelos de PyTorch desde C++, principalmente TorchScript y LibTorch.

El cuarto capítulo revisa el estado del arte. Se presentan los métodos convencionales utilizados para prevenir o reducir el bleeding, la evolución de las técnicas de separación de fuentes mediante aprendizaje profundo y algunas de las herramientas existentes. También se analiza el procesamiento neuronal de audio en tiempo real, la integración de estos sistemas en una DAW y el funcionamiento del modelo heredado que sirve como punto de partida.

El quinto capítulo describe los materiales y métodos utilizados durante el proyecto. Se presentan el equipo, el entorno de desarrollo, las bibliotecas empleadas, el modelo heredado y el material de audio utilizado en las pruebas. A continuación, se explica la metodología

seguida para exportar y validar Hybrid Demucs mediante TorchScript y para integrarlo en el complemento. El capítulo también recoge el diseño y la implementación del VST3, incluyendo la configuración multicanal, la recepción y acumulación de los bloques, la conversión entre búferes y tensores, la ejecución mediante MPS, la reconstrucción de las salidas, el solapamiento-suma, la interfaz informativa y la automatización del enrutamiento mediante ReaScript. Finalmente, se establece el procedimiento utilizado para evaluar el sistema.

El sexto capítulo presenta y analiza los resultados obtenidos. En primer lugar, se comprueba la exportación y validación de las versiones TorchScript para CPU y MPS. Después se estudian el reconocimiento del VST3 por parte de REAPER, la carga del modelo y el funcionamiento completo de la cadena multicanal. También se analizan los tiempos de inferencia, la elección de la ventana de procesamiento, la continuidad del audio y la recepción de señal en las doce pistas de salida. El capítulo termina exponiendo las limitaciones observadas durante el desarrollo y la evaluación del prototipo. De esta forma, los resultados y su interpretación se reúnen en un mismo capítulo para evitar repeticiones.

El séptimo capítulo recoge las conclusiones del trabajo. En él se valora el grado de cumplimiento de los objetivos planteados, se presentan las conclusiones principales extraídas del desarrollo y se proponen posibles líneas de trabajo futuras.

Finalmente, se incluyen las referencias bibliográficas utilizadas y una serie de anexos con información complementaria sobre la instalación y puesta en funcionamiento del sistema, la configuración multicanal de REAPER, el uso de los scripts desarrollados, los fragmentos de código más relevantes y las evidencias adicionales de las pruebas realizadas.

2. OBJETIVOS, ALCANCE Y REQUISITOS DEL PROYECTO

Una vez explicado el problema que da origen al proyecto y el enfoque seguido, en este capítulo se expone qué se pretende conseguir con el TFG. Para ello, se define primero el objetivo general y después se divide en una serie de objetivos específicos. También se establece qué aspectos forman parte del trabajo, cuáles quedan fuera de su alcance y qué condiciones debe cumplir el complemento, especialmente en relación con la gestión multicanal y el funcionamiento en tiempo real. De esta forma, al finalizar el desarrollo se podrá comprobar si los objetivos planteados inicialmente se han cumplido.

2.1. Objetivo general

El objetivo general de este TFG es desarrollar y evaluar un complemento VST3 multicanal que permita integrar y ejecutar directamente en REAPER el modelo basado en Hybrid Demucs desarrollado por [9]. El complemento se implementará en C++ utilizando JUCE y LibTorch y deberá gestionar correctamente las señales de entrada y salida, manteniendo el orden y la correspondencia entre los distintos canales. Además, todo el procesamiento deberá realizarse de manera continua y en tiempo real, con una latencia controlada y sin cortes o paradas que impidan utilizar el complemento con normalidad dentro de la DAW.

2.2. Objetivos específicos

Para alcanzar el objetivo general, el trabajo se divide en varios objetivos específicos. Estos siguen el orden general del desarrollo, desde el estudio del sistema heredado hasta las pruebas finales del complemento dentro de REAPER. Los objetivos planteados son los siguientes:

1. Analizar el funcionamiento del modelo desarrollado en el trabajo anterior, prestando atención a la forma en la que recibe y devuelve las señales, al número de canales y a la duración de los segmentos utilizados durante el procesamiento.
2. Estudiar el funcionamiento básico de los complementos de audio y la manera en la que REAPER entrega los bloques de muestras, con el fin de conocer las condiciones en las que deberá trabajar el modelo.

3. Desarrollar en C++ la estructura básica de un complemento VST3 mediante JUCE y comprobar que puede compilarse, instalarse y cargarse correctamente dentro de REAPER.
4. Configurar las entradas y salidas multicanal del complemento para trabajar con las distintas señales utilizadas por el modelo, manteniendo en todo momento el orden y la correspondencia entre los canales.
5. Preparar el modelo para poder utilizarlo desde C++ e integrarlo en el complemento mediante LibTorch, de forma que pueda cargarse y ejecutarse sin depender de la aplicación externa en Python.
6. Adaptar los datos almacenados en los búferes de audio de JUCE a los tensores que necesita el modelo y realizar también el proceso inverso para devolver las señales procesadas a REAPER.
7. Diseñar un sistema que permita acumular los pequeños bloques entregados por la DAW para crear los segmentos más largos para el modelo (contexto temporal) y organizar después la devolución del audio resultante.
8. Conseguir que el procesamiento se mantenga de forma continua y en tiempo real, evitando que la ejecución del modelo provoque cortes, paradas o vacíos audibles durante la reproducción.
9. Evaluar el funcionamiento del complemento dentro de REAPER mediante pruebas que permitan comprobar su estabilidad, los tiempos de procesamiento, la latencia introducida y la calidad de las señales obtenidas.
10. Comparar el comportamiento de la solución desarrollada con el prototipo anterior, valorando las mejoras conseguidas mediante la integración directa del modelo en un complemento VST3.

En conjunto, estos objetivos cubren tanto la parte de desarrollo e integración como la comprobación final del sistema. Su cumplimiento permitirá determinar si el modelo heredado puede utilizarse directamente dentro de REAPER mediante un complemento multicanal capaz de mantener el procesamiento en tiempo real.

2.3. Alcance del proyecto

El alcance de este TFG comprende el desarrollo de un prototipo funcional que permita utilizar el modelo heredado directamente dentro de REAPER mediante un complemento VST3. El trabajo abarca desde el estudio inicial del modelo y del sistema anterior hasta la integración mediante LibTorch y las pruebas finales realizadas dentro de la DAW.

Por otro lado, el proyecto no incluye el diseño de una nueva arquitectura neuronal, la modificación interna de Hybrid Demucs ni la repetición de su entrenamiento. Tampoco se crearán nuevos conjuntos de datos, ya que se parte del modelo, los parámetros y las señales disponibles del trabajo anterior. De la misma forma, no se realizará una comparación completa con otros modelos de separación de fuentes, puesto que el objetivo principal es integrar y adaptar la solución ya existente.

El desarrollo y las pruebas se centrarán en REAPER y en el formato VST3 dentro del entorno utilizado durante el proyecto. Por tanto, no se pretende comprobar de forma exhaustiva su compatibilidad con todas las DAW, sistemas operativos o configuraciones de hardware disponibles. Aunque la solución desarrollada podría servir como base para futuras adaptaciones, la creación de versiones para otros formatos de complemento también queda fuera del alcance de este trabajo.

En definitiva, el resultado planteado es un prototipo funcional con el que se pueda estudiar la viabilidad de ejecutar el modelo heredado como un complemento VST3 multicanal y en tiempo real. El alcance se centra en resolver los problemas necesarios para conseguir esta integración y en evaluar su funcionamiento, sin entrar en el desarrollo o entrenamiento de la red neuronal utilizada.

2.4. Requisitos de diseño y funcionamiento

Los objetivos anteriores indican qué se pretende conseguir con el proyecto, mientras que los requisitos definen las condiciones que deberá cumplir el complemento para que el resultado pueda considerarse válido. Estos requisitos se dividen en tres grupos: funcionales, técnicos y temporales.

2.4.1. Requisitos funcionales

A diferencia de los objetivos específicos, que describen las tareas necesarias para desarrollar el proyecto, estos requisitos se centran en las operaciones que deberán poder observarse durante el uso del complemento.

En primer lugar, REAPER deberá reconocer el archivo VST3 y permitir insertarlo en una pista o bus configurado para trabajar con audio multicanal. Al iniciarse, el complemento

deberá cargar el modelo exportado y quedar preparado para procesar las señales recibidas. Esta operación no deberá requerir la ejecución simultánea de la aplicación original en Python.

Durante la reproducción, el complemento deberá recibir los bloques de audio suministrados por la DAW y conservar el orden de los doce canales a lo largo de todo el procesamiento. Las muestras recibidas deberán reunirse hasta formar el contexto temporal requerido por Hybrid Demucs, procesarse mediante el modelo y devolverse posteriormente a REAPER a través de los doce canales de salida.

El procesamiento deberá iniciarse automáticamente al comenzar la reproducción, sin que el usuario tenga que ejecutar manualmente cada inferencia. Del mismo modo, deberá continuar mientras REAPER entregue nuevos bloques y detenerse de forma segura cuando se interrumpa la reproducción. La interfaz del complemento tendrá una función únicamente informativa, por lo que no será necesario modificar parámetros para iniciar el procesamiento.

Además, las doce señales generadas deberán poder recuperarse de manera independiente dentro del proyecto. Para facilitar esta operación, el sistema contará con un ReaScript encargado de crear el bus de procesamiento, las pistas de destino y las conexiones necesarias. De esta forma, el usuario podrá escuchar, supervisar y utilizar por separado cada una de las salidas sin tener que construir manualmente todo el enrutamiento multicanal.

Por tanto, desde el punto de vista funcional, el sistema se considerará válido si puede recibir una señal multicanal desde REAPER, procesarla mediante el modelo integrado y devolver sus doce salidas a las pistas correspondientes sin alterar su organización. Las tecnologías empleadas para conseguir este comportamiento se establecen en los requisitos técnicos, mientras que las condiciones relativas a los tiempos de ejecución, la latencia y la continuidad del audio se desarrollan en los requisitos temporales.

2.4.2. Requisitos técnicos

El complemento se desarrollará en C++ utilizando JUCE como entorno para crear su estructura y gestionar la comunicación con REAPER. El resultado deberá generarse en formato VST3, ya que este es el formato elegido para realizar la integración y las pruebas dentro de la DAW.

La ejecución del modelo se llevará a cabo mediante LibTorch. Para ello, el modelo desarrollado originalmente con PyTorch deberá prepararse en un formato que pueda cargarse

desde C++, como TorchScript. Los tensores construidos dentro del complemento deberán respetar el tipo de dato, el número de dimensiones, la disposición de las muestras y el orden de canales esperado por la red.

El complemento también deberá utilizar la frecuencia de muestreo y el tamaño de bloque proporcionados por REAPER. No se supondrá que todos los bloques contienen siempre el mismo número de muestras, ya que este valor puede cambiar dependiendo de la configuración de la DAW y del dispositivo de audio utilizado. La gestión interna deberá funcionar correctamente ante estas variaciones.

El modelo deberá cargarse una sola vez durante la preparación del complemento y conservarse disponible mientras se mantenga su ejecución. De este modo, se evitará repetir una operación de carga costosa cada vez que llegue un nuevo bloque de audio. Del mismo modo, se intentará reutilizar la memoria destinada a los búferes y tensores siempre que sea posible, reduciendo las reservas innecesarias durante el procesamiento.

Por último, la arquitectura deberá separar adecuadamente la recepción de los bloques de audio y las operaciones de mayor coste. La inferencia del modelo no deberá bloquear el hilo de audio durante un tiempo que impida a REAPER continuar entregando y reproduciendo las señales con normalidad.

2.4.3. Requisitos temporales

El funcionamiento en tiempo real constituye uno de los requisitos principales del proyecto. Esto no significa que el resultado tenga que obtenerse sin ninguna latencia, ya que el modelo necesita acumular un segmento de audio antes de poder procesarlo. Sin embargo, una vez iniciada la salida, el sistema deberá mantener el procesamiento de manera continua y sin generar una acumulación progresiva de retraso.

REAPER entrega el audio al complemento mediante bloques consecutivos. El tiempo disponible para atender cada bloque depende del número de muestras que contiene y de la frecuencia de muestreo utilizada, y puede calcularse mediante la siguiente expresión:

$$T_{bloque}(ms) = \frac{B}{f_s} \cdot 1000 \quad (1)$$

donde B es el número de muestras del bloque y f_s es la frecuencia de muestreo expresada en hercios (o muestras/segundo).

Esta expresión permite conocer el intervalo de tiempo correspondiente a cada bloque y utilizarlo como referencia para comprobar el funcionamiento en tiempo real. Las operaciones

realizadas durante la recepción del bloque deberán completarse en un tiempo inferior a T_{bloque} , antes de que REAPER entregue el siguiente. Si se supera este tiempo de forma repetida, pueden aparecer parones, cortes o discontinuidades en el audio.

Por otro lado, el modelo no procesa cada bloque de REAPER de manera independiente, sino que trabaja con segmentos de audio de mayor duración, acumulando muestras pasadas. Por ello, también será necesario comprobar que el tiempo empleado por el modelo en procesar un segmento de longitud N muestras (con $N > B$) no supera de forma continuada la duración T_{bloque} .

La latencia total dependerá principalmente del tiempo necesario para reunir las muestras del segmento, ejecutar el modelo y preparar la salida procesada. Esta latencia deberá mantenerse controlada y estable durante la reproducción. Por tanto, no se considerará funcionamiento en tiempo real un sistema que procese correctamente el audio, pero vaya aumentando progresivamente su retraso respecto a la señal original.

El cumplimiento de estos requisitos se comprobará mediante pruebas dentro de REAPER. En ellas se medirán los tiempos de los bloques y de la inferencia, la latencia introducida y la continuidad de las señales, prestando especial atención a la aparición de cortes, vacíos audibles o acumulaciones de audio sin procesar.

2.5. Limitaciones del trabajo

Las principales limitaciones del trabajo están relacionadas con el modelo utilizado y con las condiciones de evaluación. La calidad de las señales separadas depende de la capacidad del modelo para responder ante grabaciones diferentes de las empleadas durante su desarrollo. Por ello, los posibles errores presentes en la salida no tienen por qué proceder únicamente de la integración realizada, sino que también pueden deberse al propio comportamiento del modelo.

Asimismo, los tiempos de procesamiento y la estabilidad del sistema dependen de los recursos del equipo y de la configuración de audio utilizada. En consecuencia, los resultados permitirán valorar la viabilidad de la solución en las condiciones ensayadas, pero no deben interpretarse como una medida universal de su rendimiento.

3. FUNDAMENTOS TEÓRICOS Y TECNOLÓGICOS

El sistema desarrollado en este trabajo reúne conocimientos relacionados con el audio, la separación de señales y la programación de complementos. Antes de explicar cómo se ha llevado a cabo su desarrollo, es necesario presentar estos conceptos desde sus aspectos más básicos. Por este motivo, el contenido del capítulo se ha organizado de manera progresiva, de modo que cada apartado proporcione la base necesaria para comprender el siguiente.

En primer lugar, se explicará cómo se representa el sonido dentro de un ordenador y qué significa trabajar con varias señales de audio al mismo tiempo. Para ello, se introducirán conceptos como las muestras, la frecuencia de muestreo y los canales. Esta base permitirá comprender posteriormente cómo se organizan las grabaciones captadas mediante varios micrófonos y por qué el sonido de una misma fuente puede aparecer en más de un canal.

A partir de estos principios, se abordará el problema de separar las distintas fuentes presentes en una grabación. Primero se planteará esta tarea de forma general y, después, se explicará cómo puede aplicarse a la reducción del bleeding. A continuación, se introducirá de manera gradual el uso del aprendizaje profundo para realizar esta separación, hasta llegar al funcionamiento general de Demucs y de su variante Hybrid Demucs, en la que se basa el modelo utilizado en el proyecto.

Una vez explicado el modelo, se estudiarán las condiciones que aparecen cuando el audio debe procesarse mientras se está reproduciendo. Se aclararán conceptos como el procesamiento por bloques, la latencia y la necesidad de mantener la continuidad del sonido. También se explicará el funcionamiento general de los complementos de audio y el papel que desempeñan en este trabajo el formato VST3, JUCE y REAPER.

Por último, se presentarán las herramientas que permiten utilizar desde C++ un modelo desarrollado originalmente con PyTorch. En esta parte se introducirán TorchScript y LibTorch, además de las formas empleadas para representar el audio tanto en el complemento como en el modelo.

De esta manera, el capítulo proporcionará una base común para el resto de la memoria. Su finalidad no es describir todavía la implementación realizada, sino facilitar que los capítulos posteriores puedan centrarse directamente en el desarrollo del sistema y en los resultados obtenidos, sin tener que interrumpir su explicación para introducir nuevos conceptos. Esta base permitirá también comprender la relación entre las decisiones adoptadas durante el desarrollo y los principios teóricos y tecnológicos que justifican cada una de ellas.

3.1. Audio digital y procesamiento multicanal

Para que un ordenador pueda almacenar, reproducir o modificar un sonido, primero debe representarlo mediante datos numéricos. El audio deja así de tratarse como una variación continua y pasa a expresarse mediante una sucesión ordenada de valores que puede ser procesada por un sistema digital. En los siguientes apartados se explica cómo se realiza esta representación y cuáles son sus principales características. En primer lugar, se describe cómo una señal de audio se convierte en una secuencia de muestras. Posteriormente, se estudia cómo se organizan varias señales de forma simultánea y cómo los programas de audio dividen esas muestras en pequeños bloques para facilitar su procesamiento.

3.1.1. Representación digital del audio

El sonido se produce mediante variaciones de presión que se propagan a través del aire. Cuando estas variaciones llegan a un micrófono, se transforman en una señal eléctrica cuyo valor cambia a lo largo del tiempo. Esta señal se considera continua porque, al menos de manera ideal, puede tomar un valor diferente en cualquier instante [13]. Sin embargo, un ordenador no puede trabajar directamente con una cantidad continua de valores. Para poder almacenar y procesar la señal, necesita convertirla en una secuencia finita de números. Este proceso se lleva a cabo mediante un convertidor analógico-digital y consta principalmente de dos operaciones: el muestreo y la cuantificación [13].

El muestreo consiste en medir el valor de la señal en instantes separados por un mismo intervalo de tiempo. Cada una de estas mediciones recibe el nombre de muestra. Como resultado, la señal continua original se convierte en una sucesión ordenada de valores que representa cómo evoluciona su amplitud [14]. Una señal continua puede expresarse como $x(t)$, donde t representa el tiempo. Después del muestreo, la señal pasa a representarse como $x[n]$, donde n indica la posición de cada muestra dentro de la secuencia. Este cambio de notación permite distinguir entre la señal original, definida para cualquier instante, y la señal resultante del muestreo, formada por valores separados [14].

La cantidad de muestras obtenidas durante un segundo se denomina frecuencia de muestreo y se representa mediante f_s . Por ejemplo, una frecuencia de muestreo de 48 kHz significa que el sistema obtiene 48 000 muestras por segundo. El tiempo existente entre dos muestras consecutivas recibe el nombre de periodo de muestreo y puede expresarse en segundos como $T_s = 1/f_s$.

La frecuencia de muestreo elegida debe ser suficiente para representar las componentes de frecuencia presentes en el sonido. De acuerdo con el teorema de Nyquist, esta frecuencia debe ser superior al doble de la frecuencia más alta que se desea conservar f_{max} , de modo que $f_s > 2f_{max}$ [14]. Si no se cumple esta condición, pueden aparecer componentes de frecuencia que no estaban presentes en la señal original. Este fenómeno recibe el nombre de aliasing y provoca una representación incorrecta de la señal. Como el límite superior de la audición humana suele situarse aproximadamente en 20 kHz, en las aplicaciones musicales son habituales frecuencias de muestreo como 44,1 kHz y 48 kHz [13].

El muestreo determina en qué momentos se mide la señal, pero todavía es necesario asignar un valor numérico a cada medición. Esta segunda operación se denomina cuantificación. Durante este proceso, la amplitud de cada muestra se aproxima a uno de los niveles disponibles dentro del sistema digital [13]. El número de niveles disponibles depende de la profundidad de bits empleada. Si cada muestra se representa mediante b bits, la cantidad de niveles posibles es 2^b . Por ejemplo, una representación de 16 bits permite utilizar 65 536 niveles diferentes, mientras que una representación de 24 bits dispone de 16 777 216 niveles. Una mayor profundidad de bits permite representar variaciones de amplitud más pequeñas y reduce el error producido al aproximar cada muestra al nivel disponible más cercano [13].

Una de las formas más habituales de representar audio digital sin comprimir es la modulación por impulsos codificados o PCM, del inglés Pulse Code Modulation. En este tipo de representación, las muestras se almacenan de manera sucesiva y cada una contiene el valor de amplitud asignado a la señal en un instante determinado [13]. Dependiendo del sistema utilizado, estos valores pueden almacenarse mediante números enteros o números en coma flotante. En muchas aplicaciones de procesamiento de audio se emplean valores en coma flotante normalizados dentro del intervalo $[-1, 1]$. Los valores positivos y negativos representan las variaciones de la señal respecto a su nivel central, mientras que una sucesión de valores iguales a cero representa la ausencia de señal [13].

La duración correspondiente a una secuencia de audio puede calcularse a partir de su número de muestras N y de la frecuencia de muestreo, como $T = N/f_s$. De este modo, una secuencia de 48 000 muestras reproducida a una frecuencia de 48 kHz tiene una duración de un segundo. Esta relación será especialmente importante más adelante, ya que permitirá calcular la duración de los bloques y segmentos utilizados durante el procesamiento.

3.1.2. Señales multicanal y procesamiento por bloques

En el apartado anterior se ha descrito una señal de audio digital como una sucesión de muestras ordenadas en el tiempo. Hasta ahora, esta explicación se ha centrado en una

única señal. Sin embargo, una grabación puede estar formada por varias señales que deben almacenarse y procesarse de manera simultánea. Cada una de estas señales recibe el nombre de canal de audio. Una grabación mono contiene un único canal, mientras que una grabación estéreo utiliza dos, normalmente denominados canal izquierdo y canal derecho. Cuando el número de canales es superior, se habla de audio multicanal [13].

En una grabación realizada mediante varios micrófonos, cada micrófono puede generar su propio canal. Aunque estos canales contienen valores diferentes, todos deben compartir la misma frecuencia de muestreo y mantenerse sincronizados. Esto significa que la muestra situada en una determinada posición representa el mismo instante temporal en todos los canales. Para identificar una muestra concreta puede utilizarse la notación $x_c[n]$. En ella, c indica el canal al que pertenece la muestra y n representa su posición dentro de la secuencia. Por ejemplo, $x_3[100]$ identifica la muestra situada en la posición 100 del tercer canal.

Es importante no confundir un canal con una fuente sonora completamente aislada. Un canal representa la señal captada por un determinado micrófono o asignada a una pista, pero puede contener sonido procedente de varias fuentes. Por ejemplo, un micrófono situado cerca de un instrumento recogerá principalmente ese instrumento, aunque también puede captar, con menor intensidad, otros sonidos presentes en la grabación. Esta situación será especialmente importante al estudiar posteriormente la separación de fuentes y el problema del bleeding.

En este proyecto se trabaja con una configuración de hasta doce canales de audio. Por tanto, en cada instante no se dispone de un único valor, sino de hasta doce muestras diferentes, una por cada canal. El sistema debe mantener correctamente esta organización para que cada señal pueda procesarse y devolverse en la posición que le corresponde.

Durante la reproducción, un programa de audio no entrega necesariamente la grabación completa al complemento en una sola operación. En su lugar, divide el flujo de audio en grupos consecutivos de muestras denominados bloques. Cada bloque contiene una pequeña parte temporal de la señal y, una vez procesado, se entrega el siguiente. En un sistema multicanal, cada bloque debe contener el mismo intervalo temporal de todos los canales. Por ejemplo, si un bloque comienza en la muestra 1000 y termina en la 1511, debe incluir ese mismo conjunto de muestras para cada uno de los canales. De esta forma, las señales permanecen alineadas durante el procesamiento.

La zona de memoria utilizada para almacenar y transportar estas muestras recibe el nombre de búfer de audio. Aunque los términos bloque y búfer están relacionados, no describen exactamente lo mismo. El bloque es la parte temporal de la señal que se está procesando, mientras que el búfer es la estructura de memoria que contiene sus valores. Un búfer multicanal puede representarse como una tabla en la que cada fila corresponde a un

canal y cada columna representa una posición temporal. Todos los canales contienen el mismo número de muestras dentro del bloque, tal y como se muestra en la Figura 2.



Figura 2 Organización de los canales y las muestras dentro de un bloque de audio multicanal. Fuente: elaboración propia.

El tamaño del bloque indica el número de muestras contenidas en cada canal, no el número total de valores almacenados en el búfer. Por ejemplo, un bloque de 512 muestras y doce canales contiene 512 muestras por canal, lo que supone un total de 6144 valores numéricos.

Como se ha comentado antes, la duración temporal del bloque T_b depende de su número de muestras B y de la frecuencia de muestreo, mediante $T_b = B/f_s$. Por ejemplo, un bloque de 512 muestras procesado a una frecuencia de 48 kHz representa aproximadamente 10,67 ms de audio. Si el sistema trabaja con doce canales, el número total de valores aumenta, pero la duración temporal del bloque continúa siendo 10,67 ms, ya que todos los canales corresponden al mismo intervalo de tiempo.

En los sistemas que utilizan complementos de audio, el programa principal recibe el nombre de anfitrión. Este se encarga de reproducir la grabación y de enviar sucesivamente al complemento los bloques de muestras que deben ser procesados. En este proyecto, REAPER desempeña la función de programa anfitrión. Cada vez que el complemento recibe un bloque, puede leer y modificar las muestras contenidas en él. Una vez finalizado el tratamiento, el

bloque procesado se devuelve al anfitrión para continuar con la reproducción. Este procedimiento se repite de manera sucesiva mientras el audio se encuentra en funcionamiento.

El número de muestras que forma cada bloque depende de la configuración utilizada por el anfitrión. Aunque puede establecerse un tamaño habitual, como 512 muestras, el complemento debe trabajar con la cantidad de muestras realmente recibida en cada momento y mantener correctamente la correspondencia entre todos los canales. El tamaño del bloque influye en la forma en que se realiza el procesamiento. Un bloque pequeño representa una porción temporal más corta y obliga al sistema a efectuar el tratamiento con mayor frecuencia. En cambio, un bloque grande contiene una mayor cantidad de muestras y representa un intervalo de audio más largo. La relación entre el tamaño de los bloques, el tiempo disponible para procesarlos y la latencia se estudiará posteriormente en el apartado dedicado al procesamiento de audio en tiempo real.

Por último, los bloques enviados por el anfitrión no deben confundirse con los segmentos de audio que necesita el modelo de separación. Un bloque puede contener únicamente unos cientos de muestras, mientras que el modelo puede requerir una cantidad de audio mucho mayor para efectuar la separación. Por este motivo, será necesario reunir varios bloques consecutivos hasta formar un segmento con la longitud adecuada. Una vez procesado dicho segmento, sus resultados deberán devolverse de nuevo siguiendo el orden temporal de la señal. Los mecanismos concretos utilizados para recibir, almacenar y modificar estos bloques se explicarán más adelante, una vez presentadas las herramientas empleadas para desarrollar el complemento.

En definitiva, una señal multicanal está formada por varias secuencias de muestras que comparten la misma referencia temporal. Durante la reproducción, estas señales se entregan al complemento mediante bloques consecutivos almacenados temporalmente en un búfer. Esta organización permite comprender cómo circula el audio por el sistema y establece la base necesaria para estudiar posteriormente su procesamiento y la separación de las distintas fuentes.

3.2. El problema del bleeding y la separación de fuentes

En el apartado anterior se ha explicado cómo se representa una señal de audio digital y cómo pueden organizarse varios canales dentro de una misma grabación. Sin embargo, disponer de un canal para cada micrófono no significa que las fuentes sonoras se encuentren completamente aisladas. Cuando varios instrumentos se graban simultáneamente en un

mismo espacio, cada micrófono capta principalmente la fuente situada más cerca, pero también puede recoger parte del sonido producido por los demás instrumentos. Esta captación no deseada origina el fenómeno conocido como bleeding. A continuación, se explica por qué aparece este problema y qué consecuencias tiene durante el tratamiento de una grabación. Posteriormente, se presenta la separación de fuentes y, de manera más específica, el de-bleeding como procedimientos destinados a aumentar el grado de aislamiento entre los canales.

3.2.1. Bleeding en grabaciones multicanal

En las grabaciones musicales en directo es habitual que varios intérpretes toquen al mismo tiempo dentro de una misma sala. Para disponer de un mayor control sobre cada instrumento, se puede colocar un micrófono cerca de cada una de las fuentes sonoras. Esta técnica se conoce como microfonía cercana o close miking [4], [5].

La proximidad permite que el sonido del instrumento principal llegue al micrófono con mayor intensidad que el procedente de otras fuentes. También ayuda a reducir el efecto de la reverberación de la sala. No obstante, aunque se utilicen micrófonos direccionales y se estudie cuidadosamente su colocación, los canales obtenidos no quedan completamente aislados [3]. El sonido se propaga por el espacio en diferentes direcciones. Por este motivo, una parte de la energía producida por un instrumento puede alcanzar también los micrófonos destinados a los demás. Esta captación recibe diferentes nombres, como fuga microfónica, microphone leakage, spill o bleeding. En este trabajo se utilizará principalmente el término bleeding [3].

La cantidad de señal no deseada presente en cada canal depende de varios factores. Entre ellos se encuentran la distancia entre los instrumentos y los micrófonos, la orientación y directividad de estos, la intensidad relativa de cada fuente y las características acústicas de la sala. Las reflexiones producidas por paredes, suelo u otros elementos también pueden hacer que el sonido llegue al micrófono desde direcciones diferentes [4], [5].

Como se indicó anteriormente, no debe confundirse una fuente sonora con un canal de audio. La fuente es el sonido producido originalmente por un instrumento, mientras que el canal es la señal finalmente registrada por un micrófono. Debido al bleeding, un canal puede contener varias fuentes, aunque normalmente una de ellas presenta un nivel superior al resto.

En el sistema utilizado como punto de partida para este proyecto se dispone de hasta doce canales microfónicos. Cada uno se encuentra asociado principalmente a una determinada fuente, pero también puede contener contribuciones de los demás instrumentos

presentes durante la grabación [9]. La composición de uno de estos canales puede expresarse de forma simplificada mediante la ecuación:

$$x_c[n] = s_c[n] + b_c[n] \quad (2)$$

donde $x_c[n]$ representa la muestra n del canal c , $s_c[n]$ corresponde a la contribución de la fuente principal asociada a ese canal y $b_c[n]$ agrupa las contribuciones no deseadas procedentes de las demás fuentes.

El bleeding no debe entenderse como un ruido aleatorio añadido a la grabación. El contenido no deseado procede de los propios instrumentos y coincide en el tiempo con las señales que se quieren conservar. Por tanto, puede compartir con ellas una parte importante de su contenido frecuencial, lo que dificulta su eliminación mediante filtros sencillos.

Durante la grabación pueden utilizarse diferentes medidas para reducirlo, como acercar los micrófonos a sus fuentes, orientar sus zonas de menor sensibilidad hacia los otros instrumentos o colocar barreras absorbentes. También pueden emplearse puertas de ruido para atenuar un canal cuando no está disponible fuente principal [3]. Estas soluciones permiten disminuir parcialmente el problema, pero presentan limitaciones cuando varios instrumentos suenan al mismo tiempo. Por tanto, cuando las medidas aplicadas durante la grabación no proporcionan un aislamiento suficiente, es necesario recurrir a procedimientos capaces de analizar conjuntamente las señales y estimar qué parte de cada canal pertenece a su fuente principal.

3.2.2. Separación de fuentes y de-bleeding

La separación de fuentes de audio reúne un conjunto de técnicas cuyo objetivo es recuperar o estimar las señales que han intervenido en una grabación a partir de una o varias señales en las que aparecen combinadas [6]. Se utiliza el término “estimar” porque, durante el procesamiento, el sistema no dispone directamente de las fuentes originales y completamente aisladas. A partir de la información contenida en las señales observadas, debe generar nuevas señales que se aproximen lo máximo posible a esas fuentes.

Una situación habitual de separación musical consiste en partir de una mezcla mono o estéreo en la que aparecen conjuntamente la voz, la batería, el bajo y otros instrumentos. El sistema intenta descomponer esa mezcla y obtener una señal diferente para cada uno de sus componentes principales [6].

En una grabación multicanal se dispone de varias observaciones del mismo escenario sonoro. Cada micrófono recoge una combinación diferente de las fuentes, ya que sus niveles dependen de la posición, la distancia y la orientación. Estas diferencias proporcionan información adicional que puede aprovecharse durante la separación.

El debleeding constituye una aplicación más específica de la separación de fuentes. En este caso, no se parte únicamente de una mezcla convencional, sino de varias pistas obtenidas mediante micrófonos próximos. Cada canal ya contiene una fuente predominante, pero también incluye fugas procedentes de los demás instrumentos [3]. Por tanto, el objetivo no consiste en identificar desde cero todas las fuentes presentes en una única mezcla. Se trata de utilizar la información conjunta de los canales para conservar la fuente principal de cada uno y reducir las contribuciones no deseadas. El resultado buscado es una versión con mayor aislamiento de las señales originalmente grabadas.

En el caso de este proyecto, el sistema recibe C canales afectados por bleeding $x_c[n]$ y debe estimar C señales de salida $\hat{s}_c[n]$, donde cada salida debe mantener principalmente el instrumento asociado a su canal y atenuar, en la medida de lo posible, el contenido procedente de los otros micrófonos [9].

El hecho de disponer de una fuente predominante en cada canal constituye una información útil. Si un instrumento aparece con mayor intensidad en el micrófono situado junto a él, el sistema puede aprovechar esta relación para distinguirlo de las contribuciones presentes a menor nivel. También puede utilizar las diferencias existentes entre los demás canales para reconocer qué partes de la señal pertenecen a cada fuente [3].

No obstante, la separación continúa siendo una tarea compleja. Varios instrumentos pueden sonar simultáneamente, ocupar regiones frecuenciales semejantes o presentar características sonoras parecidas. La reverberación de la sala y las diferencias temporales entre los micrófonos dificultan aún más la identificación de cada contribución [6]. Además, eliminar una cantidad excesiva de contenido puede dañar la fuente que se pretende conservar. Una separación demasiado intensa puede introducir alteraciones o artefactos audibles, mientras que una separación insuficiente deja parte del bleeding original. Por ello, el resultado debe mantener un equilibrio entre la reducción de las interferencias y la conservación de la calidad sonora.

En definitiva, la separación de fuentes busca estimar las señales individuales presentes en una grabación, mientras que el debleeding aplica este principio a pistas obtenidas mediante micrófonos próximos. En este contexto, cada canal contiene una fuente predominante y varias contribuciones no deseadas. El sistema debe reducir estas últimas sin alterar de forma apreciable el instrumento que se desea conservar.

3.3. Separación de fuentes mediante aprendizaje profundo

Como se ha explicado en el apartado anterior, la separación de fuentes busca estimar las señales individuales presentes en una grabación. En el caso del de-bleeding, el objetivo consiste en conservar la fuente predominante de cada canal y reducir el sonido procedente de los demás instrumentos. Debido a la dificultad de distinguir estas contribuciones mediante filtros convencionales, durante los últimos años se han desarrollado métodos basados en aprendizaje profundo [6].

Estos métodos emplean modelos capaces de aprender a partir de ejemplos. Durante la fase de entrenamiento, el modelo recibe grabaciones afectadas por bleeding junto con señales de referencia que presentan un mayor grado de aislamiento. A partir de estos datos, ajusta sus parámetros internos para aprender la relación existente entre los canales de entrada y las fuentes que se desean obtener a la salida.

Una vez finalizado el entrenamiento comienza la fase denominada inferencia. En ella, el modelo aplica lo aprendido sobre grabaciones que no ha utilizado anteriormente y genera una estimación de sus fuentes. Por tanto, durante la inferencia el sistema no vuelve a entrenarse ni modifica continuamente sus parámetros, sino que utiliza una configuración ya aprendida para procesar nuevos fragmentos de audio.

Esta distinción resulta especialmente importante para delimitar el alcance del presente proyecto. El modelo utilizado fue desarrollado y entrenado previamente para trabajar con grabaciones multicanal formadas por doce canales de entrada y doce señales de salida [9]. En consecuencia, este trabajo no aborda la creación del conjunto de datos ni un nuevo proceso de entrenamiento, sino la preparación y utilización del modelo ya disponible, así como su posterior integración y ejecución dentro de un complemento de audio.

Los modelos de separación pueden trabajar con el sonido de dos formas principales. Algunos procesan directamente la señal en el dominio del tiempo. Otros convierten esa señal en una representación tiempo-frecuencia, que permite observar qué frecuencias contiene el audio y cómo varían durante la grabación. Cada forma aporta información diferente. La señal temporal conserva la forma de onda original, mientras que la representación tiempo-frecuencia ayuda a distinguir con mayor claridad los distintos componentes del sonido. Por este motivo, algunos modelos combinan ambas formas de procesamiento.

Este es el enfoque seguido por la arquitectura Hybrid Demucs utilizada como base en el presente proyecto. Su funcionamiento general se apoya en el tratamiento conjunto de información temporal y tiempo-frecuencia. No obstante, la estructura interna del modelo, sus diferentes etapas y su adaptación al procesamiento de doce canales se explicarán de forma específica en el siguiente apartado.

3.4. Arquitectura Hybrid Demucs

En el apartado anterior se ha indicado que algunos modelos de separación combinan el análisis directo de la señal de audio con una representación tiempo-frecuencia. Hybrid Demucs sigue este planteamiento y amplía la arquitectura Demucs original mediante dos formas complementarias de procesar el sonido.

Para comprender su funcionamiento, primero se presenta la estructura básica de Demucs. A continuación, se explica cómo Hybrid Demucs incorpora el procesamiento tiempo-frecuencia y combina la información obtenida por ambas ramas. Finalmente, se describe cómo se organizan las entradas y salidas del modelo utilizado en este proyecto y por qué el audio debe procesarse mediante segmentos de duración limitada.

3.4.1. Fundamentos de Demucs

Demucs es una arquitectura de aprendizaje profundo desarrollada para separar las fuentes presentes en una grabación musical. El modelo original trabaja directamente con la forma de onda: recibe las muestras que componen el audio y genera nuevas señales correspondientes a las fuentes que ha estimado [7].

Su funcionamiento puede entenderse como un proceso formado por dos partes principales. La primera analiza el audio de entrada y extrae la información necesaria para distinguir las fuentes. La segunda utiliza esa información para reconstruir por separado las señales de salida. Estas dos partes reciben los nombres de codificador y decodificador, respectivamente.

El codificador procesa el audio a través de varias etapas. Durante este recorrido, la señal original se transforma poco a poco en una representación interna que recoge sus características más importantes. De este modo, el modelo puede identificar patrones relacionados con la evolución temporal, el ritmo o los cambios de intensidad de los sonidos presentes en la grabación.

Al finalizar el codificador todavía no se han obtenido las fuentes separadas. El resultado es una representación más compacta del fragmento de audio, preparada para que el modelo pueda analizar las relaciones existentes entre sus diferentes partes. No se trata de una compresión destinada a reducir el tamaño del archivo, sino de una transformación interna que facilita el proceso de separación.

En la parte central de la arquitectura original se utilizan capas recurrentes bidireccionales. Su finalidad es relacionar la información obtenida en distintos instantes del fragmento. Esto permite que, al estimar el contenido correspondiente a un momento concreto,

el modelo también tenga en cuenta lo que sucede antes y después dentro del mismo segmento [7]. Este contexto resulta útil cuando varios instrumentos suenan al mismo tiempo y sus componentes se encuentran mezclados.

A continuación, el decodificador realiza el recorrido contrario. A partir de la información obtenida durante el análisis, reconstruye progresivamente una forma de onda para cada fuente que se desea separar. Las señales generadas conservan la duración del fragmento original, pero su contenido se distribuye entre las distintas salidas estimadas por el modelo.

Durante el análisis pueden perderse algunos detalles concretos de la señal. Para reducir este problema, Demucs incorpora conexiones directas entre las etapas correspondientes del codificador y del decodificador. Estas conexiones, conocidas como conexiones de salto, permiten recuperar información precisa del audio durante la reconstrucción. Resultan especialmente útiles para conservar cambios rápidos, como el inicio de una nota o un golpe de percusión.

La Figura 3 representa de forma simplificada el recorrido seguido por el audio dentro de Demucs. En ella se distinguen el análisis realizado por el codificador, el procesamiento central de la información y la posterior reconstrucción de las fuentes mediante el decodificador.

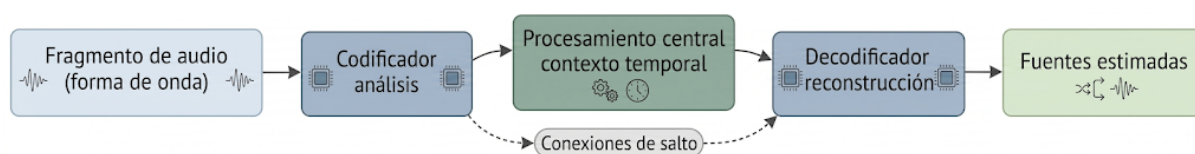


Figura 3 Esquema simplificado del funcionamiento de Demucs en el dominio temporal. Las conexiones de salto trasladan información entre las etapas correspondientes del codificador y del decodificador. Fuente: elaboración propia a partir de [7].

Por tanto, Demucs puede entenderse como un sistema que analiza un fragmento de audio, obtiene una representación interna de su contenido y, a partir de ella, reconstruye las fuentes por separado. Esta estructura constituye la base de Hybrid Demucs, que añade una segunda forma de analizar la señal mediante una representación tiempo-frecuencia.

3.4.2. Procesamiento híbrido

El Demucs original analiza el audio directamente a partir de su forma de onda. Hybrid Demucs amplía este funcionamiento incorporando una segunda forma de estudiar la misma señal. De ahí procede el término híbrido: el fragmento de audio se procesa mediante dos recorridos paralelos, denominados rama temporal y rama espectral [8].

La rama temporal trabaja directamente con las muestras del audio. Su funcionamiento sigue la estructura explicada en el apartado anterior. Primero analiza la forma de onda mediante un codificador y, posteriormente, utiliza un decodificador para reconstruir las fuentes estimadas. Al trabajar directamente con las muestras, esta rama conserva la evolución de la señal a lo largo del tiempo.

La rama espectral recibe el mismo fragmento, pero representado de otra manera. Para obtener esta nueva representación se utiliza la transformada de Fourier de tiempo corto (STFT). Esta operación divide el audio en pequeñas porciones consecutivas y determina qué frecuencias aparecen en cada una de ellas. El resultado se denomina espectrograma y permite observar cómo cambia el contenido frecuencial del sonido a lo largo del tiempo. El espectrograma no corresponde a un audio diferente, sino a otra forma de organizar la información de la misma señal. Mientras que la forma de onda muestra cómo varía directamente el audio con el tiempo, el espectrograma permite distinguir con mayor claridad las frecuencias que lo componen.

Una vez obtenido el espectrograma, la rama espectral también utiliza un codificador y un decodificador. Esta rama analiza cómo se distribuye el sonido entre las distintas frecuencias. Esta información puede resultar útil para diferenciar instrumentos que suenan al mismo tiempo, pero que presentan características frecuenciales diferentes.

Las dos ramas no permanecen separadas durante todo el proceso. Después de realizar un primer análisis, la información obtenida por ambas se combina en una parte común de la arquitectura. De esta forma, el modelo puede relacionar los cambios observados en la forma de onda con las frecuencias que aparecen en esos mismos instantes [8].

Tras esta combinación, cada rama reconstruye su propia estimación. La rama temporal genera directamente una forma de onda. En cambio, la rama espectral obtiene un espectrograma estimado que debe transformarse nuevamente en una señal de audio. Para ello se utiliza la transformada inversa de Fourier de tiempo corto (ISTFT), que realiza el proceso contrario a la STFT.

Finalmente, las formas de onda obtenidas mediante las dos ramas se combinan para generar la estimación final de cada fuente. De este modo, el resultado puede aprovechar tanto la información temporal de la señal como la distribución de su contenido frecuencial.

Este funcionamiento se resume en la Figura 4, donde se representa el recorrido del mismo fragmento de audio a través de la rama temporal y la rama espectral, así como la combinación de los resultados obtenidos por ambas.

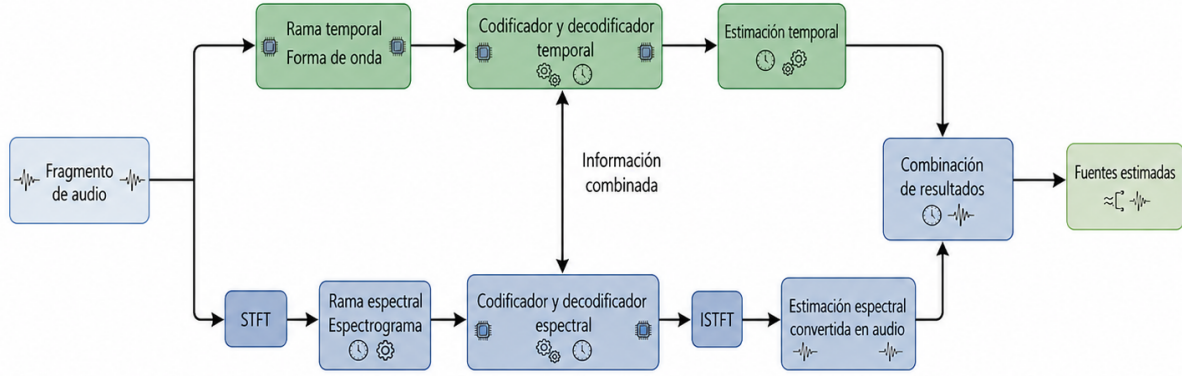


Figura 4 Esquema simplificado del procesamiento realizado por Hybrid Demucs mediante una rama temporal y una rama espectral. Fuente: elaboración propia a partir de [8].

Por tanto, Hybrid Demucs mantiene la estructura básica de Demucs, pero añade un segundo recorrido que analiza el audio mediante un espectrograma. La combinación de ambas representaciones permite utilizar información complementaria durante la separación. En el siguiente subapartado se explican la organización de las entradas y salidas del modelo y el procesamiento del audio mediante segmentos.

3.4.3. Modelo heredado: entradas, salidas y procesamiento por segmentos

El modelo empleado en este proyecto fue adaptado previamente para trabajar con un máximo de doce canales de entrada [9]. Sin embargo, esto no significa que todas las grabaciones deban contener necesariamente doce señales reales. El sistema también puede utilizarse cuando existe un número menor de canales, siempre que la entrada se prepare con las dimensiones esperadas por el modelo.

Los canales disponibles deben encontrarse sincronizados y tener la misma frecuencia de muestreo y el mismo número de muestras. De esta manera, las posiciones equivalentes de todas las señales representan el mismo instante de la grabación y pueden analizarse de forma conjunta.

Las señales disponibles pueden representarse inicialmente mediante un tensor de dimensiones $[1, C, M]$. El primer valor indica que se procesa un único fragmento (o batch), C representa el número de canales reales disponibles y M corresponde al número de muestras de cada canal. En este caso, el número de canales C debe ser menor o igual que 12.

El modelo siempre espera recibir una entrada con doce canales. Por ello, antes de realizar el procesamiento se construye un tensor de dimensiones $[1, 12, M]$. Cuando se utilizan menos de doce canales reales, las posiciones que no contienen ninguna señal se rellenan con ceros. Estos valores representan ausencia de audio y permiten mantener el tamaño de

entrada requerido por el modelo sin añadir información artificial. Por ejemplo, si se utilizan ocho canales, las posiciones correspondientes contienen sus muestras, mientras que las cuatro posiciones restantes se completan con ceros.

Cada salida representa la estimación de la fuente cercana en el canal correspondiente. Cuando se trabaja con menos canales reales, posteriormente pueden seleccionarse las salidas necesarias para la configuración utilizada. El objetivo es conservar con mayor claridad la fuente predominante y reducir la presencia del sonido procedente del resto de instrumentos. Por tanto, la salida no debe interpretarse necesariamente como una señal completamente aislada, sino como una estimación con un menor nivel de bleeding.

Las señales obtenidas conservan la duración y la posición temporal del fragmento introducido. Esto permite situarlas nuevamente en el punto correspondiente de la grabación y formar una salida continua. Sin embargo, una grabación completa puede contener una cantidad de muestras demasiado elevada para procesarla de una sola vez. Además, en un complemento de audio no es posible esperar a que termine toda la grabación antes de comenzar a generar resultados. Por este motivo, el audio se divide en fragmentos de duración limitada, denominados segmentos o ventanas. Cada segmento contiene el mismo intervalo temporal de todos los canales disponibles. Una vez estimadas las salidas, estas deben colocarse en el orden adecuado para reconstruir la grabación.

El tamaño de los segmentos influye en el funcionamiento del sistema. Una ventana corta necesita menos memoria y puede reducir el tiempo de espera, pero proporciona al modelo menos contexto sonoro. En cambio, una ventana más larga contiene más información sobre la evolución del audio, aunque también requiere más capacidad de cálculo y aumenta el tiempo necesario para obtener el resultado. La elección de este tamaño debe buscar, por tanto, un equilibrio entre el contexto disponible y el coste computacional.

Si los segmentos se procesan y se unen directamente, pueden aparecer cambios bruscos en sus límites. Para reducir este problema se puede utilizar solapamiento, haciendo que una parte de cada segmento coincida con el siguiente. Las estimaciones obtenidas en la zona compartida se combinan de manera gradual, lo que ayuda a evitar discontinuidades perceptibles entre fragmentos.

En este contexto, no debe confundirse el segmento utilizado por el modelo con el bloque de audio entregado por el programa anfitrión. Un DAW proporciona al complemento bloques normalmente más pequeños, cuyo tamaño depende de la configuración del sistema. Por ello, el complemento debe almacenar varios bloques consecutivos hasta reunir las muestras necesarias para formar la ventana de entrada del modelo. Después, las señales estimadas deben devolverse progresivamente al anfitrión. Este proceso y los tamaños finalmente utilizados se detallarán en el capítulo dedicado al desarrollo e integración del complemento.

3.5. Procesamiento de audio en tiempo real

El procesamiento de audio en tiempo real permite aplicar una operación mientras la señal se reproduce o se graba, sin necesidad de disponer previamente del archivo completo. Para hacerlo posible, el programa anfitrión entrega el audio al complemento de forma progresiva y recibe las muestras procesadas siguiendo ese mismo recorrido.

Este funcionamiento requiere organizar el audio en bloques y completar las operaciones dentro del tiempo disponible. También es necesario mantener la continuidad entre los resultados generados. En los siguientes subapartados se presentan brevemente estos conceptos y se aclara la diferencia entre los bloques utilizados por el anfitrión y los segmentos que necesita el modelo.

3.5.1. *Procesamiento mediante bloques*

Aunque el audio se percibe como una señal continua, un sistema digital lo procesa mediante grupos de muestras consecutivas denominados bloques. Durante la reproducción, el DAW reúne una cantidad determinada de muestras de los canales activos y las entrega al complemento para que pueda operar sobre ellas.

En JUCE, esta comunicación se realiza principalmente mediante la función *processBlock()*. El anfitrión llama repetidamente a esta función y proporciona un búfer que contiene el siguiente bloque de entrada. El complemento puede modificar sus muestras o almacenarlas para realizar un procesamiento posterior. Después, el resultado se devuelve al anfitrión y el mismo ciclo continúa con los bloques siguientes [15], [16].

El tamaño habitual del bloque se establece desde el DAW o desde la configuración del dispositivo de audio. Sin embargo, el complemento no debe asumir que todos los bloques recibidos tendrán siempre el mismo tamaño, ya que el número de muestras puede variar entre llamadas [15], [16].

Por tanto, el bloque puede entenderse como la unidad utilizada para intercambiar el audio entre el anfitrión y el complemento. Esto no significa que cada bloque deba introducirse directamente en el modelo, puesto que este puede necesitar reunir una cantidad diferente de muestras. Esta distinción se aclarará en el apartado 3.5.3.

3.5.2. Latencia, continuidad y plazo de procesamiento

En un sistema de audio en tiempo real, el resultado no se obtiene de manera completamente instantánea. El intervalo que transcurre desde que las muestras entran en el sistema hasta que pueden devolverse procesadas se denomina latencia. Este retraso puede estar provocado por el almacenamiento temporal del audio y por el tiempo necesario para realizar las operaciones. Por tanto, trabajar en tiempo real no significa eliminar totalmente la latencia, sino mantenerla dentro de unos límites controlados.

Además del retraso introducido, debe garantizarse la continuidad del audio. Los bloques procesados tienen que devolverse en el orden correcto y siguiendo el ritmo de reproducción. Si el resultado no está disponible cuando se necesita, pueden producirse pequeños cortes, chasquidos u otras interrupciones perceptibles.

Si el tiempo de cálculo supera repetidamente la duración del bloque, el sistema puede dejar de mantener un flujo de audio estable y producir interrupciones perceptibles [15], [16].

No debe confundirse este plazo con la latencia total. La latencia indica cuánto se retrasa la salida respecto a la entrada, mientras que el plazo de procesamiento representa el tiempo disponible para generar los resultados sin interrumpir la reproducción. Por ello, un sistema puede introducir un retraso fijo y, aun así, mantener un audio continuo. Cuando este retraso es conocido, el complemento puede comunicarlo al anfitrión para que lo tenga en cuenta [15], [16].

Esta gestión temporal adquiere especial importancia cuando se utilizan modelos de aprendizaje profundo, debido a que su coste computacional suele ser mayor que el de los efectos de audio convencionales. Las medidas adoptadas para organizar este procesamiento y controlar la latencia se explicarán posteriormente en el capítulo dedicado al desarrollo del complemento.

3.5.3. Diferencia entre bloques y segmentos

Aunque los bloques y los segmentos están formados por conjuntos de muestras consecutivas, cumplen funciones diferentes. Como se explicó en el apartado 3.5.1, el bloque es la unidad utilizada por el DAW para intercambiar audio con el complemento [15], [16]. En cambio, el segmento es la cantidad de audio preparada internamente para realizar una ejecución del modelo, tal como se introdujo en el apartado 3.4.3 [9].

La Tabla 2 resume las principales diferencias entre los bloques entregados por el anfitrión y los segmentos utilizados por el modelo.

Aspecto	Bloque de audio	Segmento del modelo
Finalidad	Intercambiar audio entre el DAW y el complemento.	Proporcionar al modelo una cantidad de audio adecuada para realizar el procesamiento.
Origen	Es entregado periódicamente por el anfitrión.	Se forma dentro del complemento a partir de las muestras recibidas.
Tamaño	Depende de la configuración del anfitrión y puede variar entre llamadas.	Se establece según las necesidades del modelo y del sistema de procesamiento.
Tratamiento	Se recibe mediante llamadas sucesivas a <i>processBlock()</i> .	Se utiliza como entrada para una ejecución del modelo.
Solapamiento	Los bloques se reciben de forma consecutiva.	Los segmentos pueden solaparse para suavizar la unión entre sus resultados.
Relación entre ambos	Sus muestras pueden almacenarse hasta reunir una cantidad suficiente.	Puede estar formado por las muestras procedentes de varios bloques.

Tabla 2 Diferencias principales entre los bloques entregados por el anfitrión y los segmentos utilizados por el modelo. Fuente: elaboración propia.

Por tanto, ambos conceptos forman parte de etapas distintas del procesamiento. El complemento recibe el audio en bloques y puede almacenarlo temporalmente hasta construir un segmento adecuado para el modelo. Una vez procesado, el resultado se conserva y se devuelve progresivamente al anfitrión siguiendo nuevamente la organización en bloques. Esta relación permite adaptar el flujo continuo del DAW a la cantidad de muestras necesaria para ejecutar el modelo.

3.6. Complementos de audio y formato VST3

Para utilizar el procesamiento descrito anteriormente dentro de un entorno habitual de producción musical, es necesario integrarlo de manera que pueda recibir y devolver audio durante la ejecución de un proyecto. Los complementos de audio permiten incorporar una función específica a un programa anfitrión sin tener que desarrollar un entorno completo de grabación, reproducción y mezcla.

En este apartado se presenta la relación entre los complementos y las estaciones de trabajo de audio digital, se describen las características básicas del formato VST3 y se introduce el papel que desempeñan JUCE y REAPER en el desarrollo del proyecto.

3.6.1. DAW y complementos de audio

Una estación de trabajo de audio digital, conocida habitualmente por las siglas DAW (Digital Audio Workstation), es una aplicación destinada a la grabación, organización, edición, reproducción y mezcla de audio. El usuario puede distribuir las señales en diferentes pistas, establecer su recorrido, ajustar sus niveles y aplicar distintos procesos dentro de un mismo proyecto. REAPER es una de las distintas DAW disponibles actualmente. Existen otros programas destinados al trabajo con audio, como Pro Tools, Logic Pro, Cubase, Ableton Live o Studio One. Aunque cada uno presenta su propio entorno y ofrece distintas posibilidades de configuración, todos permiten organizar proyectos de audio, trabajar con pistas y aplicar diferentes operaciones sobre las señales.

Además de utilizar complementos, muchas DAW permiten automatizar tareas mediante scripts. Un script es una secuencia de instrucciones que puede ser ejecutada o interpretada por el propio programa para realizar operaciones de forma automática. A diferencia de un complemento, normalmente no se inserta en la cadena de procesamiento del audio, sino que utiliza las funciones proporcionadas por la DAW para ejecutar acciones como crear pistas, modificar sus propiedades, organizar elementos o configurar conexiones. En REAPER, este sistema de automatización recibe el nombre de ReaScript. En este proyecto se utiliza posteriormente para preparar de manera automática las pistas y el enrutamiento multicanal necesarios para trabajar con las salidas del complemento.

Además de sus funciones propias, una DAW puede ampliar sus posibilidades mediante complementos de audio o plugins. Desde un punto de vista técnico, un complemento es un módulo de software compilado que normalmente se distribuye como una biblioteca dinámica o como un paquete que contiene el código binario necesario para su ejecución. Este módulo no funciona de manera independiente, sino que debe ser cargado por un programa anfitrión compatible.

Para que el anfitrión pueda reconocerlo y utilizarlo, el complemento implementa una interfaz estandarizada definida por el formato correspondiente, como VST3. Esta interfaz está formada por un conjunto de funciones, estructuras de datos y argumentos conocidos por el anfitrión. Gracias a ella, la DAW puede identificar el complemento, crear una instancia y solicitar las operaciones necesarias sin conocer su implementación interna. Algunos

complementos generan sonido, como los instrumentos virtuales, mientras que otros modifican señales existentes mediante ecualización, compresión, reverberación u otras operaciones.

Cuando se carga un complemento, el DAW actúa como anfitrión. Se encarga de insertarlo en una pista o bus, proporcionarle las señales de entrada y recoger el resultado generado. También transmite información necesaria para el procesamiento, como la frecuencia de muestreo, la configuración de canales y el estado de reproducción [15], [16]. Como se explicó en el apartado 3.5.1, el intercambio del audio se realiza de forma progresiva mediante bloques de muestras.

El complemento, por su parte, se ocupa únicamente del procesamiento para el que ha sido diseñado. Recibe las señales procedentes del anfitrión, aplica las operaciones correspondientes y devuelve el audio resultante. Esta separación permite que el DAW gestione el proyecto y su encaminamiento, mientras que el complemento realiza una función especializada.

En este proyecto, el sistema desarrollado se comporta como un complemento de procesamiento, ya que no genera audio por sí mismo, sino que actúa sobre señales multicanal previamente existentes. Su finalidad es aplicar el modelo de separación para obtener estimaciones con una menor presencia del sonido procedente de otras fuentes. Para que el DAW pueda reconocerlo, cargarlo y comunicarse con él, debe utilizarse un formato de complemento compatible, función que en este caso desempeña VST3.

3.6.2. Formato VST3

VST, siglas de Virtual Studio Technology, es una interfaz para complementos de audio desarrollada por Steinberg. Su tercera versión, denominada VST3, establece un conjunto común de reglas que permite a un DAW compatible reconocer, cargar y comunicarse con un complemento [17]. De esta forma, el complemento puede utilizarse en diferentes programas anfitriones compatibles sin necesidad de crear un mecanismo de comunicación específico para cada uno.

VST3 no determina las operaciones que deben aplicarse al audio, sino la manera en la que el anfitrión y el complemento intercambian la información. Durante la ejecución, el DAW entrega las señales de entrada y los datos necesarios para el procesamiento, mientras que el complemento devuelve el resultado. El formato también proporciona mecanismos para definir los buses y canales de audio, gestionar parámetros y automatizaciones, conservar el estado del complemento y comunicar la latencia introducida [17].

Una característica especialmente relevante para este proyecto es la posibilidad de utilizar configuraciones de entrada y salida con varios canales. Esto permite desarrollar

complementos que no se limiten a las disposiciones habituales mono o estéreo. No obstante, las configuraciones realmente disponibles dependen de los canales admitidos por el propio complemento y de que el anfitrión pueda utilizarlos. Por tanto, VST3 proporciona el mecanismo para gestionar estas configuraciones, pero no establece por sí solo el número de canales [17].

En su organización interna, VST3 diferencia la parte encargada del procesamiento de audio de la relacionada con el control de parámetros y la interfaz gráfica. Esta separación ayuda a mantener las tareas que afectan directamente al flujo de audio diferenciadas de las acciones realizadas desde la interfaz del usuario [17].

En este proyecto, el formato actúa como capa de comunicación entre el DAW y el sistema interno, pero no realiza por sí mismo la separación de fuentes. Esta tarea corresponde al código desarrollado con JUCE y a la ejecución del modelo mediante LibTorch, elementos que se explicarán en los capítulos posteriores.

3.6.3. JUCE y REAPER

JUCE es un entorno de desarrollo de código abierto basado en C++ y orientado, entre otras aplicaciones, a la creación de complementos de audio. Proporciona una estructura común para recibir y procesar las señales, definir la configuración de canales, gestionar parámetros y desarrollar la interfaz gráfica [15], [16].

Una de sus principales funciones consiste en simplificar la adaptación del código a los distintos sistemas operativos y formatos de complemento. De esta manera, el desarrollador puede centrarse en las operaciones específicas de su aplicación sin tener que implementar directamente todos los mecanismos de comunicación establecidos por VST3. En este proyecto, JUCE proporciona la estructura sobre la que se integra el procesamiento de audio y la ejecución del modelo mediante LibTorch.

REAPER, por su parte, es una estación de trabajo de audio digital (DAW) que permite grabar, editar, reproducir y procesar señales de audio. También admite complementos VST3 y ofrece un sistema flexible de encaminamiento multicanal, características necesarias para trabajar con las distintas señales de entrada y salida utilizadas en este proyecto [18].

Cuando el complemento se carga en REAPER, este actúa como programa anfitrión. Se encarga de localizarlo, proporcionar los bloques de audio y recoger las señales procesadas. Además, permite configurar los canales, insertar el complemento en una pista y comprobar su funcionamiento durante la reproducción.

Por tanto, JUCE y REAPER desempeñan funciones complementarias. JUCE se utiliza para construir la estructura interna del complemento, VST3 establece el formato mediante el

que se comunica con el exterior y REAPER constituye el entorno en el que se carga y ejecuta. Esta organización permite desarrollar el procesamiento de forma independiente y comprobar posteriormente su comportamiento dentro de un entorno real de producción de audio.

3.7. Ejecución de modelos de PyTorch desde C++

El modelo de separación utilizado en este proyecto fue implementado previamente en Python mediante la librería PyTorch [9]. Sin embargo, el complemento VST3 se implementa en C++, por lo que el modelo no puede incorporarse directamente de la misma forma en la que se utiliza desde Python. Para ejecutarlo dentro del complemento es necesario disponer de una representación compatible con C++ y preparar los datos de audio con la organización esperada por el modelo.

En este apartado se presentan los elementos que permiten realizar esta integración. En primer lugar, se explica el papel de PyTorch y TorchScript; posteriormente, se introduce LibTorch como biblioteca para ejecutar el modelo desde C++; finalmente, se describe la relación entre los búferes utilizados para almacenar el audio y los tensores empleados por el modelo. Las operaciones concretas realizadas en el complemento se detallarán en el capítulo dedicado a su desarrollo.

3.7.1. PyTorch y TorchScript

PyTorch es un entorno de código abierto orientado al desarrollo de modelos de aprendizaje automático y aprendizaje profundo. Proporciona herramientas para definir redes neuronales, entrenarlas y utilizarlas posteriormente para procesar nuevos datos. La información se representa principalmente mediante unas estructuras de datos llamadas tensores, que permiten organizar los valores en una o varias dimensiones, realizar las operaciones matemáticas necesarias y mover los datos de forma sencilla entre CPU y GPU [19].

En este proyecto no se vuelve a entrenar el modelo, sino que se utiliza uno previamente entrenado para realizar inferencia. La inferencia consiste en aplicar los parámetros (o pesos) aprendidos por el modelo a nuevas señales de entrada para obtener las estimaciones de salida. Por tanto, durante este proceso los parámetros no se modifican, sino que se emplean para efectuar la separación del audio.

Un modelo desarrollado en PyTorch no está formado únicamente por sus parámetros. También incluye la organización de sus capas y la secuencia de operaciones que transforma la entrada en la salida. Para poder trasladar este comportamiento a un entorno en el que no se ejecute Python, puede utilizarse TorchScript.

TorchScript permite obtener una representación del modelo que puede guardarse y cargarse posteriormente desde otro proceso. Esta representación reúne las operaciones necesarias para ejecutar el modelo junto con sus parámetros previamente aprendidos. De este modo, el modelo puede utilizarse desde una aplicación desarrollada en C++ sin tener que iniciar un intérprete de Python [19].

La conversión a TorchScript puede realizarse principalmente mediante scripting o mediante tracing. El primer procedimiento analiza las operaciones definidas en el código compatible del modelo, mientras que el segundo registra el recorrido realizado al procesar una entrada de ejemplo. Ambos métodos permiten obtener un módulo ejecutable, aunque su adecuación depende de la estructura y del comportamiento del modelo. El procedimiento aplicado en este proyecto se explicará posteriormente junto con la exportación realizada.

Una vez exportado, el módulo TorchScript continúa recibiendo y generando tensores. La exportación no vuelve a entrenar el modelo ni modifica la finalidad para la que fue diseñado, sino que adapta su representación para poder ejecutarlo en otro entorno. En el caso de este proyecto, permite establecer el enlace entre el modelo Hybrid Demucs previamente entrenado y el complemento desarrollado en C++.

3.7.2. *LibTorch*

LibTorch es la distribución oficial de las bibliotecas de PyTorch preparada para su utilización desde C++. Incluye los archivos de cabecera, las bibliotecas compiladas y los elementos de configuración necesarios para incorporar sus funciones a una aplicación desarrollada en este lenguaje. De esta forma, el programa puede crear y manipular tensores, realizar operaciones matemáticas y ejecutar modelos previamente exportados sin depender de un intérprete de Python durante su funcionamiento [20].

Conviene diferenciar el papel de LibTorch del correspondiente a TorchScript. Como se explicó en el apartado anterior, TorchScript proporciona una representación del modelo que puede almacenarse y utilizarse fuera de Python. LibTorch, por su parte, aporta las funciones necesarias para cargar y ejecutar esa representación desde C++. Por tanto, ambos elementos son complementarios: el modelo se exporta mediante TorchScript y posteriormente se utiliza desde la aplicación mediante LibTorch [19], [20].

Para poder emplear LibTorch, el proyecto de C++ debe estar configurado para localizar sus archivos de cabecera y enlazar las bibliotecas necesarias. Esta configuración no modifica el modelo, sino que permite que el código desarrollado pueda acceder a las funciones proporcionadas por PyTorch. El procedimiento concreto seguido para integrar estas bibliotecas en el proyecto de Xcode se detallará posteriormente.

Una vez cargado el módulo exportado, la aplicación puede proporcionarle uno o varios tensores de entrada y solicitar la ejecución de su recorrido de procesamiento. LibTorch realiza las operaciones almacenadas en el modelo y devuelve los resultados nuevamente en forma de tensores [20]. Este proceso corresponde a la fase de inferencia descrita anteriormente, por lo que se utilizan los parámetros aprendidos durante el entrenamiento sin modificarlos.

En este proyecto, LibTorch actúa como enlace entre el complemento implementado en C++ y el modelo Hybrid Demucs previamente entrenado. JUCE se encarga de recibir y organizar el audio procedente del DAW, mientras que LibTorch permite cargar el modelo y ejecutar sus operaciones. Sin embargo, el audio almacenado por JUCE no puede entregarse directamente al modelo, ya que primero debe transformarse en tensores con el tipo, las dimensiones y la disposición adecuadas. Esta conversión se introduce en el siguiente apartado.

3.7.3. Intercambio de datos entre búferes y tensores

Durante el procesamiento, las muestras de audio deben pasar entre dos formas de representación. Dentro del complemento, JUCE organiza las señales mediante búferes de audio, mientras que el modelo ejecutado mediante LibTorch utiliza tensores [15], [16], [20]. Aunque ambas estructuras almacenan valores numéricos, están diseñadas para funciones diferentes.

El búfer contiene las muestras proporcionadas por el DAW y las organiza según los canales disponibles. El tensor, en cambio, representa la entrada del modelo mediante varias dimensiones. Como se explicó en el apartado 3.4.3, estas dimensiones permiten diferenciar el segmento procesado, los canales que lo componen y las muestras correspondientes a cada uno.

Antes de ejecutar la inferencia, las muestras deben organizarse con la disposición que espera el modelo. Para ello, es necesario conservar el orden temporal del audio, mantener la correspondencia entre los canales y utilizar un tipo numérico compatible. También deben completarse las posiciones correspondientes cuando el número de señales reales sea inferior al número de canales esperado, siguiendo el procedimiento de relleno con ceros descrito

anteriormente. Esta preparación no realiza todavía la separación, sino que adapta la representación de los datos para que puedan introducirse correctamente en el modelo.

Una vez preparado el tensor de entrada, LibTorch lo entrega al módulo TorchScript y ejecuta las operaciones almacenadas en él [19], [20]. El resultado se obtiene nuevamente en forma de tensor, con las estimaciones generadas por el modelo. Estas deben interpretarse según la organización de su salida y transformarse de nuevo en muestras que puedan utilizarse dentro del complemento.

Finalmente, las muestras obtenidas se sitúan en los búferes de salida correspondientes, respetando la relación entre cada señal estimada y su canal. De este modo, JUCE puede devolver el audio procesado al DAW mediante el mismo mecanismo de intercambio por bloques descrito anteriormente.

La Figura 5 resume el recorrido seguido por las muestras durante este intercambio. El audio pasa desde los búferes de JUCE hasta el modelo ejecutado mediante LibTorch y, una vez procesado, se transforma nuevamente para ser devuelto al DAW.



Figura 5 Intercambio de datos entre los búferes de audio del complemento y los tensores utilizados por el modelo. Fuente: elaboración propia.

La conversión entre ambas representaciones constituye, por tanto, el enlace entre el flujo de audio del complemento y la ejecución del modelo. Una organización incorrecta de las dimensiones, de los canales o de las muestras podría provocar que las señales se intercambiaran, que se alterara su orden temporal o que el modelo no pudiera procesar la entrada. La implementación concreta de estas operaciones se detallará posteriormente en el capítulo dedicado al desarrollo del sistema.

4. ESTADO DEL ARTE

El presente capítulo analiza las principales soluciones desarrolladas para reducir el bleeding y separar fuentes de audio antes del inicio de este trabajo. La revisión abarca desde los procedimientos convencionales utilizados durante la grabación y la posproducción hasta los métodos basados en aprendizaje profundo.

A continuación, se estudian algunas de las herramientas disponibles para realizar estas tareas, prestando atención a su forma de funcionamiento, al tipo de señales que procesan y a su integración dentro del flujo de trabajo de una estación de trabajo de audio digital.

Finalmente, las alternativas revisadas se relacionan con el sistema basado en Hybrid Demucs del que parte este TFG. Este análisis permite identificar las limitaciones de su forma original de ejecución, justificar su integración directa en REAPER mediante un complemento VST3 y situar la necesidad concreta abordada por el presente trabajo.

4.1. Métodos convencionales para la prevención y reducción del bleeding

Antes de la aplicación de técnicas de separación de fuentes basadas en aprendizaje automático, la reducción del bleeding se abordaba principalmente mediante decisiones adoptadas durante la grabación y operaciones realizadas posteriormente durante la edición y la mezcla. Estos procedimientos continúan utilizándose en la producción musical, ya que permiten disminuir la captación no deseada sin requerir modelos previamente entrenados ni sistemas de elevada complejidad computacional.

No obstante, conviene diferenciar entre prevención, atenuación y separación. Las medidas aplicadas durante la grabación tratan de evitar que el sonido no deseado llegue al micrófono con un nivel elevado. Las herramientas de mezcla permiten atenuar determinadas regiones de la señal ya registrada. La separación, en cambio, pretende estimar de forma independiente las contribuciones que aparecen superpuestas dentro de un mismo canal. Los métodos convencionales resultan eficaces principalmente en las dos primeras tareas, pero presentan mayores dificultades para realizar la tercera.

4.1.1. Actuaciones durante la captación

La primera estrategia consiste en aumentar la relación entre la fuente principal y las fuentes interferentes antes de que la señal sea registrada. Para ello se recurre habitualmente a la microfonía cercana, situando cada micrófono próximo al instrumento que se desea captar. La reducción de la distancia aumenta el nivel relativo de la fuente principal y disminuye la influencia de otras fuentes y de la reverberación de la sala [4].

Sin embargo, acercar el micrófono no impide que este continúe recibiendo sonido por otros recorridos. La intensidad de los instrumentos, la distancia respecto a los demás micrófonos, las reflexiones de la sala y la distribución de los intérpretes determinan el nivel final del bleeding. Por esta razón, la microfonía cercana mejora el aislamiento relativo, pero no produce canales completamente independientes.

La elección de la zona de grabación y la orientación del micrófono también permiten aprovechar sus diferencias de sensibilidad según la dirección de llegada del sonido. La zona de máxima sensibilidad se dirige hacia la fuente principal, mientras que las zonas de menor captación se orientan, cuando es posible, hacia los instrumentos que generan mayor interferencia. Esta estrategia resulta especialmente útil con micrófonos cardioideos o con patrones más direccionales.

Su eficacia, no obstante, depende de las características reales del micrófono. El patrón polar puede variar con la frecuencia y el sonido reflejado puede alcanzar el micrófono desde direcciones distintas a la correspondiente al instrumento interferente. En consecuencia, orientar una zona de baja sensibilidad hacia una fuente puede reducir su componente directa, pero no necesariamente las reflexiones que llegan desde el resto de la sala.

Otra medida consiste en aumentar la separación física entre los intérpretes o utilizar elementos de aislamiento, como paneles absorbentes, pantallas acústicas o cabinas independientes. Estos elementos reducen la energía que se propaga directamente entre las fuentes y los micrófonos. Cuando las condiciones de la grabación lo permiten, el aislamiento mediante salas diferentes puede proporcionar una reducción considerable del bleeding.

Esta solución no siempre resulta aplicable. La separación física puede dificultar la comunicación entre los músicos, alterar la interpretación conjunta y modificar la acústica percibida durante la grabación. Además, los paneles no eliminan completamente la propagación por reflexión ni ofrecen el mismo grado de atenuación para todas las frecuencias, especialmente en las regiones graves, cuyas longitudes de onda son mayores.

4.1.2. Control de las relaciones de fase y tiempo

Cuando una fuente es registrada por varios micrófonos, las diferencias de distancia provocan pequeños retardos entre las versiones captadas. Al combinar las pistas, estos

retardos pueden producir refuerzos y cancelaciones dependientes de la frecuencia, dando lugar al filtrado en peine descrito en [5].

La regla 3:1 recomienda que la distancia entre dos micrófonos sea, al menos, aproximadamente tres veces la existente entre cada micrófono y su fuente principal [5]. Con ello se busca que la señal no deseada llegue con un nivel suficientemente inferior y que las interferencias producidas al mezclar los canales resulten menos perceptibles. Esta regla debe entenderse como una recomendación práctica y no como una condición capaz de garantizar el aislamiento.

En posproducción también puede modificarse el desplazamiento temporal o la polaridad de una pista para mejorar su relación de fase con las demás. Estas operaciones pueden corregir parcialmente determinados refuerzos o cancelaciones, pero no eliminan las contribuciones no deseadas. El bleeding de cada instrumento llega a través de recorridos acústicos diferentes, con distintos retardos, niveles y modificaciones espectrales. Por ello, un único desplazamiento temporal no puede alinear simultáneamente todas las contribuciones presentes en un canal.

4.1.3. Puertas de ruido y expansores

Una vez realizada la grabación, una de las soluciones más habituales consiste en utilizar puertas de ruido o expansores. Una puerta atenúa el canal cuando su nivel desciende por debajo de un umbral establecido. Si el instrumento principal no está tocando y el canal contiene únicamente sonido procedente de otros instrumentos, este mecanismo puede reducir de manera considerable la fuga presente durante esos intervalos.

El resultado depende de la configuración del umbral y de los tiempos de ataque, mantenimiento y liberación. Un umbral demasiado bajo deja pasar parte del contenido no deseado, mientras que uno excesivamente alto puede eliminar fragmentos débiles del instrumento principal. Del mismo modo, una apertura o un cierre demasiado rápidos pueden introducir transiciones poco naturales, mientras que unos tiempos largos mantienen la fuga durante más tiempo.

La principal limitación de esta técnica aparece cuando la fuente principal y la interferente suenan simultáneamente. En ese caso, el nivel conjunto supera el umbral, la puerta permanece abierta y todo el contenido del canal continúa pasando. La puerta puede reducir el bleeding en pausas o regiones de baja actividad, pero no distingue el origen de las muestras mientras varias fuentes se encuentran presentes al mismo tiempo.

4.1.4. Ecualización y filtrado

La ecualización permite atenuar regiones frecuenciales en las que predomina el instrumento interferente. También pueden utilizarse filtros paso alto, paso bajo o de banda limitada para eliminar componentes que quedan fuera de la región principal ocupada por la fuente que se desea conservar. Estos procedimientos son sencillos, presentan un coste computacional reducido y pueden aplicarse durante la reproducción mediante complementos convencionales.

Su efectividad depende de que exista una separación frecuencial suficiente entre los sonidos. En una grabación musical, varios instrumentos pueden compartir frecuencias fundamentales, armónicos y componentes transitorias. Una atenuación aplicada sobre una banda concreta actúa sobre todas las contribuciones presentes en ella, con independencia de su procedencia. Por tanto, al reducir el instrumento interferente también puede modificarse el timbre de la fuente principal.

Los filtros convencionales analizan cada canal principalmente a partir de su contenido frecuencial, pero no disponen necesariamente de información suficiente para determinar qué instrumento ha generado cada componente. Esta limitación es especialmente relevante en el sistema estudiado, donde los doce canales pueden contener a la vez contribuciones de varias fuentes.

4.1.5. Edición manual y tratamiento espectral

La edición manual permite silenciar o atenuar las regiones en las que el instrumento principal no está activo. Frente a una puerta de ruido, el técnico puede tomar decisiones específicas para cada fragmento, conservar colas naturales y evitar cortes bruscos. También puede aplicar fundidos para suavizar las transiciones entre las zonas conservadas y las atenuadas.

Las herramientas de edición espectral amplían este procedimiento al representar la señal en el dominio tiempo-frecuencia. De este modo, el usuario puede identificar visualmente determinados eventos y reducir regiones concretas sin modificar todo el intervalo temporal. Este tratamiento puede ofrecer buenos resultados ante sonidos aislados y claramente reconocibles, pero se vuelve más complejo cuando las fuentes coinciden tanto temporal como frecuencialmente.

Además, la edición manual requiere revisar individualmente las pistas y adoptar decisiones para cada evento. En una grabación multicanal extensa, este proceso puede suponer una carga de trabajo elevada y sus resultados dependen de la experiencia del

técnico. Por ello, resulta adecuado para correcciones localizadas, pero es menos práctico como procedimiento automático para tratar de forma continuada un conjunto de doce canales.

La Tabla 3 resume el alcance y las principales limitaciones de los métodos analizados.

Método	Etapas de aplicación	Principio utilizado	Limitación principal
Microfonía cercana	Grabación	Aumentar el nivel relativo de la fuente principal.	No impide la llegada directa y reflejada de otras fuentes.
Directividad y orientación	Grabación	Aprovechar las zonas de menor sensibilidad del micrófono.	La respuesta polar depende de la frecuencia y de la dirección de llegada.
Separación y barreras acústicas	Grabación	Reducir la propagación entre fuentes y micrófonos.	Puede afectar a la interpretación y ofrece aislamiento limitado, especialmente a bajas frecuencias.
Regla 3:1 y ajustes temporales	Grabación y mezcla	Reducir interferencias de fase entre canales.	Mitiga problemas de fase, pero no separa las fuentes captadas.
Puertas de ruido y expansores	Mezcla	Atenuar el canal en función de su nivel.	No distinguen las fuentes cuando suenan simultáneamente.
Ecualización y filtrado	Mezcla	Atenuar bandas frecuenciales determinadas.	También modifica la fuente principal cuando existe solapamiento espectral.
Edición manual o espectral	Postproducción	Seleccionar y corregir regiones concretas.	Requiere intervención, tiempo y fuentes suficientemente diferenciables.

Tabla 3 Comparación de los principales métodos convencionales empleados para prevenir o reducir el bleeding. Fuente: elaboración propia a partir de [1], [4] y [5].

En conjunto, los métodos convencionales permiten mejorar el aislamiento alcanzado durante la grabación y reducir la presencia de contenido no deseado en situaciones determinadas. Su principal ventaja reside en que son procedimientos conocidos, interpretables y, en muchos casos, poco exigentes desde el punto de vista computacional. Además, las medidas preventivas evitan que parte del problema llegue a registrarse, por lo que continúan siendo recomendables incluso cuando posteriormente se utilizan sistemas de separación más avanzados.

Sin embargo, estas técnicas no suelen disponer de un criterio capaz de identificar el origen de cada componente cuando varias fuentes aparecen superpuestas. Las puertas actúan según el nivel, los filtros según la frecuencia y la edición manual según la selección realizada por el usuario. Cuando los instrumentos coinciden en el tiempo y ocupan regiones espectrales semejantes, atenuar la contribución interferente sin alterar la fuente principal resulta especialmente difícil.

Esta limitación motivó el desarrollo de métodos capaces de utilizar modelos más complejos de las fuentes y analizar conjuntamente la información disponible en las grabaciones. Los primeros trabajos recurrieron a técnicas de factorización y separación estadística [3], mientras que los avances posteriores incorporaron modelos de aprendizaje profundo. La evolución de estos últimos y su aplicación a la separación musical se estudian en el apartado siguiente.

4.2. Evolución de los modelos de separación de fuentes mediante aprendizaje profundo

Como se ha visto en el apartado anterior, los métodos convencionales permiten reducir parte del bleeding, pero tienen dificultades cuando varios instrumentos suenan al mismo tiempo. En estos casos, una puerta de ruido no puede cerrar el canal y un filtro frecuencial también puede modificar el instrumento que se desea conservar.

El aprendizaje profundo permite abordar el problema de otra forma. En lugar de aplicar siempre un mismo umbral o una atenuación fija, se utilizan modelos entrenados con ejemplos de mezclas y fuentes separadas. A partir de ellos, el modelo aprende a reconocer determinados patrones del audio y utiliza esa información para estimar las señales de salida [6].

El proceso de entrenamiento y la diferencia entre entrenamiento e inferencia ya se explicaron en el capítulo 3. Por tanto, este apartado se centra en la evolución de los principales modelos de separación musical. Primero se presentan los métodos basados en espectrogramas, después los que trabajan directamente con la forma de onda y, finalmente, los modelos híbridos como Hybrid Demucs.

4.2.1. Métodos basados en espectrogramas

Los primeros modelos de aprendizaje profundo que consiguieron buenos resultados en separación musical trabajaban principalmente con espectrogramas. Esta representación,

explicada en el apartado 3.4.2, muestra cómo se distribuye el contenido frecuencial del audio a lo largo del tiempo.

A partir del espectrograma de una mezcla, la red neuronal puede determinar qué regiones pertenecen probablemente a cada fuente. Para ello, muchos modelos generan una máscara que modifica el contenido de cada zona del espectrograma. Por ejemplo, si en un determinado instante y frecuencia predomina la voz, la máscara correspondiente conserva esa región, mientras que las máscaras de los demás instrumentos la atenúan.

La principal ventaja de este enfoque es que facilita la identificación de componentes frecuenciales. Los armónicos de una voz, las frecuencias graves de un bajo o los ataques de una batería pueden presentar formas diferentes en el espectrograma. La red aprende a reconocer estas características a partir de los ejemplos utilizados durante el entrenamiento.

Sin embargo, la separación no siempre es sencilla. Dos instrumentos pueden ocupar una misma región frecuencial y sonar simultáneamente. En ese caso, el modelo debe decidir qué parte corresponde a cada fuente. Además, muchos de los primeros sistemas estimaban únicamente la magnitud del espectrograma y reutilizaban la fase de la mezcla original al reconstruir el audio. Esta solución simplificaba el procesamiento, pero podía producir alteraciones en las señales obtenidas.

Un modelo representativo de este enfoque es Open-Unmix [22]. Este sistema utiliza el espectrograma de la mezcla y una red recurrente bidireccional para analizar la evolución temporal del audio. Después de obtener las estimaciones, puede aplicar un filtrado de Wiener para distribuir de manera más precisa el contenido entre las fuentes.

Open-Unmix fue diseñado como una implementación abierta y reproducible. Sus modelos preentrenados permiten separar una canción en cuatro grupos principales: voz, batería, bajo y resto del acompañamiento. Esto facilitó su utilización como sistema de referencia para comparar nuevos métodos de separación musical [22].

No obstante, su finalidad es diferente a la del presente proyecto. Open-Unmix parte habitualmente de una mezcla estéreo y genera un conjunto de fuentes musicales predefinidas. En cambio, el sistema estudiado en este TFG recibe varias señales de micrófonos cercanos y debe reducir el bleeding presente en cada una de ellas.

4.2.2. Métodos basados en la forma de onda

Otra línea de investigación consiste en trabajar directamente con las muestras de audio. En este caso, el modelo recibe la forma de onda original y genera también las señales de salida como formas de onda. De esta manera, no es necesario convertir previamente el audio en un espectrograma ni reconstruirlo posteriormente mediante la fase de la mezcla.

Esta forma de procesamiento permite que el modelo trabaje con toda la información de la señal. Sin embargo, también presenta dificultades. El audio contiene un número muy elevado de muestras y algunos eventos musicales se extienden durante varios segundos. Por este motivo, la red debe analizar una cantidad considerable de información para relacionar los cambios rápidos de la señal con su evolución a más largo plazo.

Uno de los primeros modelos destacados en este ámbito fue Wave-U-Net [21]. Esta arquitectura adapta al audio una estructura formada por varias escalas de análisis. Durante el procesamiento, la red estudia tanto los detalles breves de la forma de onda como un contexto temporal más amplio. Después utiliza la información obtenida para generar directamente las fuentes separadas.

Wave-U-Net demostró que era posible realizar todo el proceso de separación en el dominio temporal. No obstante, sus primeras versiones todavía ofrecían resultados inferiores a los mejores modelos basados en espectrogramas. A partir de este planteamiento se desarrollaron arquitecturas más avanzadas.

Entre ellas se encuentra Demucs, cuya estructura se explicó en el apartado 3.4.1. Este modelo también trabaja directamente con la forma de onda, pero incorpora una mayor capacidad de procesamiento y utiliza una red recurrente para relacionar la información de diferentes instantes del fragmento [7].

En las pruebas publicadas por sus autores, Demucs superó a los modelos espectrales utilizados en la comparación y obtuvo una mejora en la naturalidad del audio generado. Estos resultados mostraron que el procesamiento temporal podía competir con los métodos que hasta ese momento dominaban la separación musical [7].

Aun así, el modelo no eliminaba por completo la contaminación entre las fuentes. Los propios autores observaron que parte de la voz podía permanecer en el acompañamiento y viceversa. Esto demuestra que, aunque el modelo genere directamente las formas de onda, todavía pueden aparecer errores de separación.

También debe tenerse en cuenta su coste computacional. Demucs necesita analizar segmentos con un número elevado de muestras y ejecutar una red de gran tamaño. Este aspecto resulta especialmente importante cuando se pretende utilizar el modelo durante la reproducción del audio y no únicamente como un proceso externo sin una limitación estricta de tiempo.

4.2.3. Modelos híbridos

Los métodos espectrales y temporales presentan ventajas diferentes. El espectrograma facilita el análisis de las frecuencias, mientras que la forma de onda conserva

directamente la evolución completa de la señal. En lugar de elegir solamente una de estas opciones, los modelos híbridos combinan ambas formas de procesamiento.

Hybrid Demucs sigue este planteamiento. Como se explicó en el apartado 3.4.2, el modelo dispone de una rama temporal que trabaja con la forma de onda y una rama espectral que utiliza la STFT. La información obtenida mediante ambas ramas se combina para generar las estimaciones finales [8].

La importancia de Hybrid Demucs dentro de la evolución de estos sistemas no se debe únicamente a la incorporación de una segunda rama. El modelo permite compartir información entre las representaciones temporal y espectral, de forma que cada una puede aportar los datos que resulten más útiles para separar una fuente concreta.

Estos resultados muestran las ventajas del procesamiento híbrido, pero también implican una mayor complejidad. El modelo debe ejecutar dos recorridos diferentes, combinar sus datos internos y reconstruir posteriormente las señales. Como consecuencia, aumentan las operaciones necesarias y la memoria utilizada durante la inferencia.

La Tabla 4 resume las características principales de los modelos analizados.

Modelo	Representación utilizada	Aportación principal	Limitación respecto al presente proyecto
Wave-U-Net	Forma de onda	Introdujo una separación directa del audio mediante un análisis a varias escalas temporales.	Sus primeras versiones presentaban resultados inferiores a los mejores modelos espectrales.
Open-Unmix	Espectrograma	Proporcionó un sistema abierto y reproducible para separar fuentes musicales.	Se orienta principalmente a mezclas estéreo y a cuatro fuentes predefinidas.
Demucs	Forma de onda	Mejóro la calidad de la separación directa en el dominio temporal.	Puede dejar contaminación entre las fuentes y requiere una cantidad elevada de operaciones.
Hybrid Demucs	Forma de onda y espectrograma	Combina la información temporal y frecuencial dentro de un mismo modelo.	Su mayor complejidad dificulta su ejecución con restricciones de tiempo.

Tabla 4 Comparación de algunos modelos representativos de separación musical mediante aprendizaje profundo. Fuente: elaboración propia a partir de [7], [8], [21] y [22].

4.2.4. Aplicación al de-bleeding de grabaciones multicanal

La mayoría de los modelos anteriores se desarrollaron para separar una canción mono o estéreo en varias pistas. En este tipo de problema, todas las fuentes aparecen reunidas dentro de una misma mezcla y el sistema intenta recuperar categorías como la voz, la batería o el bajo.

El de-bleeding de grabaciones realizadas con microfonía cercana presenta una situación diferente. Se dispone de varios canales sincronizados y cada uno de ellos contiene principalmente el instrumento situado junto a su micrófono. Sin embargo, también aparecen contribuciones de los demás instrumentos.

La existencia de varios canales proporciona información adicional. Un mismo instrumento puede aparecer con mayor intensidad en su micrófono principal y con menor nivel en los demás. También puede llegar con pequeños retardos o con una modificación de su contenido frecuencial debido a la posición de cada micrófono y a las reflexiones de la sala.

Un modelo multicanal puede analizar conjuntamente estas diferencias para reconocer qué contenido debe conservarse en cada salida. Por ejemplo, si un sonido aparece con mayor claridad en uno de los canales y de forma más débil en los restantes, esta relación puede ayudar al sistema a identificar cuál es su señal principal y cuáles son sus contribuciones de bleeding.

No obstante, un modelo entrenado para separar cuatro fuentes desde una mezcla estéreo no puede utilizarse directamente para esta tarea. Es necesario adaptar el número de entradas y salidas, mantener la correspondencia entre los canales y entrenar el sistema con grabaciones que representen el tipo de contaminación que se desea reducir.

También cambia la forma de interpretar las salidas. En la separación musical habitual, cada salida suele representar una categoría concreta. En el de-bleeding multicanal, el objetivo consiste en obtener una versión más aislada de la fuente predominante en cada canal. Por tanto, las salidas deben conservar el mismo orden y la misma relación con las entradas.

El trabajo desarrollado previamente por Béjar Martos llevó a cabo esta adaptación utilizando Hybrid Demucs como punto de partida [9]. El modelo se preparó para procesar conjuntamente señales procedentes de varios micrófonos cercanos y reducir las contribuciones no deseadas. Sus características concretas, el conjunto de datos utilizado y el procedimiento original de ejecución se estudiarán en el apartado 4.5.

Esta diferencia debe tenerse en cuenta al comparar los resultados publicados. Un buen funcionamiento sobre una mezcla estéreo no garantiza el mismo comportamiento en una grabación multicanal afectada por bleeding. El resultado depende tanto de la arquitectura como de los datos y de la tarea para la que se haya entrenado el modelo.

Los modelos revisados muestran que la separación de fuentes puede plantearse de distintas formas según la representación del audio y el tipo de señales que se desean obtener. Estas diferencias también se reflejan en las herramientas disponibles, ya que cada una adopta un procedimiento y un entorno de uso determinados. Algunas de las alternativas existentes se presentan en el siguiente apartado.

4.3. Herramientas existentes para la reducción del bleeding y la separación de fuentes

Los modelos estudiados en el apartado anterior han dado lugar a diferentes herramientas que permiten separar fuentes o reducir el contenido no deseado de una grabación. Algunas se ofrecen como programas comerciales con una interfaz preparada para usuarios de audio, mientras que otras se distribuyen como proyectos abiertos que deben ejecutarse mediante Python o desde una línea de comandos. Aunque todas están relacionadas con la separación de señales, no comparten necesariamente el mismo objetivo, las mismas entradas ni el mismo modo de utilización.

Una de las soluciones comerciales más conocidas es iZotope RX. Este programa incluye De-bleed, una función diseñada específicamente para reducir la presencia de una señal que se ha introducido en otra pista [23]. En su funcionamiento clásico, el usuario proporciona la pista que desea limpiar y otra señal que contiene con mayor claridad la fuente interferente. El sistema estudia la relación entre ambas y utiliza esa información para atenuar el contenido no deseado.

Este método puede utilizarse, por ejemplo, cuando una pista vocal contiene el sonido procedente de unos auriculares o cuando un micrófono ha captado parte de otro instrumento. Su principal limitación es que requiere disponer de una referencia adecuada y mantener ambas señales correctamente sincronizadas. Además, normalmente se aplica sobre una pista concreta, por lo que el tratamiento de una grabación con muchos micrófonos puede requerir varias operaciones.

RX 12 mantiene este funcionamiento mediante el modo Classic, pero incorpora además un nuevo modo de De-bleed basado en aprendizaje automático. Este puede trabajar sin una pista de referencia y utilizarse como complemento durante la reproducción [23]. Sin embargo, no permite seleccionar libremente cualquier instrumento o fuente. El usuario debe elegir el tipo de señal que desea aislar entre ocho opciones predefinidas: voz, caja, bombo, toms, platos, batería, guitarra y piano. Por tanto, puede resolver diferentes casos habituales de bleeding, pero no constituye una solución adaptable a cualquier instrumento ni a una configuración personalizada de fuentes.

RX 12 fue publicado el 28 de abril de 2026, cuando el presente TFG ya se encontraba en desarrollo [24]. Se incluye en este apartado para reflejar el estado actual de este tipo de herramientas, aunque no formaba parte de las alternativas disponibles al comienzo del proyecto. Su aparición también muestra el interés reciente por trasladar la reducción del bleeding basada en aprendizaje automático a complementos que puedan utilizarse directamente dentro de una DAW.

RX también dispone de Music Rebalance, una función destinada a separar una mezcla musical en cuatro grupos: voz, bajo, percusión y resto de instrumentos. El usuario puede modificar el nivel de cada grupo o exportar las señales estimadas por separado [23]. Esta herramienta resulta útil cuando únicamente se dispone de la mezcla final de una canción, pero su objetivo es diferente al del de-bleeding de una grabación multicanal. Sus salidas representan categorías musicales generales y no las fuentes principales asociadas a cada micrófono.

Otra alternativa es SpectraLayers Pro. Este programa representa el audio mediante capas y ofrece tanto herramientas de separación musical como una función específica denominada DeBleed. Para reducir una interferencia, el usuario selecciona la capa que desea limpiar y las capas que contienen las posibles fuentes del bleeding. Después puede ajustar la sensibilidad y el grado de reducción aplicado [25].

SpectraLayers también incluye Unmix Song, que permite dividir una mezcla en voz, batería, bajo, guitarra, piano, instrumentos de viento y resto del acompañamiento. Las señales obtenidas se almacenan como capas independientes y pueden editarse o exportarse posteriormente [25].

El programa puede ejecutarse de forma independiente y también integrarse mediante ARA en distintas estaciones de trabajo, entre ellas REAPER. Esta integración permite acceder a los fragmentos de la sesión sin tener que exportarlos manualmente. Sin embargo, su flujo de trabajo continúa estando orientado al análisis y la edición de capas. No se presenta como un único modelo que reciba simultáneamente las doce señales de micrófono y devuelva doce salidas manteniendo la correspondencia entre ellas.

Junto a estas aplicaciones comerciales existen alternativas abiertas como Demucs (y derivados) y Spleeter. Las implementaciones habituales de estos sistemas permiten introducir un archivo de audio y generar posteriormente varios archivos con las fuentes separadas. Demucs obtiene normalmente voz, batería, bajo y acompañamiento [26], mientras que Spleeter dispone de configuraciones para generar dos, cuatro o cinco stems [27].

La principal ventaja de estas herramientas es que su código puede utilizarse y adaptarse para desarrollar nuevas soluciones. No obstante, su uso habitual se realiza fuera de la DAW. El usuario debe ejecutar el programa, esperar a que finalice la separación y cargar después los archivos generados en su proyecto de audio. Además, sus modelos

preentrenados están orientados principalmente a separar categorías fijas desde una mezcla mono o estéreo.

La Tabla 5 resume las alternativas analizadas y sus principales diferencias respecto al sistema desarrollado en este trabajo.

Herramienta	Función principal	Forma habitual de uso	Diferencia respecto al presente TFG
iZotope RX De-bleed	Reducir una interferencia presente en una pista. Modo Classic. necesita una referencia y el modo basado en aprendizaje automático: admite ocho tipos de señal predefinidos.	Editor de audio y, desde RX 12, complemento de tiempo real.	Se aplica sobre la señal que se desea limpiar y no se plantea como un procesamiento conjunto de doce entradas y salidas.
iZotope RX Music Rebalance	Separar una mezcla en voz, bajo, percusión y resto de instrumentos.	Editor o complemento en las versiones recientes.	Utiliza cuatro categorías fijas y parte normalmente de una mezcla musical.
SpectraLayers Pro	Reducir <i>bleeding</i> entre capas o separar una canción en varios grupos instrumentales.	Su funcionamiento está orientado al análisis y la edición de capas.	Puede dejar contaminación entre las fuentes y requiere una cantidad elevada de operaciones.
Demucs	Separar un archivo musical en varios <i>stems</i> .	Python o línea de comandos.	La implementación disponible procesa archivos externos y no constituye por sí sola un complemento para REAPER.
Spleeter	Separar un archivo en dos, cuatro o cinco <i>stems</i> .	Python o línea de comandos.	Trabaja con categorías predefinidas y requiere un flujo externo a la DAW.

Tabla 5 Comparación de herramientas existentes para la reducción del bleeding y la separación de fuentes. Fuente: elaboración propia a partir de [23], [25], [26] y [27].

Estas herramientas demuestran que actualmente existen diferentes formas de reducir el bleeding o separar los componentes de una grabación. Por tanto, el objetivo del presente TFG no es desarrollar la primera solución capaz de realizar esta tarea. Su aportación se encuentra en adaptar un modelo concreto, previamente entrenado para señales obtenidas

mediante microfonía cercana, y trasladarlo a un complemento VST3 que pueda utilizarse directamente en REAPER.

El modelo heredado puede trabajar con hasta doce canales de entrada sincronizados y generar el mismo número de señales de salida [9]. A diferencia de las herramientas que dividen una mezcla estéreo en categorías específicas, como voz, batería o bajo, el modelo en [9] es agnóstico al instrumento, de tal forma que cada salida debe mantener la correspondencia con su canal de entrada y conservar principalmente la fuente asociada a ese micrófono, reduciendo la presencia del sonido captado desde los demás.

De acuerdo con la documentación consultada, entre las alternativas analizadas no se ha encontrado una herramienta que reúna exactamente estas características: procesamiento de grabaciones multicanal obtenidas mediante microfonía cercana, gestión de hasta doce entradas y salidas, funcionamiento con cualquier instrumento e integración directa en REAPER. El proyecto no busca sustituir a las soluciones comerciales existentes, sino estudiar cómo trasladar este sistema específico desde un entorno experimental hasta una forma de uso más cercana al trabajo habitual dentro de una DAW.

Esta transición no consiste únicamente en cargar el modelo dentro de un complemento. También requiere adaptar su funcionamiento al intercambio de audio por bloques de la DAW, mantener la organización multicanal y controlar el tiempo necesario para generar los resultados. Por este motivo, el siguiente apartado analiza el sistema heredado, las limitaciones de su forma original de ejecución y los cambios generales necesarios para realizar la integración propuesta.

4.4. Sistema heredado

El antecedente directo de este TFG es el Trabajo Fin de Máster “Separation of Stems from Close-Microphone Mixtures for Spatialization of Audio Studio Recordings”, realizado por María Dolores Béjar Martos [9]. Además del desarrollo y entrenamiento del modelo, dicho trabajo incluyó una primera forma de utilizarlo desde REAPER. Por tanto, para situar correctamente el presente proyecto, es necesario diferenciar los elementos que se heredan de las limitaciones que existían en aquella integración.

4.4.1. Sistema de partida y funcionamiento previo

El trabajo anterior desarrolló una adaptación multicanal de Hybrid Demucs orientada a grabaciones realizadas mediante microfónica cercana. El modelo fue entrenado para reducir el bleeding entre señales captadas de manera simultánea y para trabajar con configuraciones de hasta doce fuentes. A partir de las señales de entrada, genera una estimación por cada canal, procurando conservar la fuente principal y reducir el sonido procedente de los demás micrófonos [9].

Tanto el modelo entrenado como sus parámetros, la configuración utilizada, el conjunto de datos y los estudios realizados sobre su capacidad de separación forman parte del sistema heredado. Estos elementos ya habían sido desarrollados y evaluados antes del comienzo de este TFG. Por ello, el presente trabajo no plantea un nuevo entrenamiento ni modifica la arquitectura de Hybrid Demucs, cuyo funcionamiento general se explicó en el capítulo 3.

Además de evaluar el modelo mediante archivos de audio, el trabajo previo desarrolló una primera integración con REAPER. Aunque esta solución se presentaba como un prototipo de plugin, el modelo no se cargaba realmente dentro de la DAW mediante un formato de complemento como VST3. El procesamiento se realizaba en un programa externo escrito en Python y conectado con REAPER mediante BlackHole, un dispositivo de audio virtual para macOS [9].

Durante la reproducción, REAPER enviaba las señales multicanal a través de BlackHole. El programa externo recibía estas muestras, ejecutaba el modelo mediante PyTorch y devolvía las estimaciones obtenidas a otros canales del mismo dispositivo virtual. Finalmente, las señales procesadas se dirigían hacia nuevas pistas de REAPER para poder escucharlas y continuar trabajando con ellas.

El programa acumulaba cinco segundos de audio antes de ejecutar el modelo. Siguiendo la diferencia establecida en el apartado 3.5.3, esta cantidad de audio debe entenderse como un segmento de procesamiento y no como uno de los pequeños bloques entregados normalmente por una DAW. Una vez reunido el segmento, el modelo realizaba la separación y el resultado se devolvía progresivamente a REAPER [9].

Esta solución permitió comprobar que el modelo podía utilizarse junto con una DAW y que sus salidas podían devolverse manteniendo la organización multicanal. Sin embargo, el procesamiento continuaba dependiendo de una aplicación situada fuera de REAPER y el resultado no estaba disponible hasta que se había reunido el segmento completo y finalizado la inferencia.

4.4.2. Limitaciones y transición hacia la integración propuesta

La utilización del sistema anterior requería preparar un entorno de Python, ejecutar el programa externo, configurar BlackHole y establecer manualmente las rutas necesarias para enviar y recuperar las señales. En una configuración de doce canales, era necesario organizar tanto las doce señales de entrada como las doce salidas procesadas. Aunque este procedimiento resultaba válido para demostrar el funcionamiento del modelo, aumentaba el número de elementos que debían mantenerse configurados y coordinados.

La principal limitación estaba relacionada con el funcionamiento temporal. El sistema necesitaba recibir cinco segundos de audio antes de comenzar cada ejecución, a los que se añadía el tiempo empleado por el modelo para generar el resultado. Además, se detectaron pequeñas interrupciones audibles entre algunos de los segmentos procesados. Por ello, el propio trabajo anterior describía el prototipo como una solución aproximada al tiempo real, pero no como un procesamiento completamente continuo [9].

Entre las líneas futuras propuestas se encontraban la mejora de la gestión de los segmentos, la reducción de las discontinuidades y la implementación del modelo como un verdadero complemento VST. Estas necesidades constituyen el punto de partida de la integración en este TFG [9].

La transición propuesta conserva el modelo previamente entrenado, pero modifica el sistema encargado de ejecutarlo y de intercambiar el audio con REAPER. En lugar de enviar las señales hacia una aplicación independiente, se plantea incorporar el procesamiento dentro de un complemento desarrollado en C++. Para ello, TorchScript proporciona una representación ejecutable del modelo, LibTorch permite cargarla desde C++ y JUCE aporta la estructura necesaria para construir el complemento VST3 [15], [17], [19], [20].

De esta forma, el procesador puede insertarse directamente en REAPER y recibir las señales mediante el mecanismo habitual de comunicación entre una DAW y un complemento. Este planteamiento permite prescindir del programa externo y del encaminamiento mediante BlackHole, manteniendo el procesamiento dentro del propio proyecto de audio.

No obstante, el uso de VST3 no resuelve por sí solo todas las limitaciones del sistema anterior. REAPER entrega el audio mediante bloques normalmente más pequeños que los segmentos requeridos por el modelo. Por ello, el complemento debe reunir las muestras recibidas, conservar el orden de los canales, ejecutar la separación y devolver los resultados siguiendo el ritmo de reproducción. También debe controlar el tiempo empleado y evitar que el retraso aumente progresivamente o que aparezcan interrupciones entre segmentos.

Una vez explicado el funcionamiento del sistema anterior y las limitaciones que motivan su evolución, el siguiente apartado recoge una comparación final entre las alternativas estudiadas y el planteamiento seguido en este TFG.

4.5. Síntesis comparativa y posicionamiento del presente trabajo

Los apartados anteriores muestran que las soluciones estudiadas actúan de formas diferentes y no siempre persiguen el mismo resultado. Algunas tratan de evitar o reducir el bleeding, mientras que otras buscan separar automáticamente las fuentes presentes en una grabación. La Tabla 6 resume las principales diferencias y relaciona cada alternativa con la necesidad abordada en este TFG.

Alternativa	Características principales	Relación con el presente trabajo
Métodos convencionales	Incluyen medidas aplicadas durante la grabación y procedimientos posteriores, como la corrección temporal, las puertas y expansores, el filtrado o la edición manual y espectral.	Pueden reducir el <i>bleeding</i> de forma relativamente sencilla, pero su resultado depende de las señales y de los ajustes realizados. No generan automáticamente una estimación separada para cada canal.
Modelos de aprendizaje profundo	Utilizan representaciones temporales, espectrales o híbridas para aprender a separar fuentes. Entre los modelos revisados se encuentran Wave-U-Net, Open-Unmix, Demucs e Hybrid Demucs.	Ofrecen una mayor capacidad de separación, aunque sus versiones generales suelen estar orientadas a mezclas estéreo y a un conjunto determinado de fuentes.
Herramientas existentes	Soluciones como RX, SpectraLayers, Demucs o Spleeter permiten realizar el procesamiento mediante editores, complementos o aplicaciones externas.	Facilitan el acceso a estas técnicas, pero presentan configuraciones y formas de integración diferentes a las requeridas por el modelo multicanal heredado.
Sistema heredado	Utiliza una adaptación de Hybrid Demucs para trabajar con hasta doce entradas y salidas correspondientes. Su ejecución se realizaba desde Python y se conectaba con REAPER mediante BlackHole [9].	Responde al tipo de grabación estudiado, pero depende de una aplicación externa, utiliza segmentos de cinco segundos y puede producir esperas y discontinuidades entre resultados.
Planteamiento del presente TFG	Conserva el modelo entrenado y propone ejecutarlo desde un complemento VST3 desarrollado con C++, JUCE y LibTorch.	Busca integrar el procesamiento directamente en REAPER, mantener la organización multicanal y adaptar la ejecución del modelo a un flujo de audio continuo y con un tiempo de procesamiento controlado.

Tabla 6 Comparación entre las alternativas revisadas y el planteamiento del presente TFG.

Fuente: elaboración propia a partir de los apartados anteriores.

De todas las alternativas analizadas, el sistema más cercano al objetivo de este trabajo es el prototipo heredado, ya que utiliza el mismo modelo y mantiene la misma organización multicanal. La diferencia principal no se encuentra en el proceso de separación realizado por la red, sino en la forma de ejecutar el modelo y de intercambiar el audio con REAPER.

Por ello, el presente TFG se sitúa entre la separación de fuentes mediante aprendizaje profundo y el desarrollo de complementos de audio. El trabajo parte de un modelo cuya capacidad de separación ya ha sido estudiada y se centra en comprobar si puede trasladarse a una integración directa y continua dentro de una DAW. A partir de este punto, los capítulos siguientes describen el procedimiento seguido y la evaluación del sistema desarrollado.

5. MATERIALES Y MÉTODOS

En este capítulo se presentan los materiales utilizados y el procedimiento seguido para desarrollar el sistema. A diferencia de los capítulos anteriores, donde se explicaron los fundamentos técnicos y se revisaron las soluciones existentes, a partir de este punto la memoria se centra en el trabajo realizado durante el TFG.

En primer lugar, se describen el equipo, el entorno de desarrollo, las herramientas empleadas, el modelo heredado y las señales utilizadas durante las pruebas. A continuación, se explica la metodología seguida desde el análisis del sistema de partida y las tareas preliminares, hasta la exportación de Hybrid Demucs y su integración en el complemento. Finalmente, se desarrolla la arquitectura de la solución y se define el procedimiento utilizado para evaluar su funcionamiento dentro de REAPER.

Para facilitar la lectura, en este capítulo se recogen las decisiones, los parámetros y las operaciones necesarias para comprender el desarrollo. Los comandos de instalación y la configuración detallada de las dependencias se incluyen en el Anexo A. El enrutamiento multicanal de REAPER se amplía en el Anexo B, mientras que la localización y ejecución de los scripts y archivos complementarios se resumen en el Anexo C. Por último, el Anexo D reúne las mediciones y los resultados adicionales que no resulta necesario mostrar de forma completa en el cuerpo de la memoria.

Para facilitar la consulta y la reproducción del trabajo, el código fuente, los scripts de exportación y validación, el proyecto JUCE y el ReaScript desarrollado se han reunido en un repositorio público denominado “TFG-Hybrid-Demucs-VST3”. El repositorio también incluye un archivo README con una descripción de su contenido y las indicaciones básicas para preparar el proyecto. El material se encuentra disponible en el siguiente enlace [29]:

<https://github.com/esm00046/TFG-Hybrid-Demucs-VST3>.

5.1. Materiales y entorno de desarrollo

El desarrollo del proyecto requirió combinar herramientas de Python, utilizadas para trabajar inicialmente con los modelos, con un entorno de C++ destinado a la construcción del complemento VST3. También fue necesario disponer de una DAW en la que cargar el complemento, configurar las señales multicanal y realizar las pruebas de funcionamiento.

5.1.1. Equipo y sistema operativo

El trabajo se realizó en un MacBook equipado con un procesador Apple M4 y con la versión 26.3 de macOS. En este mismo equipo se prepararon y exportaron los modelos, se compiló el complemento VST3 y se llevaron a cabo las pruebas dentro de REAPER. Por tanto, los tiempos de procesamiento y el comportamiento temporal presentados posteriormente corresponden a esta plataforma concreta.

Al no disponer de una tarjeta gráfica NVIDIA, no era posible utilizar CUDA. Durante las primeras fases se trabajó con la CPU para simplificar las pruebas y comprobar cada parte de la integración de forma progresiva. Posteriormente, se utilizó Metal Performance Shaders (MPS), que permite ejecutar las operaciones de PyTorch en la GPU integrada de los equipos Apple.

Un modelo empleado en las pruebas preliminares basado en Pytorch y más pequeño que el modelo definitivo se validó tanto en CPU como mediante MPS. Una vez completada esta fase, el modelo Hybrid Demucs definitivo también se ejecutó mediante MPS dentro del complemento. De esta forma, fue posible validar el funcionamiento de ambos procesadores y utilizar la aceleración de la GPU en la versión final del sistema.

5.1.2. Herramientas y bibliotecas utilizadas

La Tabla 7 resume las principales herramientas empleadas y la función que desempeñaron durante el desarrollo.

Herramienta o tecnología	Utilización en el proyecto
Git y GitHub	Descarga y consulta del repositorio que contenía el modelo heredado, sus archivos de configuración y los recursos proporcionados por los tutores. También gestionar versiones y publicar el código de este TFG.
Visual Studio Code	Revisión del código de Python, ejecución de los scripts y preparación de las pruebas de exportación y validación de los modelos.
Miniconda y Python	Creación de un entorno aislado para instalar las dependencias y ejecutar el código heredado. El archivo de entorno original tuvo que adaptarse a macOS eliminando las dependencias relacionadas con CUDA.
PyTorch	Carga de los modelos y sus parámetros, ejecución de las primeras pruebas y preparación de las exportaciones

	necesarias para utilizar los modelos desde C++.
TorchScript	Generación de los archivos .pt que reúnen las operaciones del modelo y sus parámetros en una representación que puede cargarse fuera de Python [19].
LibTorch	Carga y ejecución de los módulos TorchScript desde el código C++ del complemento [20].
JUCE y Projucer	Creación de la estructura del complemento, configuración del formato VST3 y generación del proyecto utilizado posteriormente en Xcode [15], [16].
Xcode	Compilación, depuración y configuración del proyecto para enlazar JUCE con LibTorch en macOS.
REAPER	Carga del complemento VST3, configuración de las pistas y realización de las pruebas funcionales, multicanal y temporales [18].
ReaScript	Automatización de la creación del bus multicanal, la inserción del complemento y la preparación de las pistas destinadas a recibir sus salidas.

Tabla 7 Principales herramientas utilizadas durante el desarrollo. Fuente: elaboración propia.

El proyecto de Python se preparó a partir del entorno proporcionado junto con el repositorio heredado. Esta configuración tuvo que ajustarse a las características del MacBook, ya que el archivo original incluía dependencias de CUDA. La instalación completa, las modificaciones realizadas y las versiones de las dependencias se recogen en el Anexo A.

Por otra parte, el proyecto del complemento se generó mediante JUCE y se compiló en Xcode. La integración de LibTorch hizo necesario configurar el estándar C++17, las rutas de los archivos de cabecera, las bibliotecas dinámicas y las opciones de enlazado. En este apartado únicamente se identifican las herramientas utilizadas, mientras que el proceso seguido para conectarlas se explica en el apartado 5.2.2 y su configuración detallada se incluye en el Anexo A.

5.1.3. Modelo heredado y material de audio

El material principal utilizado como punto de partida procede del trabajo desarrollado por María Dolores Béjar Martos [9]. Se recibió un repositorio que contenía la implementación del modelo, sus archivos de configuración, el checkpoint obtenido durante el entrenamiento, los cuadernos utilizados para mostrar ejemplos de inferencia y diferentes señales de audio destinadas a las pruebas.

El modelo definitivo corresponde a una adaptación multicanal de Hybrid Demucs implementada mediante la clase `HDemucs`, definida en el archivo `demucs/hdemucs.py`. Su configuración se obtiene del archivo `conf.yaml`, mientras que los parámetros entrenados se cargan desde el checkpoint `best_epoch=155.ckpt`. El cuaderno de Jupyter `testmodel2.ipynb` permitió comprobar cómo se creaba el modelo, cómo se cargaban estos parámetros y cómo se preparaban las señales antes de ejecutar la inferencia.

Estos elementos se utilizaron como material heredado y no se realizó un nuevo entrenamiento. El trabajo desarrollado en este TFG parte del modelo ya obtenido y se centra en preparar una representación compatible con C++, incorporarla al complemento y adaptar su ejecución al funcionamiento de REAPER.

El modelo espera una entrada formada por doce canales y genera el mismo número de señales estimadas. Esta configuración condicionó el diseño del complemento, ya que era necesario conservar la correspondencia entre cada entrada y su salida. También debía garantizarse que el tensor entregado al modelo mantuviera siempre sus doce canales internos, incluso cuando en REAPER se utilizaran menos señales reales. La solución aplicada para estos casos se explica en el apartado 5.2.3.

Antes de trabajar con el modelo multicanal se utilizó el modelo Denoiser [28], de menor tamaño, como material de pruebas. Su finalidad no era sustituir al modelo definitivo, sino comprobar mediante un modelo más sencillo (diseñado por sus autores para funcionar en tiempo real en CPU) la exportación a TorchScript, la carga desde LibTorch (C++) y el intercambio de las muestras entre los búferes de JUCE y los tensores de PyTorch. Este modelo trabajaba con señales mono a 16 kHz y se probó tanto en CPU como mediante MPS.

Para las pruebas finales con Hybrid Demucs, se utilizaron ficheros de audio multicanal con hasta doce canales, por lo que permitió comprobar la configuración completa de doce entradas y doce salidas del modelo. Las pruebas se realizaron en REAPER a 48 kHz y se utilizaron para analizar la carga del complemento, la correspondencia entre canales, la ejecución mediante MPS, la continuidad del audio y el funcionamiento del enrutamiento multicanal.

La configuración seguida para introducir el fichero de audio en REAPER y distribuir las salidas del complemento se recoge en el Anexo B. Las mediciones obtenidas durante las pruebas se presentan de forma resumida en el capítulo de resultados y se amplían en el Anexo D.

5.1.4. *Projucer: reacción del esqueleto del plugin*

Projucer es la herramienta incluida en JUCE para crear y configurar proyectos. Al seleccionar la plantilla destinada al desarrollo de complementos de audio, genera la estructura inicial del proyecto, permite establecer el formato de salida (en este caso VST3) y crea los archivos que posteriormente se compilan mediante un entorno de desarrollo como Xcode. El esqueleto básico generado por Projucer separa el procesamiento de audio de la interfaz gráfica. Para ello, utiliza cuatro archivos principales, resumidos en la Tabla 8.

Archivo	Función principal
PluginProcessor.h	Declara la clase principal del procesador, sus variables internas y los métodos que deben ser sobrescritos para comunicarse con el anfitrión.
PluginProcessor.cpp	Implementa el procesamiento del audio, la inicialización de los recursos, la configuración de los buses y la creación de la instancia del complemento.
PluginEditor.h	Declara la clase correspondiente a la interfaz gráfica y los componentes visuales que contiene.
PluginEditor.cpp	Implementa la construcción, representación y distribución de los elementos de la interfaz.

Tabla 8 Archivos principales de un proyecto de complemento de audio generado mediante Projucer. Fuente: elaboración propia.

La clase declarada en *PluginProcessor.h* debe heredar de `juce::AudioProcessor`. Esta clase base define la interfaz mediante la que el DAW (REAPER) se comunica con el complemento y establece las funciones que deben implementarse para recibir, procesar y devolver el audio. En el proyecto desarrollado, esta clase se denominó *Demucs12AudioProcessor* y contiene tanto las estructuras utilizadas para gestionar los bloques como el módulo TorchScript (modelo exportado) cargado mediante LibTorch [11], [16].

Uno de sus métodos principales de esta clase es `prepareToPlay(double sampleRate, int maximumExpectedSamplesPerBlock)`, que el plugin ejecuta antes de comenzar el procesamiento. El argumento `sampleRate` indica la frecuencia de muestreo utilizada por el anfitrión, expresada en hercios, mientras que `maximumExpectedSamplesPerBlock` proporciona el tamaño máximo de bloque previsto. Esta función se utiliza para reservar memoria, inicializar los búferes y preparar aquellos elementos que dependen de la configuración de REAPER. Cuando estos recursos dejan de ser necesarios, el plugin llama a `releaseResources()`.

El procesamiento de cada bloque se realiza mediante `processBlock(juce::AudioBuffer<float>& buffer, juce::MidiBuffer&`

midiMessages. El primer argumento contiene las muestras de audio correspondientes a los canales de entrada y salida. El segundo contiene los eventos MIDI asociados al bloque, aunque no se utiliza en este complemento. El contenido del búfer puede modificarse directamente, de modo que las muestras de entrada se sustituyen por los resultados que deben devolverse a REAPER. Como el número real de muestras puede variar entre llamadas, se consulta mediante `buffer.getNumSamples()` en lugar de asumir un valor fijo.

Otros métodos relevantes son `isBusesLayoutSupported()`, que permite aceptar o rechazar las configuraciones de canales propuestas por REAPER, y `getStateInformation()` y `setStateInformation()`, destinados a guardar y recuperar el estado del complemento. En este proyecto, la interfaz no contiene parámetros modificables, por lo que la gestión del estado es mínima, mientras que la validación de los buses resulta esencial para admitir configuraciones de entre uno y doce canales.

El tiempo empleado por `processBlock()` debe ser inferior a la duración del bloque suministrado por la DAW. Si el procesamiento supera este intervalo, REAPER no recibe la salida a tiempo y aparecen huecos o discontinuidades durante la reproducción. Este comportamiento se comprobó mediante un complemento de prueba en el que se introdujo artificialmente un tiempo de procesamiento superior al disponible. Como puede apreciar en la Figura 6, durante la caracterización inicial también se observó, por ejemplo, que un bloque de 512 muestras a 48 kHz representa aproximadamente 10.67 ms.

The screenshot shows the `processBlock` function in `SourceSeparationVSTAudioProcessor.cpp`. Line 142 has a debug statement: `DBG("Bloque actual -> muestras: " + juce::String(numSamples) //Muestra en la consola de Xcode un mensaje con toda la información que acabas de calcular.`. Below the code, the Xcode console shows the following debug output:

```
> this = (SourceSeparationVSTAudioProcessor *) 0x123f3fe0
> buffer = (juce::AudioBuffer<float> &) 0x000000016bd94518
> noDenormals (juce::ScopedNoDenormals)
L totalNumInputChannels = (int) 2
L totalNumOutputChannels = (int) 2
L numSamples = (const int) 512
L sampleRate = (const double) 48000
L blockDurationMs = (const double) 10.666666666666666
```

Figura 6 Caracterización de un bloque de 512 muestras recibido por el complemento a 48 kHz durante una llamada a `processBlock()`. Fuente: elaboración propia.

La interfaz gráfica se organiza de forma equivalente mediante los archivos `PluginEditor.h` y `PluginEditor.cpp`. La clase del editor hereda de

`juce::AudioProcessorEditor` y mantiene una referencia al procesador para consultar la información que debe mostrarse. El método `paint(juce::Graphics& g)` se utiliza para dibujar el fondo, los textos y los demás elementos gráficos, mientras que `resized()` establece la posición y el tamaño de los componentes cuando cambia el área disponible.

Finalmente, el procesador indica que dispone de una interfaz mediante `hasEditor()` y crea su instancia mediante `createEditor()`. En *Demucs12*, la interfaz tiene una función únicamente informativa. Por este motivo, el procesamiento del audio y la ejecución del modelo permanecen dentro de *PluginProcessor*, independientemente de que la ventana del editor esté abierta o cerrada.

5.1.5. Exportación de modelos con TorchScript

Los modelos desarrollados mediante PyTorch se definen y entrenan habitualmente desde Python. Sin embargo, el complemento se implementó en C++ y debía ejecutar Hybrid Demucs mediante LibTorch sin mantener un intérprete de Python ni una aplicación externa en funcionamiento. Para hacerlo posible, el modelo se exportó mediante el paquete TorchScript, que permite generar una representación de las operaciones necesarias para la inferencia y de los parámetros aprendidos durante el entrenamiento [19], [20].

En este proyecto se utilizó el procedimiento denominado *tracing*, disponible mediante `torch.jit.trace()`. Este método ejecuta el modelo con un tensor de ejemplo y registra las operaciones realizadas durante esa ejecución. El resultado es un módulo TorchScript que puede guardarse en un archivo y cargarse posteriormente desde C++.

Antes de realizar la exportación fue necesario reconstruir Hybrid Demucs a partir de los elementos proporcionados con el proyecto heredado. La arquitectura se obtuvo de la clase *HDemucs*, definida en *demucs/hdemucs.py*, mientras que los valores necesarios para configurarla se leyeron directamente desde *conf.yaml*. Posteriormente, se cargaron los parámetros entrenados almacenados en *checkpoints/best_epoch=155.ckpt*.

Los nombres incluidos en el diccionario de parámetros del checkpoint contenían el prefijo `model.`, que no formaba parte de los nombres esperados por una instancia directa de *HDemucs*. Por este motivo, el prefijo se eliminó antes de efectuar la carga. También se descartaron otros prefijos como `auralossnew`. (que pertenece a una función de pérdidas utilizada durante el entrenamiento). Una vez recuperados los parámetros, el modelo se situó en modo de inferencia mediante `eval()` y se configuró para trabajar con datos de tipo *float32*.

Para realizar el trazado se creó un tensor de ejemplo con dimensiones [1, 12, 8192]. La primera dimensión representa un único elemento en el lote, la segunda corresponde a los doce canales de entrada y la tercera contiene las muestras de cada canal. Este tensor no

representa una señal concreta empleada para entrenar el modelo, sino que permite a TorchScript ejecutar y registrar el recorrido de las operaciones que forman la inferencia.

La exportación se realizó mediante dos scripts: *demucs_export_torchscript_CPU.py* mantuvo el modelo y el tensor de ejemplo en CPU y generó el archivo *demucs_export_torchscript_CPU.pt*. Por su parte, *demucs_export_torchscript_MPS.py* comprobó la disponibilidad del backend MPS, trasladó el modelo y la entrada a este dispositivo y generó *demucs_export_torchscript_MPS.pt*. En ambos casos se utilizó `torch.jit.trace()` y el módulo resultante se almacenó mediante `save()`.

La Figura 7 muestra los fragmentos principales del script de exportación para MPS. En la Figura 7 (a) se recoge la creación del tensor de ejemplo con dimensiones [1, 12, 8192] y su asignación al dispositivo seleccionado. En la Figura 7 (b) se muestra la ejecución del trazado mediante `torch.jit.trace()` y el guardado del módulo TorchScript resultante en el archivo .pt.

```

38 def main():
141 # =====
142 # 6. Crear entrada de ejemplo EN MPS
143 # =====
144 #
145 # La forma usada será:
146 #
147 # [1, 12, 8192]
148 #
149 # Es decir:
150 # batch = 1
151 # canales = 12
152 # muestras = 8192
153 #
154
155 audio_channels = confdemucs["audio_channels"]
156
157 example_num_samples = 8192
158
159 example_input = torch.randn(
160     1,
161     audio_channels,
162     example_num_samples,
163     dtype=torch.float32,
164     device=device
165 )
166
167 print("\nEntrada de ejemplo:")
168 print("Dispositivo example_input:", example_input.device)
169 print("Tipo example_input:", example_input.dtype)
170 print("Forma example_input:", example_input.shape)
171
172 # Comprobamos que el modelo está en MPS.
173 for name, param in model.named_parameters():
174     print("\nPrimer parámetro:", name)
175     print("Dispositivo parámetro:", param.device)
176     print("Tipo parámetro:", param.dtype)
177     break

```

(a)

```

179 # =====
180 # 7. Probar forward antes de exportar
181 # =====
182 #
183 print("\nProbando forward en MPS antes de exportar...")
184 #
185 with torch.inference_mode():
186     output = model(example_input)
187
188 torch.mps.synchronize()
189
190 print("Forward completado correctamente.")
191 print("Dispositivo salida:", output.device)
192 print("Tipo salida:", output.dtype)
193 print("Forma salida:", output.shape)
194 print("Output abs max:", output.abs().max().item())
195
196 # =====
197 # 8. Hacer trace en MPS
198 # =====
199 #
200 print("\nExportando modelo TorchScript en MPS...")
201 #
202 with torch.inference_mode():
203     traced_model = torch.jit.trace(
204         model,
205         example_input,
206         strict=False
207     )
208
209 # =====
210 # 9. Guardar modelo
211 # =====
212
213 traced_model.save(str(output_path))
214
215 print("\nModelo exportado correctamente.")
216 print("Guardado en:", output_path)

```

(b)

Figura 7 Fragmentos principales del script de exportación de Hybrid Demucs mediante TorchScript para MPS: (a) creación del tensor de ejemplo; (b) trazado del modelo y guardado del módulo .pt. Fuente: elaboración propia.

En este trabajo, la extensión `.pt` identifica el módulo TorchScript generado durante la exportación. Este archivo contiene el grafo de operaciones registrado mediante *tracing* y los tensores correspondientes a los parámetros entrenados. No contiene el conjunto de datos, el proceso de entrenamiento ni el código completo del proyecto original. Tampoco debe confundirse con el archivo `.ckpt`, que conserva el estado obtenido durante el entrenamiento. El módulo `.pt` está preparado para inferencia y puede cargarse desde C++ mediante `torch::jit::load()`.

Por tanto, la exportación no vuelve a entrenar Hybrid Demucs ni modifica sus parámetros, sino que transforma el modelo heredado en una representación ejecutable mediante LibTorch. Los scripts empleados y los comandos necesarios para generar los módulos TorchScript se recogen en el Anexo C, mientras que el código completo se encuentra disponible en el repositorio del proyecto.

5.2. Diseño e implementación del complemento VST3

El complemento VST3 se encarga de conectar el flujo de audio de REAPER con el modelo Hybrid Demucs exportado a TorchScript. La implementación se realizó en C++ utilizando JUCE para construir el complemento, mientras que LibTorch permitió cargar y ejecutar el modelo en C++ sin depender del intérprete de Python.

En este apartado se describe la organización interna del complemento y las principales etapas necesarias para adaptar el funcionamiento del modelo a un flujo de audio continuo. Para ello, se aborda la integración de JUCE y LibTorch, la configuración multicanal, la acumulación de los bloques recibidos, la creación de los tensores, la ejecución mediante MPS o CPU y la reconstrucción de las señales de salida. Finalmente, se presenta la interfaz gráfica del plugin y el script implementado para facilitar su uso dentro de REAPER (enrutamiento automático de las pistas de entrada/salida).

5.2.1. Arquitectura general del sistema

La arquitectura desarrollada se organiza alrededor de tres elementos principales: REAPER, el complemento VST3 (al que hemos nombrado *Demucs12*) y el módulo Hybrid Demucs. REAPER actúa como aplicación DAW anfitriona y se encarga de configurar el proyecto de audio, gestionar las pistas y entregar periódicamente los bloques de audio al complemento. El VST3 funciona como intermediario entre los búferes proporcionados por la

DAW y el formato de entrada requerido por el modelo, mientras que Hybrid Demucs realiza el procesamiento de las señales.

Dentro del complemento se distinguen dos partes. El **procesador** de audio contiene el núcleo funcional y se encarga de recibir los canales, preparar la entrada, ejecutar el modelo y devolver los resultados a REAPER. Por otro lado, el **editor** gestiona la representación gráfica y muestra información sobre el estado del sistema. Esta separación permite mantener la interfaz apartada de las operaciones que se realizan sobre el audio.

El procesamiento principal se ejecuta dentro de la función `processBlock()`, que JUCE llama cada vez que REAPER entrega un nuevo bloque de audio. A partir de estas llamadas, el complemento conserva las muestras necesarias para formar el segmento (más extenso) utilizado por el modelo, organiza los datos de los distintos canales y genera el tensor de entrada al modelo. Una vez finalizada la inferencia, las salidas se reconstruyen y se copian nuevamente a los búferes de JUCE antes de devolver el control a la DAW.

Hybrid Demucs requiere siempre doce canales, aunque el proyecto de REAPER pueda proporcionar un número inferior de señales reales. Por este motivo, el complemento mantiene internamente una estructura fija de doce canales y completa con ceros aquellos que no contienen audio. De esta manera, el modelo recibe siempre la organización con la que fue entrenado, mientras que el usuario puede trabajar con grabaciones formadas por entre uno y doce canales.

La ejecución del modelo, en forma de módulo TorchScript, se realiza mediante la librería LibTorch. Debido al coste computacional de Hybrid Demucs, se prioriza el uso de MPS para aprovechar la aceleración proporcionada por Apple Metal. La CPU se mantiene como alternativa cuando la ruta acelerada no está disponible o no puede utilizarse. Esta selección no modifica el recorrido general de las señales, ya que el complemento prepara y recupera los datos siguiendo la misma estructura independientemente del dispositivo empleado.

La arquitectura se completa con un script de automatización para REAPER. Este elemento no forma parte del plugin VST3, sino que actúa como herramienta de apoyo para automatizar las tareas necesarias para su uso: crear un bus multicanal (entrada del plugin), insertar el complemento y generar las pistas asociadas a las señales procesadas (salidas). De esta forma, se evita realizar manualmente un enrutamiento que requiere configurar varias pistas y conexiones.

Como se muestra en la Figura 7, cada componente cumple una función diferenciada: REAPER gestiona la reproducción y el enrutamiento de audio; JUCE define la API que permite la comunicación con la DAW así como los búferes de audio de entrada/salida; LibTorch permite ejecutar el modelo desde C++; y Hybrid Demucs genera las doce señales procesadas.

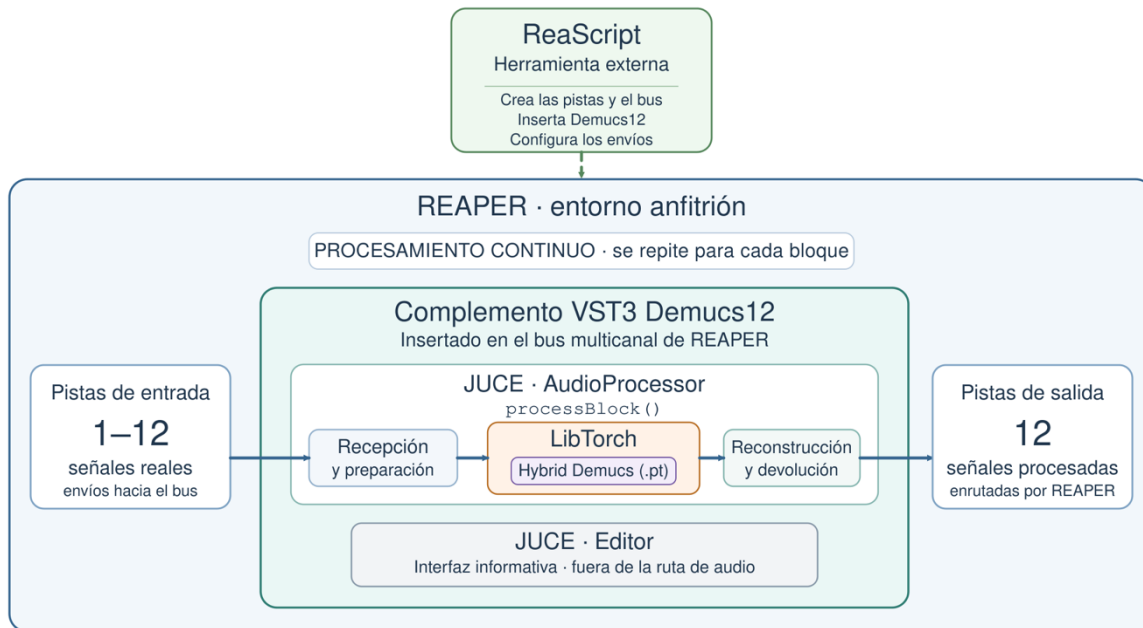


Figura 8 Arquitectura general del sistema formado por REAPER, el complemento VST3 desarrollado con JUCE, LibTorch, el módulo Hybrid Demucs y el script de automatización del enrutamiento. Fuente: elaboración propia.

5.2.2. Proyecto de Xcode: integración de LibTorch

Una vez establecida la arquitectura general del sistema, el siguiente paso consistió en incorporar LibTorch al proyecto desarrollado con JUCE. Ambas herramientas cumplen funciones diferentes dentro del complemento. JUCE proporciona la estructura necesaria para generar el formato VST3, comunicarse con REAPER y gestionar los búferes de audio, mientras que LibTorch permite cargar y ejecutar desde C++ el modelo previamente exportado a TorchScript.

El proyecto del complemento *Demucs12* se configuró mediante Projucer y se compiló posteriormente en Xcode. El uso de LibTorch se realiza únicamente en los archivos *PluginProcessor.h* y *PluginProcessor.cpp*, ya que la carga y ejecución del modelo forman parte del procesador de audio y no de la interfaz gráfica.

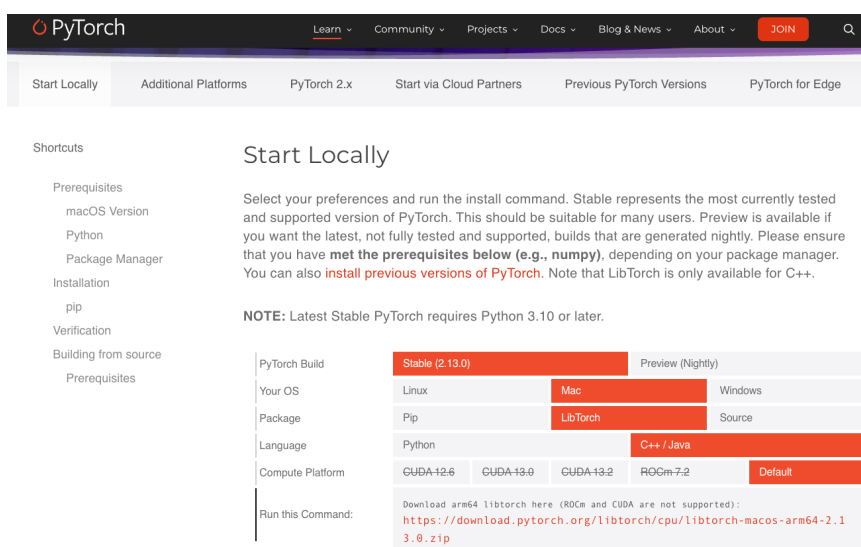
En primer lugar, se descargó desde la página oficial de PyTorch la distribución precompilada de LibTorch para macOS. Se seleccionó el paquete estable destinado a C++ y compatible con la arquitectura ARM64 del equipo utilizado. Una vez descargado y descomprimido, el paquete proporcionaba las cabeceras necesarias para utilizar la API de PyTorch y las bibliotecas que debían enlazarse posteriormente con el complemento. La

selección del paquete compatible con macOS y la arquitectura ARM64 se muestra en la Figura 8 (a).

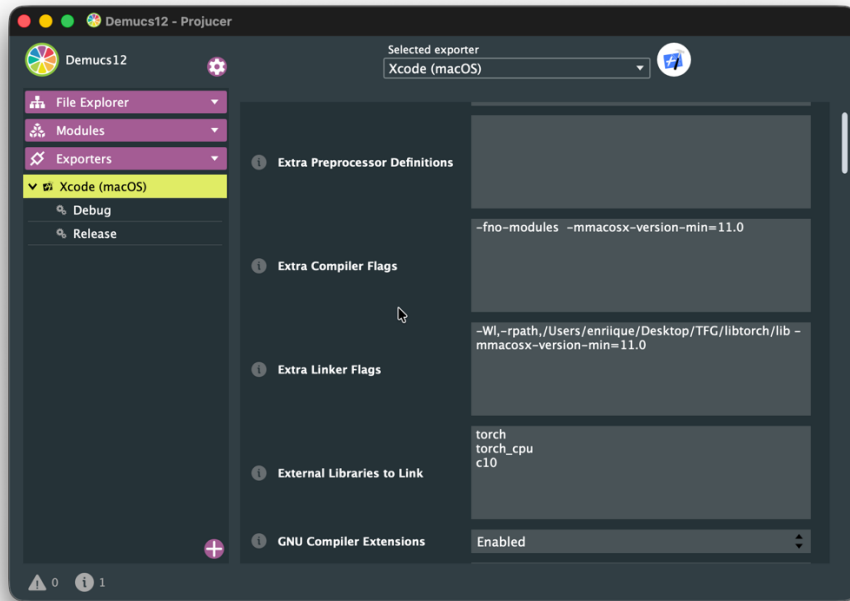
Para que el compilador pudiera localizar los archivos de cabecera de LibTorch, se incorporaron de forma recursiva las carpetas *libtorch/include* y *libtorch/include/torch/csrc/api/include* a las rutas de búsqueda de cabeceras. También se añadió *libtorch/lib* como ruta de búsqueda de bibliotecas. Los valores absolutos dependían de la ubicación de LibTorch en el equipo de desarrollo, por lo que los comandos y rutas concretas se recogen en el Anexo A.

La configuración restante se estableció en el exportador para Xcode ofrecido por Projucer. Entre las opciones añadidas se encontraba *-fno-modules*, utilizada como parte de la configuración de compatibilidad durante la compilación, y el establecimiento de macOS 11.0 como versión mínima del sistema. En las opciones de enlazado se incorporó un *rpath* dirigido a la carpeta *lib* de LibTorch, permitiendo localizar sus bibliotecas dinámicas durante la ejecución. Finalmente, se añadieron *torch*, *torch_cpu* y *c10* en el apartado destinado a las bibliotecas externas. La configuración aplicada en el exportador de Xcode de Projucer se recoge en la Figura 8 (b).

Una vez configurados los ajustes de compilación y enlazado, Xcode permite generar correctamente el archivo VST3 compilado. El complemento puede copiarse en el directorio de complementos de macOS y ser detectado por REAPER. Con ello se comprobó que *Demucs12* era reconocido por la DAW y que el procesador podía acceder a las funciones de LibTorch necesarias para cargar el modelo.



(a)



(b)

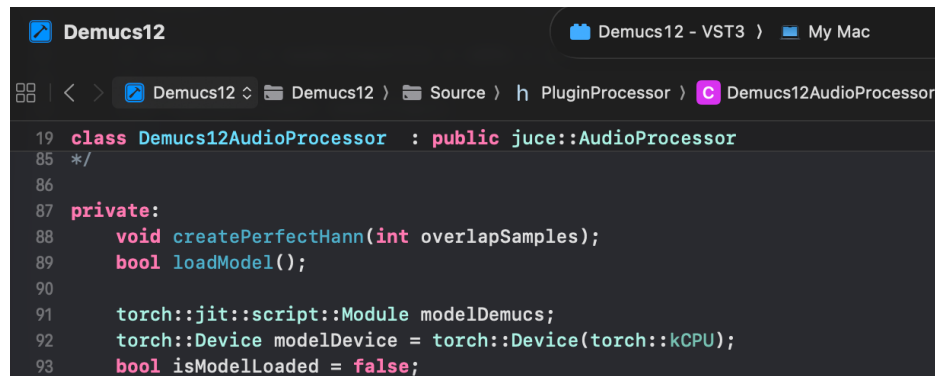
Figura 9 Obtención y configuración de LibTorch: (a) selección de la distribución estable para C++ y macOS ARM64 en la página oficial de PyTorch; (b) configuración del exportador de Xcode en Projucer, incluyendo las opciones de compilación, la ruta de búsqueda de y las bibliotecas externas. Fuente: (a) captura de la página oficial de PyTorch [19]; (b) elaboración propia.

5.2.3. Carga del modelo con LibTorch

Para utilizar la API C++ de PyTorch desde el código del complemento se debe incorporar la cabecera `#include <torch/script.h>`. A partir de ella es posible acceder a las clases y funciones proporcionadas por LibTorch para representar dispositivos, definir tensores, cargar modelos exportados y ejecutar posteriormente la inferencia.

La clase principal del plugin se denomina `Demucs12AudioProcessor`, que hereda de `juce::AudioProcessor`, la clase base proporcionada por la API de JUCE para implementar la parte de procesamiento de señal del plugin. Esta clase se define en el fichero `PluginProcessor.h`. Se añadió la función privada `loadModel()`, encargada de preparar el modelo antes de comenzar el procesamiento. También se añadió como miembro de la clase el objeto `modelDemucs`, de tipo `torch::jit::script::Module`, que mantiene en memoria el módulo TorchScript durante todo el tiempo de vida del plugin. Junto a este objeto se declaró la variable `modelDevice`, inicializada con CPU como dispositivo por defecto, y

una variable de estado booleana `isModelLoaded`, cuyo valor inicial es *false*. Esta última permite conservar el resultado de la carga y comprobar desde otras partes del complemento si el modelo se encuentra disponible. Las declaraciones utilizadas para almacenar el modelo, seleccionar el dispositivo y registrar su estado se muestran en la Figura 9.



```
19 class Demucs12AudioProcessor : public juce::AudioProcessor
85 */
86
87 private:
88     void createPerfectHann(int overlapSamples);
89     bool loadModel();
90
91     torch::jit::script::Module modelDemucs;
92     torch::Device modelDevice = torch::Device(torch::kCPU);
93     bool isModelLoaded = false;
```

Figura 10 Definición de la clase principal del complemento y sus miembros añadidos: la función de carga del modelo, el módulo TorchScript, el dispositivo de ejecución y la variable utilizada para registrar el estado del modelo. La clase se declara en *PluginProcessor.h*.

Fuente: elaboración propia.

La carga del modelo se implementó en el fichero *PluginProcessor.cpp* en el cuerpo de la función `loadModel()`. En primer lugar, esta función comprueba la disponibilidad del backend MPS mediante `torch::mps::is_available()`. Si está disponible, se selecciona el dispositivo `torch::kMPS` y se utiliza el archivo *demucs_export_torchscript_MPS.pt* (nuestro modelo Hybrid Demus exportado para MPS). En caso contrario, se mantiene la ejecución en CPU y se selecciona *demucs_export_torchscript_CPU.pt*. De este modo, el complemento dispone de una ruta alternativa cuando la aceleración mediante Apple Metal no puede utilizarse.

Una vez determinado el archivo correspondiente, el módulo se carga mediante `torch::jit::load()`. A continuación, `modelDemucs.to(modelDevice)` traslada sus parámetros al dispositivo seleccionado (MPS o CPU) y `modelDemucs.eval()` establece el modo de evaluación, de forma que el modelo queda preparado para realizar inferencia sin aplicar las operaciones propias de la fase de entrenamiento.

La secuencia se encuentra dentro de una estructura `try-catch`. Si todas las operaciones finalizan correctamente, `loadModel()` devuelve *true*. Si LibTorch genera una excepción `e` de tipo `c10::Error`, se muestra en consola la información correspondiente al error invocando a `e.what()` y la función devuelve *false*. El resultado booleano permite actualizar la variable `isModelLoaded` y conocer si el modelo puede utilizarse antes de

intentar ejecutar la inferencia. La Figura 10 muestra el fragmento principal de `loadModel()`, incluyendo la selección del dispositivo, la carga del módulo y el control de posibles excepciones.



```
41 bool Demucs12AudioProcessor::loadModel()
42 {
43     // =====
44     // 1. Selección del dispositivo
45     // =====
46     // Primero intentamos usar MPS/Metal, que es la opción GPU en Mac.
47     // Si no estuviera disponible, usamos CPU como segunda opción.
48     //
49     if (torch::mps::is_available())
50     {
51         modelDevice = torch::Device(torch::kMPS);
52         std::cout << "MPS is available. Using Apple Metal GPU." << std::endl;
53         modelpath += "demucs_export_torchscript_MPS.pt";
54     }
55     else
56     {
57         modelDevice = torch::Device(torch::kCPU);
58         std::cout << "MPS not available. Using CPU." << std::endl;
59         modelpath += "demucs_export_torchscript_CPU.pt";
60     }
61     // =====
62     // 2. Carga del modelo TorchScript
63     // =====
64     std::cout << "Loading model from: " << std::endl;
65     std::cout << modelpath << std::endl;
66     modelDemucs = torch::jit::load(modelpath);
67     // Movemos el modelo al dispositivo seleccionado.
68     modelDemucs->to(modelDevice);
69     // Modo inferencia/evaluación.
70     modelDemucs->eval();
71     std::cout << "Model loaded successfully." << std::endl;
72     return true;
73 }
74 catch (const c10::Error& e)
75 {
76     std::cerr << "Error loading the model: " << e.what() << std::endl;
77     return false;
78 }
79 //=====
```

Figura 11 Fragmento de la función `loadModel()` utilizado para seleccionar el dispositivo, cargar el módulo TorchScript, establecer el modo de evaluación y controlar posibles errores de LibTorch. Fuente: elaboración propia.

La función `loadModel()` se invoca durante la inicialización del complemento, en el constructor de la clase principal del plugin, y no en cada llamada a `processBlock()`. Esta organización evita leer repetidamente el archivo `.pt` y mantiene el modelo disponible en memoria para procesar los sucesivos segmentos de audio. Las operaciones relacionadas con la formación de los tensores y la llamada a la función `forward()` del modelo se desarrollan posteriormente en los apartados 5.2.5 y 5.2.6.

La integración realizada estableció la unión entre la infraestructura de audio proporcionada por JUCE y el entorno de inferencia de PyTorch. A partir de esta base pudo abordarse la configuración multicanal del complemento y la adaptación de los canales proporcionados por REAPER a la estructura requerida por Hybrid Demucs.

5.2.4. Configuración multicanal del complemento

La configuración multicanal debía permitir la comunicación entre dos estructuras con requisitos diferentes. Por un lado, REAPER puede trabajar con proyectos que contengan entre una y doce señales reales. Por otro, Hybrid Demucs espera siempre una entrada formada por doce canales y genera el mismo número de canales de salida. Por este motivo, se diseñó una configuración flexible en la comunicación entre la DAW y el VST3, manteniendo internamente la estructura fija requerida por el modelo.

El campo *Plugin Channel Configurations* de Projucer se mantuvo vacío. Por tanto, no se estableció una lista fija de combinaciones de entrada y salida durante la generación del proyecto. La configuración de los buses se definió directamente en la clase `Demucs12AudioProcessor` mediante las funciones proporcionadas por JUCE, permitiendo que REAPER negociara con el complemento la distribución que debía utilizarse.

En el constructor del procesador se creó un bus principal de entrada denominado `Input` y otro de salida denominado `Output`. Ambos se declararon como buses activos y se inicializaron con una configuración estéreo mediante `juce::AudioChannelSet::stereo()`. Esta configuración representa el formato inicial con el que se instancia el complemento, pero no limita su funcionamiento exclusivamente a dos canales.

La aceptación de otras distribuciones se controla mediante `isBusesLayoutSupported()`. La aplicación anfitriona llama a esta función cuando propone una determinada configuración. En primer lugar, se recuperan los conjuntos correspondientes al bus principal de entrada y al de salida mediante `getMainInputChannelSet()` y `getMainOutputChannelSet()`. A continuación, ambos conjuntos se comparan directamente. Si la distribución de entrada y la de salida no coinciden, la función devuelve `false`. Esta condición exige que el complemento tenga la misma organización y el mismo número de canales en ambos sentidos. Por tanto, son válidas configuraciones como 2 entradas y 2 salidas, 4 entradas y 4 salidas o 12 entradas y 12 salidas, mientras que se rechazan combinaciones asimétricas como 2 entradas y 12 salidas.

Una vez comprobada esta igualdad, se obtiene el número de canales mediante `inputLayout.size()`. La configuración se acepta únicamente cuando este valor se encuentra entre uno y `modelChannels`, variable que contiene el número de canales aceptado por el modelo (doce, en nuestro caso).

La relación entre la configuración inicial de los buses y la validación de las distribuciones propuestas por REAPER se muestra en la Figura 11.

```

Demucs12
Demucs12 - VST3 My Mac
Demucs12 Demucs12 Source C PluginProcessor Demucs12AudioProcessor::Demucs12AudioProcessor()

11 //=====
12 //Constructor
13 //=====
14 Demucs12AudioProcessor::Demucs12AudioProcessor()
15 : AudioProcessor(BusesProperties()
16     .withInput("Input", juce::AudioChannelSet::stereo(), true)
17     .withOutput("Output", juce::AudioChannelSet::stereo(), true))
18 {
19     isModelLoaded = loadModel();
20 }

```

(a)

```

Demucs12
Demucs12 - VST3 My Mac
Demucs12 Demucs12 Source C PluginProcessor Demucs12AudioProcessor::isBusesLayoutSupported(layouts)

289 bool Demucs12AudioProcessor::isBusesLayoutSupported(const BusesLayout& layouts) const
290 {
291     const auto& inputLayout = layouts.getMainInputChannelSet();
292     const auto& outputLayout = layouts.getMainOutputChannelSet();
293
294     // La entrada y la salida deben tener el mismo número de canales reales.
295     // Ejemplo válido:
296     //     2 entradas -> 2 salidas
297     //     4 entradas -> 4 salidas
298     //     12 entradas -> 12 salidas
299     //
300     // Ejemplo no válido:
301     //     2 entradas -> 12 salidas
302     //     12 entradas -> 2 salidas
303     //
304     if (inputLayout != outputLayout)
305         return false;
306
307     const int numberOfChannels = inputLayout.size();
308
309     // El modelo interno siempre trabaja con 12 canales, pero el usuario
310     // puede usar el plugin con menos canales reales.
311     //
312     // Internamente los canales que falten se rellenarán con ceros.
313     //
314     return numberOfChannels >= 1 && numberOfChannels <= modelChannels;
315 }

```

(b)

Figura 12 Configuración multicanal del procesador: (a) declaración de los buses principales de entrada y salida con una distribución estéreo inicial; (b) validación de configuraciones simétricas comprendidas entre uno y doce canales mediante `isBusesLayoutSupported()`. Fuente: elaboración propia.

La posibilidad de aceptar menos de doce canales facilita la utilización y comprobación del complemento con proyectos de diferentes tamaños. Sin embargo, esta flexibilidad no modifica las dimensiones requeridas por Hybrid Demucs. Antes de realizar la inferencia, los canales disponibles se sitúan en sus posiciones correspondientes dentro de una estructura interna de doce canales y las posiciones restantes se completan con ceros. El procedimiento concreto utilizado para introducir las muestras y completar los canales ausentes se desarrolla en los apartados 5.3.4 y 5.3.5.

En la configuración final utilizada desde REAPER, la pista sobre la que se aplica el complemento debe ser un bus multicanal. Si las grabaciones originales de micrófono cercano se encuentran en pistas individuales, deben por tanto redirigirse primero a las posiciones correspondientes de este bus. Cuando el proyecto dispone de menos de doce señales, las

posiciones que no reciben ningún envío son rellenadas con ceros por el plugin, de forma que la entrada presentada al modelo conserva siempre sus doce canales.

La igualdad entre el número de canales de entrada y salida también simplifica la devolución de los resultados. Cada posición del búfer de salida mantiene una correspondencia directa con la posición utilizada en la entrada, evitando distribuciones asimétricas que podrían provocar una interpretación incorrecta de los canales por parte de REAPER.

Con esta configuración, el complemento puede ser reconocido inicialmente como un procesador estéreo, aceptar posteriormente distribuciones multicanal compatibles y trabajar con el bus de doce canales empleado en el proyecto definitivo. Una vez establecida esta organización, el siguiente paso consistió en recibir y acumular los bloques de audio proporcionados por REAPER.

5.2.5. Recepción y acumulación de bloques de audio

Una vez configurados los buses multicanal del complemento, fue necesario adaptar los bloques de audio entregados por REAPER a un tamaño de entrada más conveniente para Hybrid Demucs. La DAW llama periódicamente a la función `processBlock()` y proporciona en cada llamada un bloque cuyo número de muestras depende de la configuración del sistema. Sin embargo, el modelo no trabaja directamente con estos bloques aislados, sino con una ventana temporal fija algo más extensa (para proporcionar un mayor contexto temporal a la red), que en nuestro caso hemos establecido en 8192 muestras por canal, donde la justificación detallada de la configuración se encuentra en el apartado 6.3 junto con los resultados de uso.

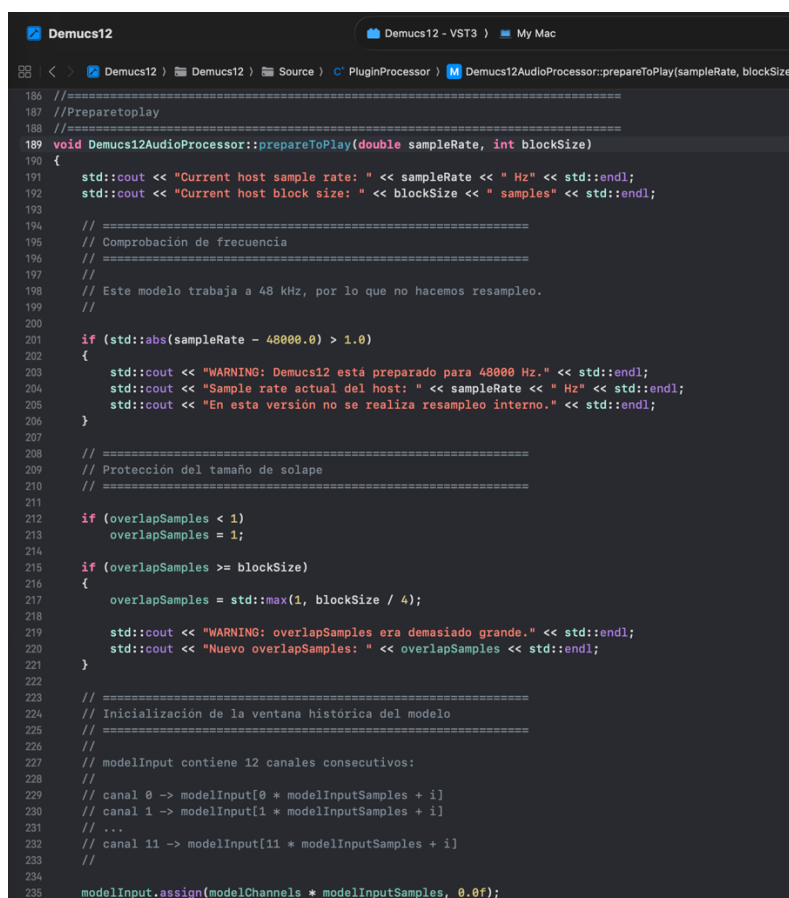
Para resolver esta diferencia se implementó una ventana histórica deslizante. En *PluginProcessor.h* se definieron los parámetros `modelChannels`, con un valor de doce canales, `modelInputSamples`, con 8192 muestras, y `overlapSamples`, establecido inicialmente en 512 muestras (este valor determina el número de muestras descartadas al final de la salida, pero que son almacenadas y solapadas con el inicio de la salida para el siguiente bloque). El vector de C++ `modelInput` es la ventana deslizante encargada de almacenar de forma continua las 8192 muestras más recientes recibidas por el complemento.

Antes de comenzar la reproducción, el DAW llama a la función `prepareToPlay()` del plugin, a la que proporciona la frecuencia de muestreo y el tamaño de bloque previsto. En esta función se comprueba que la frecuencia sea de 48 kHz, ya que el modelo fue entrenado para trabajar con este valor y el complemento no realiza remuestreo interno. Si REAPER

utiliza una frecuencia diferente, se muestra una advertencia para indicar que la configuración no coincide con la esperada.

En `prepareToPlay()` también se comprueba que el número de muestras de solape sea válido respecto al tamaño de bloque comunicado por REAPER. El solape debe contener al menos una muestra y ser inferior al tamaño del bloque. Si su valor fuera igual o superior, se sustituiría por una cuarta parte del tamaño del bloque. La utilización de este solape en la reconstrucción de la salida se explica posteriormente en el apartado dedicado al procesamiento mediante solapamiento-suma.

La memoria destinada a las muestras de entrada para el modelo se reserva mediante `modelInput.assign()`, inicializándose con ceros. El vector contiene doce regiones consecutivas de 8192 muestras, una para cada canal, y se inicializa completamente a cero. Por tanto, su tamaño total es de 98304 valores de tipo *float*. Esta inicialización proporciona un estado conocido al comenzar la reproducción y representa como silencio el contexto temporal que todavía no se ha recibido desde REAPER. La configuración inicial realizada en `prepareToPlay()` se muestra en la Figura 12.



```
186 //=====
187 //PreparetoPlay
188 //=====
189 void Demucs12AudioProcessor::prepareToPlay(double sampleRate, int blockSize)
190 {
191     std::cout << "Current host sample rate: " << sampleRate << " Hz" << std::endl;
192     std::cout << "Current host block size: " << blockSize << " samples" << std::endl;
193
194     // =====
195     // Comprobación de frecuencia
196     // =====
197     //
198     // Este modelo trabaja a 48 kHz, por lo que no hacemos resampleo.
199     //
200
201     if (std::abs(sampleRate - 48000.0) > 1.0)
202     {
203         std::cout << "WARNING: Demucs12 está preparado para 48000 Hz." << std::endl;
204         std::cout << "Sample rate actual del host: " << sampleRate << " Hz" << std::endl;
205         std::cout << "En esta versión no se realiza resampleo interno." << std::endl;
206     }
207
208     // =====
209     // Protección del tamaño de solape
210     // =====
211
212     if (overlapSamples < 1)
213         overlapSamples = 1;
214
215     if (overlapSamples >= blockSize)
216     {
217         overlapSamples = std::max(1, blockSize / 4);
218
219         std::cout << "WARNING: overlapSamples era demasiado grande." << std::endl;
220         std::cout << "Nuevo overlapSamples: " << overlapSamples << std::endl;
221     }
222
223     // =====
224     // Inicialización de la ventana histórica del modelo
225     // =====
226     //
227     // modelInput contiene 12 canales consecutivos:
228     //
229     // canal 0 -> modelInput[0 * modelInputSamples + i]
230     // canal 1 -> modelInput[1 * modelInputSamples + i]
231     // ...
232     // canal 11 -> modelInput[11 * modelInputSamples + i]
233     //
234
235     modelInput.assign(modelChannels * modelInputSamples, 0.0f);
```

Figura 13 Inicialización del procesamiento en `prepareToPlay()`, incluyendo la comprobación de la frecuencia de muestreo, la protección del tamaño de solape y la reserva de la ventana histórica multicanal. Fuente: elaboración propia.

La recepción efectiva del audio se realiza en `processBlock()`. Esta función recibe como argumento un `AudioBuffer<float>` que contiene las muestras suministradas por REAPER y un búfer MIDI que no se utiliza en este complemento. Al comienzo de cada llamada se comprueba el valor de `isModelLoaded`. Si el modelo no se ha cargado correctamente, la función termina sin modificar el búfer, de manera que el audio de entrada continúa su recorrido sin ser procesado.

A continuación, se obtiene el número real de muestras del bloque mediante `buffer.getNumSamples()` y se almacena en la variable `B`. También se consulta el número de canales presentes en el búfer. El valor de `B` se obtiene en cada llamada y no se fija directamente en el código, ya que el tamaño entregado por el anfitrión puede depender de la configuración de REAPER y del dispositivo de audio. Por su parte, las variables `N` y `L` representan respectivamente las 8192 muestras de la ventana completa y las 512 muestras de solapamiento.

También se calculan `realInputChannels` y `realOutputChannels`, limitando el número de canales disponibles al máximo de doce admitido por el modelo. Para ello se toma el menor valor entre los canales declarados por el complemento, los canales presentes en el búfer y `modelChannels`. Esta comprobación evita acceder a canales inexistentes y permite mantener el funcionamiento descrito en el apartado anterior cuando REAPER utiliza menos de doce canales.

Antes de actualizar el historial de muestras, se realizan varias comprobaciones de seguridad. Si el tamaño de `modelInput` no coincide con el producto entre el número de canales y la longitud de la ventana, el vector vuelve a inicializarse con ceros. Del mismo modo, se comprueba el tamaño de las estructuras auxiliares utilizadas posteriormente para reconstruir la salida. Además, el bloque debe ser mayor que el solape y la suma de ambos debe caber dentro de la ventana del modelo. Si alguna de estas condiciones no se cumple, se muestra el error correspondiente y se interrumpe el procesamiento de esa llamada.

La ventana histórica se organiza por canales consecutivos. Las muestras del primer canal ocupan las primeras 8192 posiciones de `modelInput`, las del segundo canal ocupan las siguientes y así sucesivamente hasta el canal doce. De forma general, la muestra i del canal c se encuentra en la posición $c \cdot N + i$.

En cada llamada a `processBlock()` se recorre esta estructura para actualizar los doce canales. En primer lugar, `std::memmove()` desplaza el contenido existente `B` posiciones hacia la izquierda. Como consecuencia, se eliminan las B muestras más antiguas y se conservan las $N-B$ más recientes. Se utiliza `memmove()` porque las regiones de origen y destino pertenecen al mismo vector y se encuentran parcialmente solapadas.

Después del desplazamiento, las B posiciones liberadas al final de cada canal se completan con el bloque recién recibido. Si el canal c existe realmente en REAPER, se obtiene su puntero de lectura mediante `buffer.getReadPointer(c)` y sus muestras se copian con `std::memcpy()`. Si el canal no existe, ese mismo intervalo se rellena con ceros mediante `std::fill()`. De esta forma, la ventana mantiene siempre doce canales, aunque el proyecto únicamente proporcione audio real en una parte de ellos.

La recepción de los datos y el mantenimiento del historial se muestran en la Figura 13. En la parte (a) se observa la obtención del tamaño del bloque y la determinación de los canales reales disponibles, mientras que la parte (b) recoge el desplazamiento de la ventana, la copia de las nuevas muestras y el relleno con ceros de los canales ausentes.

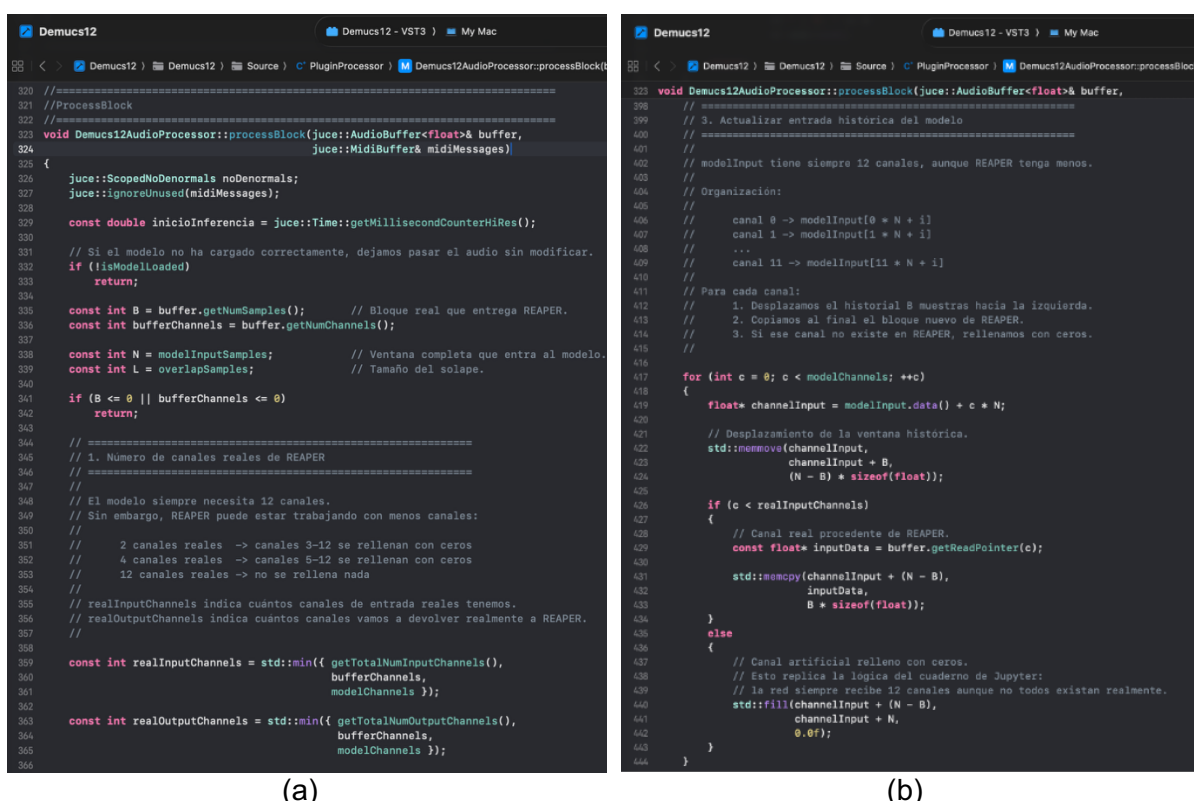


Figura 14 Recepción y actualización de la entrada histórica en `processBlock()`: (a) obtención del tamaño del bloque y del número de canales reales; (b) desplazamiento del historial, incorporación de las nuevas muestras y relleno de los canales ausentes. Fuente: elaboración propia.

Como resultado de esta etapa, `modelInput` contiene en todo momento una representación continua de los fragmentos de audio más recientes, organizada como doce canales de 8192 muestras. Esta memoria contigua se utiliza a continuación para construir un

objeto tensor con dimensiones [1, 12, 8192] que se proporciona al módulo TorchScript, procedimiento que se describe en el siguiente apartado.

5.2.6. Ejecución del modelo y aceleración mediante MPS

Una vez actualizado el historial de entrada, sus muestras deben convertirse al formato utilizado por LibTorch. Esta transformación no se realiza directamente desde el `AudioBuffer` de JUCE. Como se explicó anteriormente, las muestras recibidas desde REAPER se copian primero en el vector `modelInput`, que contiene una región de memoria continua organizada por canales.

La creación del tensor se realiza mediante `torch::from_blob()`, utilizando el puntero devuelto por `modelInput.data()` como dirección inicial de los datos de origen. Al llamar a esta función se especifican las dimensiones [1, 12, N]. La primera dimensión representa un único elemento en el lote, la segunda corresponde a los doce canales del modelo y la tercera contiene las N muestras de cada canal. En la configuración utilizada, N toma el valor de 8192, por lo que el tensor resultante presenta dimensiones [1,12,8192].

También se establece explícitamente el tipo `torch::kFloat32`, coincidente con el tipo float utilizado por el vector y con la precisión empleada durante la exportación del modelo. Esta correspondencia evita realizar conversiones numéricas adicionales antes de la inferencia. La función `from_blob()` crea inicialmente una vista sobre la memoria de `modelInput`, pero no pasa a ser propietaria de esos datos. Por este motivo, se aplica la función `clone()` para generar un tensor independiente que conserva una copia de las muestras actuales. De esta manera, las modificaciones posteriores de la ventana histórica no afectan al tensor que se entrega al modelo.

Finalmente, `inputTensor.to(modelDevice)` sitúa el tensor en el mismo dispositivo que el módulo TorchScript. Si se ha seleccionado MPS, los datos se trasladan a la memoria utilizada por este backend, si el complemento funciona mediante CPU, el tensor permanece en memoria RAM. El proceso completo de creación y preparación del tensor de entrada se muestra en la Figura 14.

```

323 void Demucs12AudioProcessor::processBlock(juce::AudioBuffer<float>& buffer,
445
446     try
447     {
448         // =====
449         // 4. Crear tensor de entrada [1, 12, N]
450         // =====
451
452         torch::Tensor inputTensor = torch::from_blob(
453             modelInput.data(),
454             {1, modelChannels, N},
455             torch::TensorOptions().dtype(torch::kFloat32)
456         ).clone();
457
458         inputTensor = inputTensor.to(modelDevice);

```

Figura 15 Conversión de la memoria continua de `modelInput` en un tensor de tipo `float32` con dimensiones `[1, 12, N]` y traslado al dispositivo seleccionado. Fuente: elaboración propia.

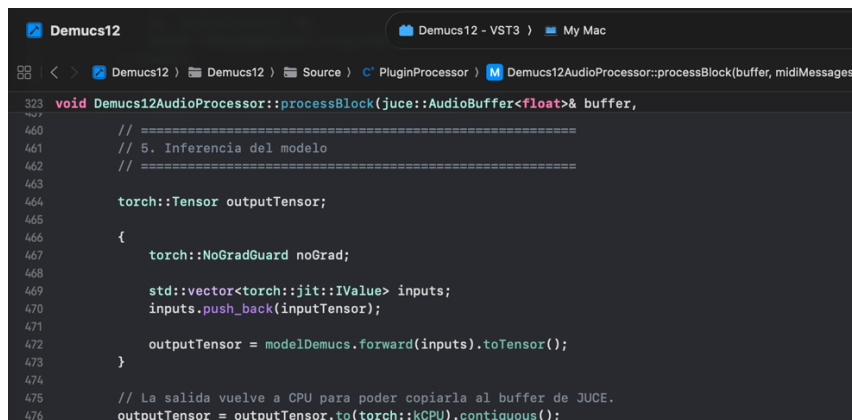
El recorrido inverso se realiza después de ejecutar el modelo. El tensor de salida se traslada a CPU y se convierte en una estructura contigua mediante su función `.contiguous()`, lo que permite acceder posteriormente a sus muestras gracias al puntero devuelto por `.data_ptr<float>()`. A partir de este puntero se recuperan los datos de cada canal y se copian al búfer de JUCE. La ejecución de la inferencia, la comprobación de la forma del tensor de salida y la reconstrucción de los canales se desarrollan en los apartados siguientes.

La inferencia se introduce dentro del ámbito de un objeto `torch::NoGradGuard`. Este elemento desactiva el cálculo y almacenamiento de gradientes, ya que durante el funcionamiento del complemento no se actualizan los parámetros del modelo. De esta manera, LibTorch ejecuta únicamente las operaciones necesarias para obtener la salida.

Un modelo en TorchScript recibe sus argumentos mediante objetos de tipo `torch::jit::IValue`. Por este motivo, se crea un vector de entradas y se incorpora `inputTensor` mediante `push_back()`. A continuación, la llamada `modelDemucs.forward(inputs)` ejecuta las operaciones registradas durante la exportación del modelo. El resultado se devuelve inicialmente como un `IValue` y se recupera como tensor mediante `.toTensor()`.

La ejecución utiliza automáticamente el dispositivo almacenado en `modelDevice`. Como se explicó en el apartado 5.2.2, si el backend MPS se encuentra disponible, el modelo y el tensor de entrada se sitúan en dicho dispositivo. En ese caso, las operaciones de Hybrid Demucs se ejecutan mediante la aceleración proporcionada por Apple Metal. Si MPS no está disponible, el mismo recorrido se mantiene utilizando CPU, sin necesidad de implementar una segunda llamada a `forward()`.

Como se mencionó antes, después de la inferencia, `outputTensor` se traslada a CPU mediante `.to(torch::kCPU)`, ya que el búfer de JUCE se encuentra en la memoria principal y no puede acceder directamente a los datos almacenados en MPS. También se aplica `.contiguous()` para garantizar que las muestras queden organizadas en una región de memoria continua, facilitando su lectura posterior mediante un puntero. La secuencia de ejecución y recuperación del tensor se muestra en la Figura 15.



```

323 void Demucs12AudioProcessor::processBlock(juce::AudioBuffer<float>& buffer,
460 // =====
461 // 5. Inferencia del modelo
462 // =====
463
464     torch::Tensor outputTensor;
465
466     {
467         torch::NoGradGuard noGrad;
468
469         std::vector<torch::jit::IValue> inputs;
470         inputs.push_back(inputTensor);
471
472         outputTensor = modelDemucs.forward(inputs).toTensor();
473     }
474
475     // La salida vuelve a CPU para poder copiarla al buffer de JUCE.
476     outputTensor = outputTensor.to(torch::kCPU).contiguous();

```

Figura 16 Ejecución del módulo TorchScript sin cálculo de gradientes, preparación de los argumentos mediante `IValue` y traslado del tensor resultante desde el dispositivo de inferencia hasta CPU. Fuente: elaboración propia.

Antes de utilizar el resultado se comprueba que el tensor tenga tres dimensiones y una forma compatible con la esperada. El tamaño del lote debe ser uno, la salida debe proporcionar al menos los doce canales del modelo y el número de muestras debe permitir recuperar tanto el bloque que se devolverá a REAPER como la región utilizada para el solape. Si alguna de estas condiciones no se cumple, se muestra un mensaje de error y se interrumpe el procesamiento del bloque antes de copiar datos no válidos.

La ejecución se encuentra además protegida mediante bloques `catch` para capturar tanto las excepciones propias de LibTorch, representadas por `c10::Error`, como otras excepciones estándar de C++. De esta forma, cualquier fallo producido durante la inferencia queda registrado sin propagarse directamente al anfitrión. Una vez validado el tensor de salida, sus muestras pueden recuperarse y reconstruirse para formar los bloques que se envían nuevamente a REAPER, procedimiento descrito en el siguiente apartado.

5.2.7. Reconstrucción de la salida y solapamiento-suma

Una vez finalizada la inferencia, el tensor de salida contiene los doce canales generados por el modelo. Para recuperar el fragmento correspondiente a la llamada actual se

calculan dos posiciones: `blockStart` indica el comienzo de las B muestras que se devolverán a REAPER, mientras que `tailStart` señala el inicio de las últimas L muestras, reservadas para mezclarlas con el bloque siguiente.

Antes de comenzar la reproducción, en la función `prepareToPlay()` se inicializa un vector `prevTailOut` con espacio para doce colas de `overlapSamples` (L) muestras, donde se almacenan las ventanas para realizar el solapamiento entre bloques.

Las ventanas se crean en la función `createPerfectHann()`. Esta función genera una ventana de Hanning de longitud $2L$ y la divide en dos mitades: la primera (creciente), almacenada en `winFirstHalf`, realiza el aumento progresivo del comienzo del segmento actual, mientras que `winSecondHalf` almacena la segunda mitad (decreciente), que atenúa la cola (últimas muestras) que se utilizará para solapar con el comienzo del siguiente bloque. Ambas mitades son complementarias, por lo que para cada posición del solapamiento se cumple que `winFirstHalf[i] + winSecondHalf[i] = 1`.

Esta propiedad permite mantener una ganancia conjunta constante durante la suma y suavizar la transición entre bloques consecutivos. La inicialización de las estructuras y la creación de las ventanas se muestran en la Figura 16.

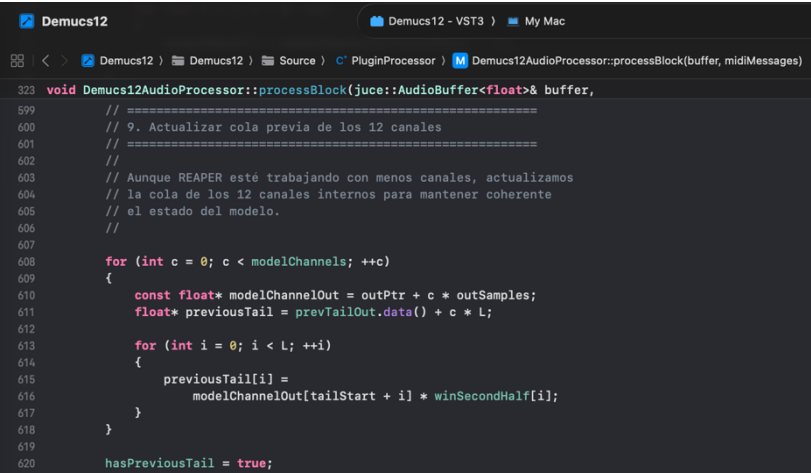


Figura 17 Preparación del solapamiento-suma: (a) inicialización de la cola anterior, creación de las ventanas y estado inicial; (b) generación de la ventana de Hanning y separación de sus mitades de entrada y salida. Fuente: elaboración propia.

La reconstrucción se realiza únicamente sobre los canales de salida disponibles en REAPER. Para cada uno se obtiene un puntero de escritura al `AudioBuffer`, el comienzo del canal correspondiente en el tensor y la cola almacenada durante la llamada anterior.

Las primeras L muestras forman la región de solape. El inicio del bloque actual se multiplica por `winFirstHalf` y se suma a `previousTailOut`, un buffer que contiene la cola enventanada por `winSecondHalf` correspondiente a la llamada anterior. En el primer bloque de la ejecución, las muestras se copian directamente para evitar aplicar una transición desde una cola inexistente. Las muestras intermedias del bloque, comprendidas entre L y B , quedan fuera del solape y se trasladan directamente al búfer de JUCE.

Después de reconstruir el bloque se actualiza `prevTailOut`. Para ello se toman las últimas L muestras de cada una de las doce salidas, comenzando en `tailStart`, y se multiplican por `winSecondHalf`. Estas muestras no se devuelven a REAPER, pero quedan almacenadas hasta la siguiente llamada a `processBlock()`. La aplicación del solapamiento-suma y la actualización posterior de la cola se recogen en la Figura 17.



```
Demucs12
Demucs12 - VST3
My Mac

Demucs12
Demucs12
Source
PluginProcessor
Demucs12AudioProcessor::processBlock(buffer, midiMessages)

323 void Demucs12AudioProcessor::processBlock(juce::AudioBuffer<float>& buffer,
599 // =====
600 // 9. Actualizar cola previa de los 12 canales
601 // =====
602 //
603 // Aunque REAPER esté trabajando con menos canales, actualizamos
604 // la cola de los 12 canales internos para mantener coherente
605 // el estado del modelo.
606 //
607
608 for (int c = 0; c < modelChannels; ++c)
609 {
610     const float* modelChannelOut = outPtr + c * outSamples;
611     float* previousTail = prevTailOut.data() + c * L;
612
613     for (int i = 0; i < L; ++i)
614     {
615         previousTail[i] =
616             modelChannelOut[tailStart + i] * winSecondHalf[i];
617     }
618 }
619
620 hasPreviousTail = true;
```

(a)

```

Demucs12
Demucs12 - VST3
My Mac
Demucs12
Demucs12
Source
PluginProcessor
Demucs12AudioProcessor::processBlock(buffer, midiMessages)

323 void Demucs12AudioProcessor::processBlock(juce::AudioBuffer<float>& buffer,
548 // =====
549 // 8. Copiar salida SOLO a los canales reales de REAPER
550 // =====
551 //
552 // La red produce 12 salidas, pero si REAPER está trabajando
553 // con menos canales, solo devolvemos esos canales reales.
554 //
555 // Ejemplo:
556 // REAPER tiene 2 canales -> devolvemos salidas 1 y 2
557 // REAPER tiene 4 canales -> devolvemos salidas 1, 2, 3 y 4
558 // REAPER tiene 12 canales -> devolvemos las 12 salidas
559 //
560
561 for (int c = 0; c < realOutputChannels; ++c)
562 {
563     float* outputData = buffer.getWritePointer(c);
564
565     const float* modelChannelOut = outPtr + c * outSamples;
566     float* previousTail = prevTailOut.data() + c * L;
567
568     // -----
569     // Primer tramo: solapamiento-suma
570     // -----
571     //
572     // Mezclamos:
573     //   cola anterior suavizada
574     //   +
575     //   inicio del bloque actual suavizado
576     //
577
578     for (int i = 0; i < L; ++i)
579     {
580         const float currentFadeIn =
581             modelChannelOut[blockStart + i] * winFirstHalf[i];
582
583         if (hasPreviousTail)
584             outputData[i] = previousTail[i] + currentFadeIn;
585         else
586             outputData[i] = modelChannelOut[blockStart + i];
587     }
588
589     // -----
590     // Segundo tramo: parte directa del bloque
591     // -----
592
593     for (int i = L; i < B; ++i)
594     {
595         outputData[i] = modelChannelOut[blockStart + i];
596     }
597 }

```

(b)

Figura 18 Reconstrucción continua de la salida: (a) suma de la cola anterior con el inicio suavizado del bloque actual y copia de la parte directa; (b) almacenamiento de la nueva cola ponderada para la siguiente llamada. Fuente: elaboración propia.

Aunque REAPER utilice menos canales, las colas internas de las doce salidas se actualizan para mantener coherente el estado del procesamiento. Los canales adicionales del búfer que no correspondan a salidas reales se limpian para evitar datos residuales. Finalmente, las muestras no finitas se sustituyen por cero y la amplitud se limita al intervalo comprendido entre -0.95 y 0.95 .

Mediante este procedimiento, los bloques generados de forma independiente se enlazan progresivamente antes de devolverse a REAPER. El solapamiento-suma reduce las discontinuidades que podrían aparecer en sus límites y permite mantener una salida multicanal continua durante la reproducción.

5.2.8. Interfaz gráfica y automatización del enrutamiento

El último elemento desarrollado fue la interfaz gráfica del complemento. A diferencia del procesador de audio, esta parte no interviene directamente en la inferencia ni permite modificar los parámetros del modelo. Su finalidad es meramente informativa: identificar el prototipo y presentar de forma resumida las características principales de la implementación.

La interfaz se implementó en los archivos *PluginEditor.h* y *PluginEditor.cpp*. En este último se definieron tanto el aspecto visual de los componentes como su distribución dentro de la ventana, principalmente mediante las funciones `paint()` y `resized()`.

Como se observa en la Figura 18, la ventana muestra una descripción general del sistema, un indicador visual READY y dos paneles con información sobre el procesamiento implementado y la configuración utilizada. Entre los datos presentados se encuentran el uso de Hybrid Demucs, TorchScript, LibTorch y MPS, así como los doce canales internos, la ventana de 8192 muestras, el solape de 512 muestras y la frecuencia de trabajo de 48 kHz. Estos elementos permiten consultar rápidamente las características del prototipo, pero no funcionan como controles susceptibles de ser modificados.

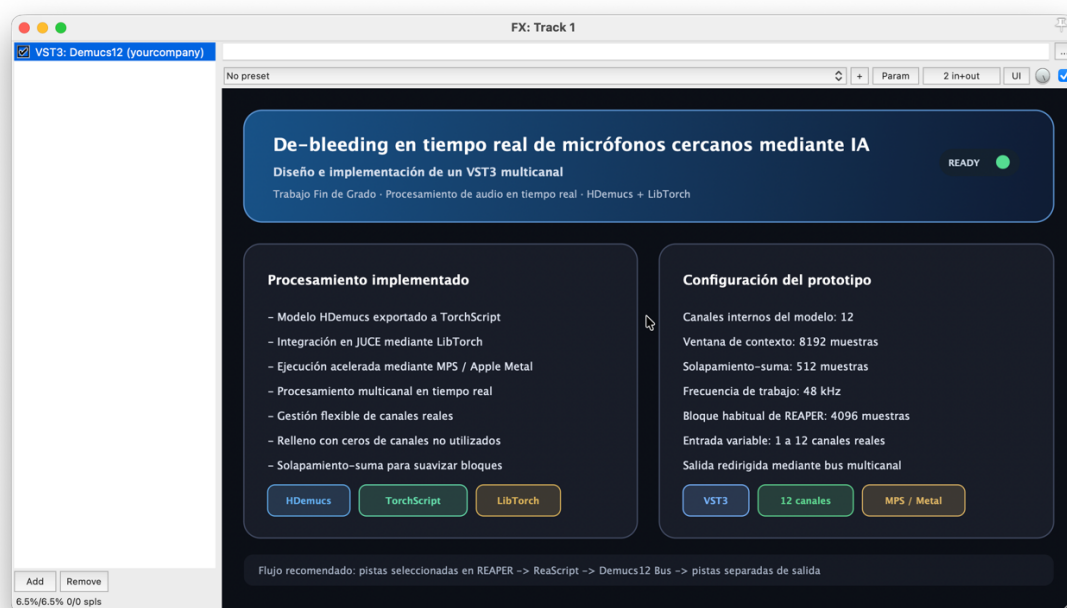


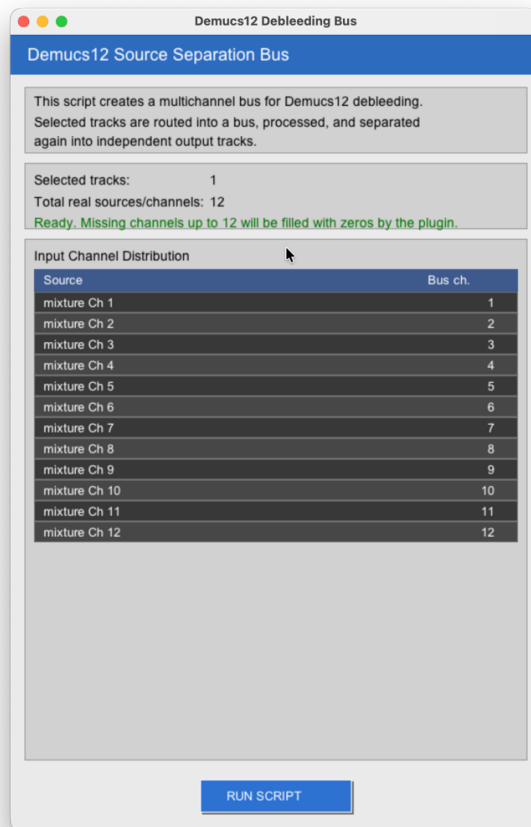
Figura 19 Interfaz informativa del complemento *Demucs12* mostrada dentro de REAPER, con un resumen del procesamiento implementado y de la configuración utilizada por el prototipo. Fuente: elaboración propia.

Además de la interfaz del VST3, se desarrolló un script externo para REAPER mediante ReaScript, escrito en lenguaje LUA, para automatizar la preparación del enrutamiento en REAPER. Esta herramienta no forma parte del procesamiento interno del complemento ni modifica el modelo, sino que configura las pistas y las conexiones necesarias para utilizarlo dentro de un proyecto multicanal.

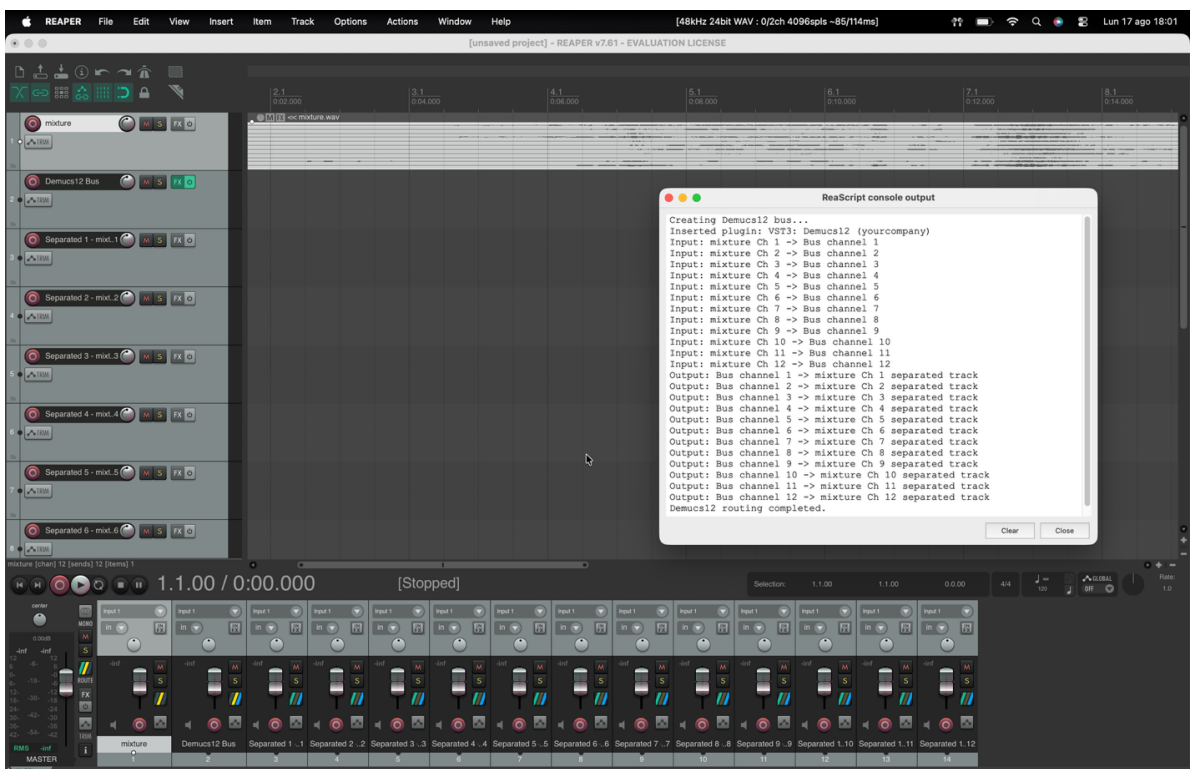
La primera pantalla del script permite revisar las pistas (tracks) seleccionadas por el usuario en la interfaz de REAPER y la distribución de sus canales antes de realizar cambios. En el ejemplo mostrado en la Figura 19 (b) se utiliza una única pista multicanal denominada *mixture*, cuyos doce canales se asignan respectivamente a los canales 1 a 12 del bus. La ventana también informa de que, cuando se dispone de menos de doce canales reales, los canales restantes serán completados con ceros por el propio complemento. Una vez comprobada la distribución, el usuario puede iniciar la configuración mediante el botón RUN SCRIPT.

Al ejecutar el script se crea una nueva pista denominada *Demucs12 Bus*, se inserta en ella nuestro complemento VST3 y se conectan los canales de la señal de entrada con los canales correspondientes del bus. A continuación, se crean las pistas de salida para recibir los canales procesados por parte del plugin. Esto permite al usuario disponer de las pistas individuales con bleeding reducido, para poder seguir aplicando nuevos efectos sobre cada una de ellas. La consola de ReaScript registra estas operaciones y permite comprobar la correspondencia establecida entre las entradas, el bus y las salidas.

Las dos etapas de esta automatización se muestran en la Figura 19. La parte (a) presenta la pantalla previa de comprobación y distribución de canales, mientras que la parte (b) muestra el proyecto resultante en REAPER, formado por la pista original, el bus de procesamiento y las doce pistas separadas de salida.



(a)



(b)

Figura 20 Automatización del enrutamiento multicanal mediante ReaScript: (a) comprobación de las pistas seleccionadas y asignación de los canales de entrada; (b) resultado de la ejecución, con el bus *Demucs12*, las pistas de salida y el registro de las conexiones realizadas. Fuente: elaboración propia.

Esta separación mantiene diferenciadas las funciones de cada componente. La interfaz del VST3 describe visualmente el prototipo, el script prepara el proyecto de REAPER y el procesador ejecuta el tratamiento de las señales. Aunque el mismo enrutamiento podría realizarse manualmente, su automatización reduce el número de operaciones necesarias y evita errores al configurar las conexiones de los doce canales.

5.3. Metodología de evaluación

Una vez finalizada la implementación del complemento, se definió un procedimiento de evaluación destinado a comprobar su funcionamiento dentro de REAPER, analizar su comportamiento temporal y comparar las señales obtenidas con las generadas mediante la ejecución original del modelo. El objetivo de esta etapa no fue volver a describir el desarrollo interno del sistema, sino establecer las condiciones y valorar los resultados.

Todas las pruebas se realizaron en el mismo equipo macOS empleado durante el desarrollo, manteniendo las versiones de REAPER, JUCE, LibTorch y el resto de las dependencias utilizadas para compilar el complemento.

REAPER se configuró con una frecuencia de muestreo de 48 kHz, coincidente con la empleada durante el entrenamiento del modelo y con la frecuencia admitida por el prototipo. Las pruebas principales se realizaron con bloques de 4096 muestras y una ventana de entrada de 8192 muestras para el modelo. El complemento se insertó en un bus de doce canales y el enrutamiento se preparó mediante el ReaScript descrito anteriormente. De esta forma, las señales de entrada se enviaron al bus de procesamiento y cada uno de los doce canales generados pudo recuperarse en una pista independiente.

La evaluación funcional comenzó comprobando que REAPER reconocía el archivo VST3, que el complemento podía insertarse en una pista multicanal y que el módulo TorchScript se cargaba sin producir excepciones. Durante la reproducción se revisó la recepción de los canales de entrada, la ejecución del modelo y la aparición de señal en las doce pistas de salida. También se comprobó que la organización de los canales se mantuviera

de acuerdo con el enrutamiento establecido y que el proyecto pudiera reproducirse y detenerse sin provocar el cierre del anfitrión.

La evaluación temporal se planteó comparando el tiempo empleado por el procesamiento con la duración disponible para cada bloque de audio. Para bloques de 4096 muestras a 48 kHz, el tiempo disponible por llamada es aproximadamente de 85.33 ms. Las mediciones obtenidas durante llamadas consecutivas permitieron determinar si la inferencia y el resto de las operaciones podían completarse dentro de este intervalo. Las primeras ejecuciones se observaron de forma diferenciada, ya que la inicialización del backend podía producir un tiempo superior al registrado posteriormente en régimen estable.

La continuidad del audio se evaluó reproduciendo fragmentos completos y revisando la posible aparición de cortes, silencios inesperados, saturaciones o transiciones audibles entre bloques. Esta comprobación se realizó tanto mediante escucha como mediante la observación de las formas de onda obtenidas al registrar las pistas de salida. De esta manera, pudo valorarse el efecto del mecanismo de solapamiento-suma y comprobar si las señales mantenían una evolución continua durante la reproducción.

Para estudiar la separación se analizaron individualmente las doce pistas generadas por el complemento. Se revisó la presencia de contenido en cada salida, la correspondencia entre los canales y la posible aparición de restos procedentes de otras señales. Esta evaluación combinó la escucha de las pistas con la inspección de sus formas de onda, permitiendo identificar tanto el contenido separado como posibles diferencias de nivel o artefactos.

Finalmente, se utilizó la ejecución original del modelo como referencia. La misma señal multicanal se procesó mediante el entorno original y mediante el complemento, manteniendo la frecuencia de muestreo, el orden de los canales y la duración del fragmento. Las salidas correspondientes se compararon atendiendo a su distribución por canales, estructura temporal, comportamiento de la amplitud y contenido perceptible. Esta comparación permitió valorar hasta qué punto la exportación a TorchScript y su ejecución continua dentro de REAPER conservaban el comportamiento del sistema de partida.

La Tabla 9 resume los bloques de evaluación y las evidencias consideradas en cada uno de ellos.

Bloque de evaluación	Aspectos comprobados	Evidencias utilizadas
Funcionamiento general	Reconocimiento del VST3, carga del modelo y reproducción	Estado del complemento, consola y funcionamiento en REAPER
Procesamiento multicanal	Recepción de las entradas y generación de las doce salidas	Pistas creadas y señales registradas

Comportamiento temporal	Tiempo de procesamiento respecto a la duración del bloque	Mediciones de llamadas consecutivas
Continuidad del audio	Cortes, discontinuidades, saturación y transiciones entre bloques	Escucha y observación de las formas de onda
Separación y comparación	Contenido de las salidas y relación con el modelo original	Audios procesados y análisis comparativo

Tabla 9 Pruebas y criterios empleados para evaluar el funcionamiento del complemento.

Fuente: elaboración propia.

Los resultados más representativos obtenidos mediante este procedimiento se presentan y analizan en el capítulo siguiente. Las mediciones completas, los registros adicionales y las evidencias complementarias se incluyen en el Anexo D.

6. RESULTADOS Y DISCUSIÓN

Las pruebas realizadas con el complemento *Demucs12* permitieron obtener resultados sobre su integración en REAPER, el funcionamiento del procesamiento multicanal, el comportamiento temporal de la inferencia y las señales generadas por el modelo. Se comprobó la posibilidad de cargar el módulo exportado a TorchScript, procesar una entrada de doce canales y recuperar las salidas correspondientes mediante el enrutamiento configurado en la estación de trabajo de audio.

Durante la reproducción también se obtuvieron mediciones del tiempo empleado en procesar los bloques y se observó la continuidad de las señales reconstruidas. Por otro lado, las salidas generadas por el complemento se analizaron individualmente y se contrastaron con las obtenidas mediante la ejecución original del modelo, con el propósito de identificar las semejanzas y diferencias producidas por su adaptación al entorno VST3.

Los resultados se presentan junto con su interpretación para valorar no solo el funcionamiento alcanzado, sino también las condiciones en las que puede utilizarse el prototipo y las limitaciones observadas durante las pruebas.

6.1. Exportación y validación del modelo en TorchScript

El primer resultado obtenido fue la exportación de Hybrid Demucs desde el checkpoint *best_epoch=155.ckpt* a dos archivos TorchScript. Se generó una versión destinada a CPU, denominada *demucs_export_torchscript_CPU.pt*, y otra preparada para el backend MPS, denominada *demucs_export_torchscript_MPS.pt*.

La exportación en CPU comenzó cargando la configuración y los parámetros del modelo original. La consola confirmó que se utilizaron doce canales de audio, una frecuencia de muestreo de 48 kHz y una entrada de ejemplo de tipo float32 con dimensiones [1, 12, 8192]. Antes de iniciar la conversión se ejecutó una llamada a `forward()`, que finalizó correctamente y produjo una salida con las mismas dimensiones. Después de esta comprobación se realizó el trazado y el archivo fue almacenado en la carpeta *exported_models*.

La Figura 20 muestra las dos etapas principales de este proceso. En la parte (a) se observa la carga del checkpoint y la ejecución correcta del modelo antes de exportarlo, mientras que la parte (b) recoge la finalización del trazado y el mensaje que confirma la creación del archivo para CPU.

```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

● (demucs) enrique@Air-de-Enrique TFG_PluginVST3_demucs12 % /opt/miniconda3/envs/demucs/bin/python /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/conf.yaml
PyTorch: 2.5.1
Usando dispositivo: cpu

Rutas:
Config: /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/conf.yaml
Checkpoint: /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/checkpoints/best_epoch=155.ckpt
Salida: /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/exported_models/demucs_export_torchscript_CPU.pt

Configuración usada para crear HDemucs:
sources: ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11']
audio_channels: 12
cac: False
samplerate: 48000
channels: 48
segment: 8
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/scripts/demucs_export_torchscript_CPU.py:110: FutureWarning:
, which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code du
usted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the func
e allowed to be loaded via this mode unless they are explicitly allowed by the user via 'torch.serialization.add_safe_globals'. We re
e full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  checkpoint = torch.load(model_path, map_location="cpu")
/opt/miniconda3/envs/demucs/lib/python3.9/site-packages/bitsandbytes/cextension.py:34: UserWarning: The installed version of bitsandbytes
quantization are unavailable.
  warn("The installed version of bitsandbytes was compiled without GPU support. "
'NoneType' object has no attribute 'cadam32bit_grad_fp32'

Modelo cargado correctamente.

Entrada de ejemplo:
Dispositivo example_input: cpu
Tipo example_input: torch.float32
Forma example_input: torch.Size([1, 12, 8192])

Primer parámetro: encoder.0.conv.weight
Dispositivo parámetro: cpu
Tipo parámetro: torch.float32

Probando forward en CPU antes de exportar...
Forward completado correctamente.
Dispositivo salida: cpu
Tipo salida: torch.float32
Forma salida: torch.Size([1, 12, 8192])
Output abs max: 3.5595920085906982

Exportando modelo TorchScript en CPU...

```

(a)

```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

(demucs) enrique@Air-de-Enrique TFG_PluginVST3_demucs12 % /opt/miniconda3/envs/demucs/bin/python /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/s
Exportando modelo TorchScript en CPU...
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:602: TracerWarning: Converting a tensor to a Python float might cause the trace
low of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  le = int(math.ceil(x.shape[-1] / hl))
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:30: TracerWarning: Converting a tensor to a Python boolean might cause the trace
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  max_pad = max(padding_left, padding_right)
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:31: TracerWarning: Converting a tensor to a Python boolean might cause the trace
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  if length <= max_pad:
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:38: TracerWarning: Converting a tensor to a Python boolean might cause the trace
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  assert out.shape[-1] == length + padding_left + padding_right
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:39: TracerWarning: Converting a tensor to a Python boolean might cause the trace
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  assert (out[...], padding_left: padding_left + length) == x0.all()
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:611: TracerWarning: Converting a tensor to a Python boolean might cause the trace
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  assert z.shape[-1] == le + 4, (z.shape, x.shape, le)
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:134: TracerWarning: Converting a tensor to a Python boolean might cause the trace
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  if not le % self.stride == 0:
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:140: TracerWarning: Converting a tensor to a Python boolean might cause the trace
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  assert inject.shape[-1] == y.shape[-1], (inject.shape, y.shape)
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/demucs.py:38: TracerWarning: Converting a tensor to a Python boolean might cause the trace
low of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  if self.max_steps is not None and T > self.max_steps:
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:332: TracerWarning: Converting a tensor to a Python boolean might cause the trace
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  assert z.shape[-1] == length, (z.shape[-1], length)
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:760: TracerWarning: Converting a tensor to a Python boolean might cause the trace
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  assert pre.shape[2] == 1, pre.shape
/opt/miniconda3/envs/demucs/lib/python3.9/site-packages/openunmix/filtering.py:21: TracerWarning: torch.tensor results are registered as constants in the trace. You can safely
n to create tensors out of constant variables that would be the same every time you call this function. In any other case, this might cause the trace to be incorrect.
  pi = 2 * torch.asin(torch.tensor(1.0))
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:622: TracerWarning: Converting a tensor to a Python float might cause the trace
low of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
  le = hl * int(math.ceil(length / hl)) + 2 * pad

Modelo exportado correctamente.
Guardado en: /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/exported_models/demucs_export_torchscript_CPU.pt

```

(b)

Figura 21 Resultado de la exportación de Hybrid Demucs para CPU: (a) carga del modelo y comprobación de la entrada y salida; (b) finalización del trazado y almacenamiento del archivo *demucs_export_torchscript_CPU.pt*. Fuente: elaboración propia.

El mismo procedimiento se realizó posteriormente mediante MPS. En este caso, PyTorch indicó que el backend se encontraba construido y disponible, y los parámetros del modelo se trasladaron al dispositivo mps:0. La inferencia previa a la exportación volvió a generar una salida de tipo float32 con dimensiones [1, 12, 8192]. Tras completar el trazado, el archivo *demucs_export_torchscript_MPS.pt* se guardó correctamente en la misma carpeta de modelos exportados.

La Figura 21 recoge los resultados de esta segunda exportación. La parte (a) muestra la disponibilidad de MPS, la carga del checkpoint y la ejecución correcta de `forward()`. En la parte (b) aparece el mensaje final de exportación y la ruta del archivo generado.

```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
● (base) enrique@Air-de-Enrique TFG_PluginVST3_demucs12 % conda activate demucs
● (demucs) enrique@Air-de-Enrique TFG_PluginVST3_demucs12 % /opt/miniconda3/envs/demucs/bin/python /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/scripts/demucs_export_torchscript_MPS.py
PyTorch: 2.5.1
MPS construido: True
MPS disponible: True
Usando dispositivo: mps

Rutas:
Config: /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/conf.yaml
Checkpoint: /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/checkpoints/best_epoch=155.ckpt
Salida: /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/exported_models/demucs_export_torchscript_MPS.pt

Configuración usada para crear HDemucs:
sources: ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11']
audio_channels: 12
cac: False
sample_rate: 48000
channels: 48
segment: 8
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/scripts/demucs_export_torchscript_MPS.py:115: FutureWarning: You are using `torch.load` which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/issues/9398 for details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed by loaded data and may reduce performance. This change is part of a series of security-related changes with the goal of making the PyTorch ecosystem safer and more robust to adversarial attacks. We recommend you start setting `weights_only=True` for all your calls to `torch.load` as soon as possible to avoid any issues in the future.
checkpoint = torch.load(model_path, map_location="cpu")
/opt/miniconda3/envs/demucs/lib/python3.9/site-packages/bitsandbytes/cextension.py:34: UserWarning: The installed version of bitsandbytes was compiled without quantization support.
warn("The installed version of bitsandbytes was compiled without GPU support. "
'NoneType' object has no attribute 'cadam32bit_grad_fp32'

Modelo cargado correctamente.

Entrada de ejemplo:
Dispositivo example_input: mps:0
Tipo example_input: torch.float32
Forma example_input: torch.Size([1, 12, 8192])

Primer parámetro: encoder.0.conv.weight
Dispositivo parámetro: mps:0
Tipo parámetro: torch.float32

Probando forward en MPS antes de exportar...
Forward completado correctamente.
Dispositivo salida: mps:0
Tipo salida: torch.float32
Forma salida: torch.Size([1, 12, 8192])
Output abs max: 3.7194743156433105

Exportando modelo TorchScript en MPS...

```

(a)


```
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

(demucs) enrique@Air-de-Enrique TFG_PluginVST3_demucs12 % /opt/miniconda3/envs/demucs/bin/python /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/M
Exportando modelo TorchScript en MPS...
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:602: TracerWarning: Converting a tensor to a Python flo
w of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    le = int(math.ceil(x.shape[-1] / hl))
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:30: TracerWarning: Converting a tensor to a Python bool
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    max_pad = max(padding_left, padding_right)
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:31: TracerWarning: Converting a tensor to a Python bool
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    if length <= max_pad:
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:38: TracerWarning: Converting a tensor to a Python bool
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    assert out.shape[-1] == length + padding_left + padding_right
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:39: TracerWarning: Converting a tensor to a Python bool
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    assert (out[..., padding_left: padding_left + length] == x0).all()
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:611: TracerWarning: Converting a tensor to a Python boo
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    assert z.shape[-1] == le + 4, (z.shape, x.shape, le)
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:134: TracerWarning: Converting a tensor to a Python boo
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    if not le % self.stride == 0:
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:140: TracerWarning: Converting a tensor to a Python boo
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    assert inject.shape[-1] == y.shape[-1], (inject.shape, y.shape)
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/demucs.py:38: TracerWarning: Converting a tensor to a Python boole
low of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    if self.max_steps is not None and T > self.max_steps:
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:332: TracerWarning: Converting a tensor to a Python boo
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    assert z.shape[-1] == length, (z.shape[-1], length)
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:760: TracerWarning: Converting a tensor to a Python boo
flow of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    assert pre.shape[2] == 1, pre.shape
/opt/miniconda3/envs/demucs/lib/python3.9/site-packages/openunmix/filtering.py:21: TracerWarning: torch.tensor results are registered as constants in t
n to create tensors out of constant variables that would be the same every time you call this function. In any other case, this might cause the trace t
pi = 2 * torch.asin(torch.tensor(1.0))
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/demucs/hdemucs.py:622: TracerWarning: Converting a tensor to a Python flo
low of Python values, so this value will be treated as a constant in the future. This means that the trace might not generalize to other inputs!
    le = hl * int(math.ceil(length / hl)) + 2 * pad

Modelo exportado correctamente.
Guardado en: /Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/exported_models/demucs_export_torchscript_MPS.pt
```

(b)

Figura 22 Resultado de la exportación de Hybrid Demucs para MPS: (a) selección del dispositivo y comprobación de la inferencia; (b) finalización del trazado y almacenamiento del archivo *demucs_export_torchscript_MPS.pt*. Fuente: elaboración propia.

Durante ambas exportaciones aparecieron varios mensajes `TracerWarning`. Estas advertencias se produjeron porque algunas operaciones internas convertían valores contenidos en tensores a tipos propios de Python, como valores booleanos o números en coma flotante. Al emplear `torch.jit.trace()`, estos valores pueden quedar registrados como constantes y no adaptarse automáticamente a entradas con características diferentes.

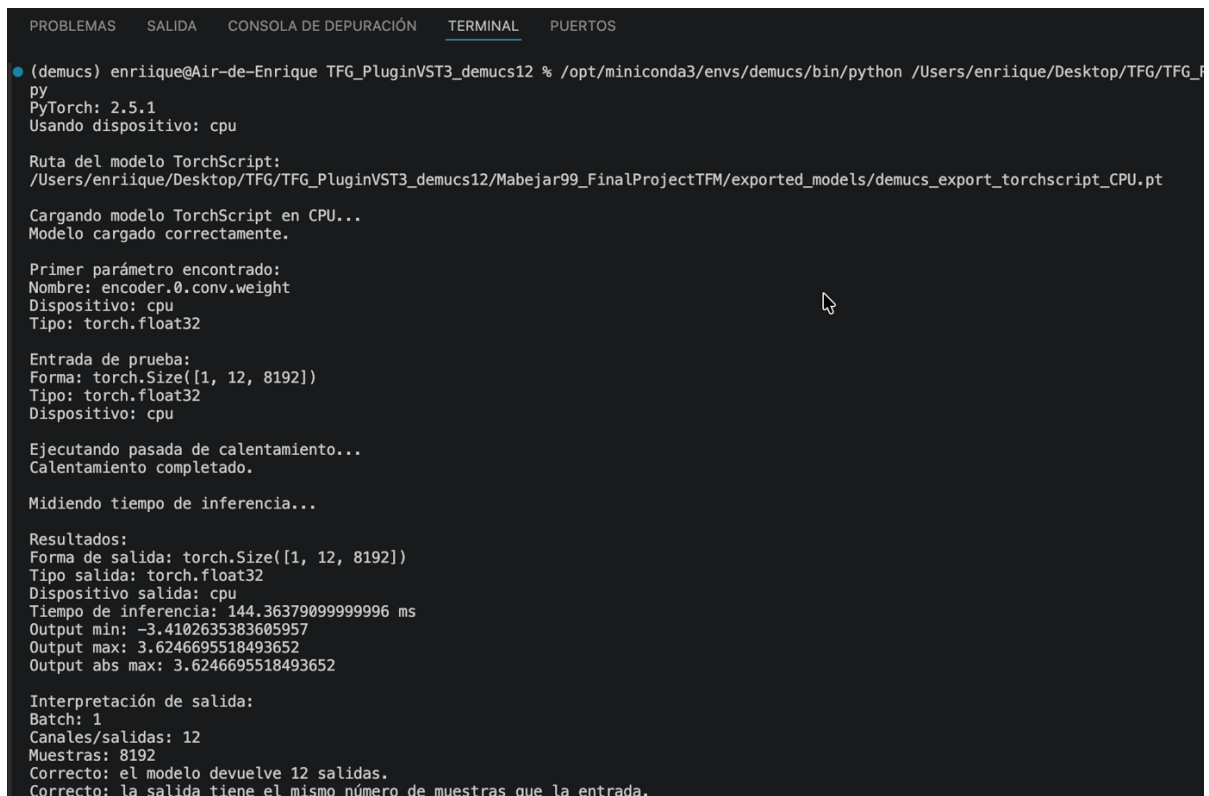
Estos mensajes no detuvieron el proceso ni impidieron generar los archivos, como confirman los mensajes finales mostrados en las Figuras 20 y 21. Además, la configuración final utiliza siempre una entrada fija de doce canales y 8192 muestras, coincidente con la utilizada durante el trazado. No obstante, las advertencias indican que el funcionamiento con otras dimensiones no queda garantizado y tendría que comprobarse de forma independiente.

Una vez exportadas las dos versiones, no se incorporaron directamente al complemento. Primero se utilizaron los scripts *test_demucs_export_torchscript_CPU.py* y *test_demucs_export_torchscript_MPS.py* para cargar los archivos generados y realizar una inferencia independiente.

En la prueba de CPU, el archivo se cargó correctamente, sus parámetros permanecieron en el dispositivo CPU y se procesó un tensor de prueba con dimensiones [1, 12, 8192]. Tras una primera pasada de calentamiento, la inferencia devolvió una salida float32 con doce canales y 8192 muestras.

En la prueba mediante MPS se comprobó nuevamente que el backend estaba disponible. El módulo se cargó en mps:0 y procesó una entrada con el mismo formato. La salida volvió a presentar dimensiones [1, 12, 8192]. En ambos casos se generaron valores finitos y distintos de cero, descartando una salida vacía o una pérdida de canales durante la exportación.

La Figura 22 muestra la validación posterior de los dos archivos. En ambas pruebas puede observarse la carga correcta del módulo, el dispositivo utilizado, la forma de la entrada y la forma de la salida obtenida.



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
• (demucs) enrique@Air-de-Enrique TFG_PluginVST3_demucs12 % /opt/miniconda3/envs/demucs/bin/python /Users/enrique/Desktop/TFG/TFG_F
py
PyTorch: 2.5.1
Usando dispositivo: cpu

Ruta del modelo TorchScript:
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/exported_models/demucs_export_torchscript_CPU.pt

Cargando modelo TorchScript en CPU...
Modelo cargado correctamente.

Primer parámetro encontrado:
Nombre: encoder.0.conv.weight
Dispositivo: cpu
Tipo: torch.float32

Entrada de prueba:
Forma: torch.Size([1, 12, 8192])
Tipo: torch.float32
Dispositivo: cpu

Ejecutando pasada de calentamiento...
Calentamiento completado.

Midiendo tiempo de inferencia...

Resultados:
Forma de salida: torch.Size([1, 12, 8192])
Tipo salida: torch.float32
Dispositivo salida: cpu
Tiempo de inferencia: 144.36379099999996 ms
Output min: -3.4102635383605957
Output max: 3.6246695518493652
Output abs max: 3.6246695518493652

Interpretación de salida:
Batch: 1
Canales/salidas: 12
Muestras: 8192
Correcto: el modelo devuelve 12 salidas.
Correcto: la salida tiene el mismo número de muestras que la entrada.
```

(a)

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
• (demucs) enrique@Air-de-Enrique TFG_PluginVST3_demucs12 % /opt/miniconda3/envs/demucs/bin/python /Users/enrique/Desktop/TFG/TFG_py
PyTorch: 2.5.1
MPS construido: True
MPS disponible: True

Ruta del modelo TorchScript:
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/exported_models/demucs_export_torchscript_MPS.pt

Cargando modelo TorchScript...
Modelo cargado correctamente.

Primer parámetro encontrado:
Nombre: encoder.0.conv.weight
Dispositivo: mps:0
Tipo: torch.float32

Entrada de prueba:
Forma: torch.Size([1, 12, 8192])
Tipo: torch.float32
Dispositivo: mps:0

Ejecutando pasada de calentamiento...
Calentamiento completado.

Midiendo tiempo de inferencia...

Resultados:
Forma de salida: torch.Size([1, 12, 8192])
Tipo salida: torch.float32
Dispositivo salida: mps:0
Tiempo de inferencia: 126.10166699999992 ms
Output min: -3.9042391777038574
Output max: 3.5522966384887695
Output abs max: 3.9042391777038574

Interpretación de salida:
Batch: 1
Canales/salidas: 12
Muestras: 8192
Correcto: el modelo devuelve 12 salidas.
Correcto: la salida tiene el mismo número de muestras que la entrada.
```

(b)

Figura 23 Prueba de los modelos después de su exportación: (a) carga e inferencia de la versión para CPU; (b) carga e inferencia de la versión para MPS. Fuente: elaboración propia.

Por tanto, los resultados confirmaron tanto la creación de los archivos TorchScript como su capacidad para cargarse. Esta validación previa permitió continuar con su incorporación mediante LibTorch al complemento VST3 sin atribuir posibles errores posteriores al proceso de exportación.

6.2. Integración y funcionamiento del complemento VST3

Después de comprobar que los modelos TorchScript funcionaban correctamente, el siguiente paso fue verificar que podían utilizarse dentro del complemento desarrollado con JUCE y LibTorch. La compilación en Xcode generó el plugin *Demucs12* en formato VST3, que posteriormente se instaló en la carpeta de complementos de macOS.

La comprobación más importante no fue únicamente que Xcode terminara la compilación, sino que REAPER fuera capaz de encontrar y utilizar el archivo generado.

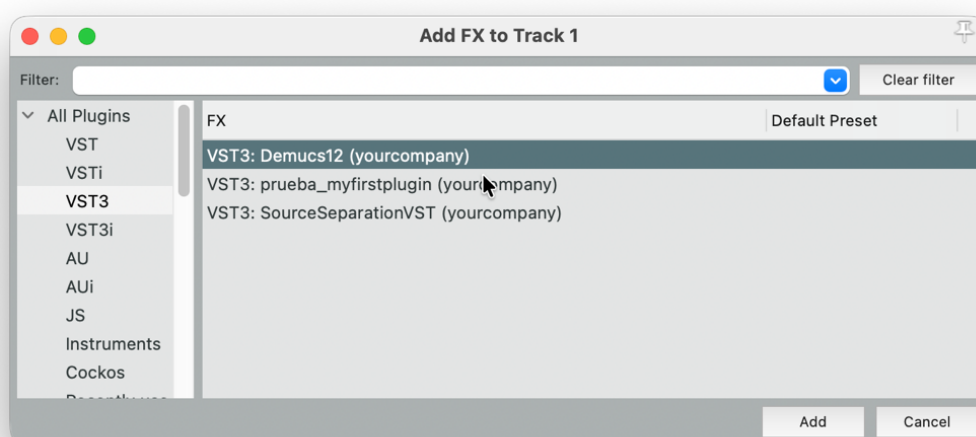
Después de actualizar la lista de efectos, *Demucs12* apareció dentro de la categoría VST3. Esto confirmó que el complemento se había instalado en la ubicación correcta y que REAPER reconocía la compilación como un efecto válido.

El siguiente paso fue comprobar que el modelo también se cargaba en el interior del complemento. Para poder ver los mensajes generados durante este proceso, REAPER se ejecutó desde la terminal. Al insertar *Demucs12*, la consola indicó que MPS estaba disponible y que se utilizaría la GPU del equipo mediante Apple Metal.

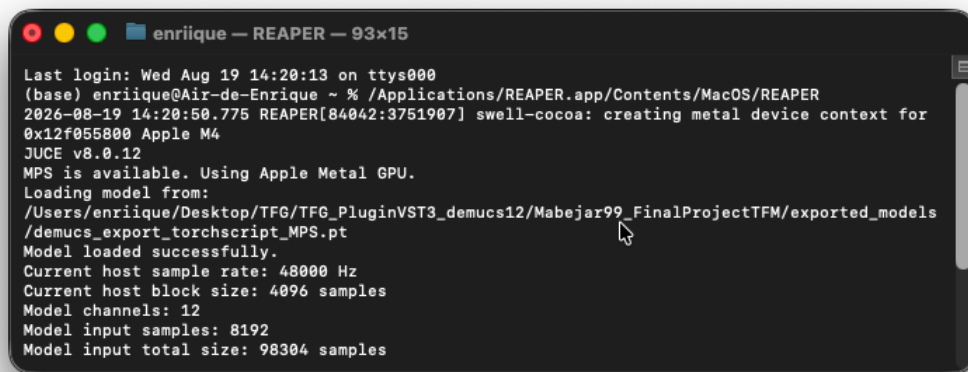
A continuación, el complemento buscó el archivo *demucs_export_torchscript_MPS.pt* dentro de la carpeta *exported_models*. El mensaje “Model loaded successfully” confirmó que LibTorch había localizado y cargado correctamente el modelo exportado. De esta forma, se comprobó que el resultado no se limitaba a mostrar el complemento en REAPER, sino que este también podía acceder al modelo necesario para realizar la separación.

La terminal mostró además los principales valores utilizados durante el procesamiento. REAPER trabajaba con una frecuencia de muestreo de 48000 Hz y proporcionaba bloques de 4096 muestras. Por su parte, Hybrid Demucs mantenía sus doce canales y una ventana de entrada de 8192 muestras. Esto daba lugar a una memoria de entrada formada por 98304 muestras en total, resultado de multiplicar los doce canales por las 8192 muestras de cada uno.

La Figura 23 reúne las dos comprobaciones realizadas. En la parte (a) se observa que REAPER reconoce *Demucs12* como un complemento VST3. En la parte (b) se muestran la selección de MPS, la carga correcta del modelo y los valores de procesamiento recibidos al iniciar el complemento.



(a)

A terminal window titled "enrique — REAPER — 93x15" showing the startup sequence of REAPER. The logs indicate a successful login, the creation of a metal device context for an Apple M4, and the use of Apple Metal GPU. A model is loaded from a specific path, and the terminal displays key configuration parameters: sample rate of 48000 Hz, block size of 4096 samples, 12 model channels, 8192 model input samples, and a total input size of 98304 samples.

```
enrique — REAPER — 93x15
Last login: Wed Aug 19 14:20:13 on ttys000
(base) enrique@Air-de-Enrique ~ % /Applications/REAPER.app/Contents/MacOS/REAPER
2026-08-19 14:20:50.775 REAPER[84042:3751907] swell-cocoa: creating metal device context for
0x12f055800 Apple M4
JUICE v8.0.12
MPS is available. Using Apple Metal GPU.
Loading model from:
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/exported_models
/demucs_export_torchscript_MPS.pt
Model loaded successfully.
Current host sample rate: 48000 Hz
Current host block size: 4096 samples
Model channels: 12
Model input samples: 8192
Model input total size: 98304 samples
```

(b)

Figura 24 Comprobación de la integración del complemento: (a) reconocimiento de *Demucs12* dentro de REAPER; (b) selección de MPS, carga del modelo TorchScript y configuración inicial del procesamiento. Fuente: elaboración propia.

Estas pruebas confirmaron que las distintas partes del sistema quedaban conectadas correctamente. REAPER reconocía el complemento, JUICE recibía la configuración de audio, LibTorch cargaba el archivo TorchScript y MPS quedaba seleccionado para ejecutar el modelo. A partir de este punto ya era posible comenzar las pruebas con señales de audio y analizar el tiempo de procesamiento, la continuidad de la reproducción y las salidas obtenidas.

6.3. Rendimiento temporal

Una vez cargado el complemento, el siguiente paso fue comprobar si Hybrid Demucs podía procesar cada bloque antes de que REAPER necesitara el siguiente, es decir, representa un tiempo disponible de aproximadamente 85.33 ms.

La primera configuración probada utilizaba una ventana de 16384 muestras. Sin embargo, el tiempo de inferencia superaba los 85.33 ms disponibles, por lo que el modelo no terminaba el procesamiento dentro del plazo marcado por REAPER. Por este motivo, esta opción se descartó y la ventana se redujo a 8192 muestras.

Con la nueva configuración, el complemento trabajó mediante MPS y mantuvo un solapamiento de 512 muestras. La terminal registró cinco tiempos de inferencia comprendidos entre 53.70 y 64.79 ms. La media de estas mediciones fue de 56.56 ms, por debajo del límite temporal del bloque.

La Tabla 10 resume los tiempos de inferencia obtenidos durante el funcionamiento del complemento mediante MPS.

Medición	Tiempo
Tiempo mínimo	53.70 ms
Tiempo medio	56.56 ms
Tiempo máximo	64.79 ms
Tiempo disponible por bloque	85.33 ms
Margen en el caso más desfavorable	20.54 ms

Tabla 10 Resumen de los tiempos de inferencia obtenidos durante el funcionamiento del complemento mediante MPS. Fuente: elaboración propia.

Incluso en la medición más alta, la inferencia terminó aproximadamente 20,54 ms antes de que finalizara el tiempo disponible. Por tanto, en las condiciones probadas, el modelo pudo procesar las ventanas a la velocidad necesaria para mantener la reproducción en tiempo real.

La Figura 24 muestra las mediciones registradas durante la ejecución. En todas ellas se mantienen los doce canales de entrada y salida, los bloques de 4096 muestras, la ventana de 8192 muestras, el solapamiento de 512 muestras y el uso de MPS.

```

(base) enrique@Air-de-Enrique ~ % /Applications/REAPER.app/Contents/MacOS/REAPER
2026-08-19 14:52:37.832 REAPER[84582:3779837] swell-cocoa: creating metal device context for 0x152852800 Apple M4
JUCE v8.0.12
MPS is available. Using Apple Metal GPU.
Loading model from:
/Users/enrique/Desktop/TFG/TFG_PluginVST3_demucs12/Mabejar99_FinalProjectTFM/exported_models/demucs_export_torchscript_MPS.pt
Model loaded successfully.
Current host sample rate: 48000 Hz
Current host block size: 4096 samples
Model channels: 12
Model input samples: 8192
Model input total size: 98304 samples
Overlap samples: 512
Current host sample rate: 48000 Hz
Current host block size: 4096 samples
Model channels: 12
Model input samples: 8192
Model input total size: 98304 samples
Overlap samples: 512
Inference time: 53.9265 ms | Samples: 4096 | Real input channels: 12 | Real output channels: 12 | Model channels: 12 | Window: 8192 | Overlap: 512 | Device: MPS
Inference time: 53.6971 ms | Samples: 4096 | Real input channels: 12 | Real output channels: 12 | Model channels: 12 | Window: 8192 | Overlap: 512 | Device: MPS
Inference time: 53.7815 ms | Samples: 4096 | Real input channels: 12 | Real output channels: 12 | Model channels: 12 | Window: 8192 | Overlap: 512 | Device: MPS
Inference time: 56.5984 ms | Samples: 4096 | Real input channels: 12 | Real output channels: 12 | Model channels: 12 | Window: 8192 | Overlap: 512 | Device: MPS
Inference time: 64.7867 ms | Samples: 4096 | Real input channels: 12 | Real output channels: 12 | Model channels: 12 | Window: 8192 | Overlap: 512 | Device: MPS

```

Figura 25 Tiempos de inferencia registrados durante el procesamiento de bloques de 4096 muestras mediante MPS. Fuente: elaboración propia.

Estos valores corresponden al tiempo empleado por la inferencia, que constituye la operación con mayor carga dentro de `processBlock()`. La reproducción estable observada en REAPER permitió comprobar que el resto de las operaciones también podían realizarse sin impedir la entrega del siguiente bloque. Esto permite hablar de procesamiento en tiempo real para la configuración utilizada.

Además del tiempo de procesamiento, se revisó la unión entre los bloques consecutivos. Antes de implementar el solapamiento-suma, no se apreciaban cortes ni

chasquidos durante la escucha. Sin embargo, al observar la señal ampliada en el software Audacity podían distinguirse pequeñas discontinuidades en algunos límites de bloque. Después de aplicar el solapamiento-suma con 512 muestras, estas variaciones dejaron de apreciarse en la forma de onda y tampoco apareció ningún efecto audible durante la reproducción. Esta comprobación fue visual y auditiva, pero no se conservó una captura que permitiera incluir una comparación gráfica en la memoria.

Los resultados muestran que la reducción de la ventana a 8192 muestras permitió cumplir el tiempo disponible por bloque, mientras que el solapamiento ayudó a mantener una transición continua entre fragmentos. De esta forma, la configuración final consiguió combinar la ejecución de Hybrid Demucs mediante MPS con una reproducción estable dentro de REAPER.

6.4. Resultados de la separación multicanal

Después de comprobar el rendimiento temporal, se realizó una prueba completa de separación utilizando el archivo *mixture.wav* disponible en el repositorio de GitHub del proyecto.

El archivo se reprodujo desde REAPER y se envió al bus multicanal que contenía el complemento *Demucs12*. Durante la reproducción, el modelo procesó la señal en tiempo real y entregó sus resultados a través de los doce canales de salida. El enrutamiento creado mediante el ReaScript permitió recoger cada uno de estos canales en una pista independiente.

Las doce pistas recibieron señal y ninguna de las salidas quedó vacía. La distribución de los canales se mantuvo de acuerdo con el enrutamiento configurado y el proyecto pudo reproducirse y detenerse correctamente. Tampoco se observaron errores durante la inferencia ni interrupciones que impidieran completar el procesamiento. Por tanto, la cadena completa funcionó como estaba previsto: REAPER envió el audio al complemento, Hybrid Demucs realizó la separación y las doce salidas regresaron a sus pistas correspondientes.

El complemento no genera directamente archivos de audio, ya que está pensado para procesar la señal en tiempo real dentro de REAPER. Por este motivo, no se dispone de doce archivos renderizados procedentes de esta ejecución concreta. Las salidas se escucharon y comprobaron directamente en las pistas creadas dentro del proyecto.

Como referencia adicional, el repositorio incluye una prueba offline realizada con *mixture.wav* y los archivos correspondientes a las doce salidas generadas por el modelo. Estos archivos permiten conocer el contenido esperado en cada canal y sirven para comprobar el comportamiento de la separación fuera de REAPER. También se incluyen los

scripts y el resto de los elementos necesarios para volver a ejecutar las pruebas descritas en este capítulo.

Debe tenerse en cuenta que los archivos de la prueba offline no son grabaciones obtenidas directamente del complemento. Por tanto, no se realizó una comparación numérica muestra a muestra entre ambas ejecuciones. La validación del VST3 fue principalmente funcional: se comprobó que el modelo procesaba la entrada en tiempo real, que las doce salidas recibían señal y que el contenido se distribuía correctamente dentro de REAPER.

En conjunto, esta prueba confirmó que la integración desarrollada permite utilizar el modelo Hybrid Demucs como un procesador multicanal dentro de la DAW.

6.5. Limitaciones y valoración global

En primer lugar, todas las pruebas se realizaron en un único ordenador macOS y utilizando REAPER como anfitrión. El procesamiento en tiempo real se comprobó mediante MPS, aprovechando la GPU del equipo a través de Apple Metal. Aunque la versión para CPU del modelo TorchScript pudo cargarse y ejecutar una inferencia independiente, no se demostró que pudiera mantener el procesamiento en tiempo real dentro del complemento. Tampoco se probó el VST3 en otras DAW, sistemas operativos o configuraciones de hardware, por lo que no puede asegurarse el mismo comportamiento fuera del entorno utilizado.

El funcionamiento en tiempo real también depende de una configuración concreta. Los resultados se obtuvieron con una frecuencia de muestreo de 48 kHz, bloques de 4096 muestras, una ventana de 8192 muestras y un solapamiento de 512 muestras. La ventana inicial de 16384 muestras superaba el tiempo disponible por bloque y tuvo que descartarse. Aunque el complemento obtiene el tamaño real de los bloques entregados por REAPER, no se comprobó su rendimiento con todas las configuraciones posibles del anfitrión.

Otra limitación es que el complemento no realiza remuestreo interno. El modelo fue preparado para trabajar a 48 kHz y, si REAPER utiliza otra frecuencia, únicamente se muestra una advertencia. Por tanto, para mantener las condiciones esperadas por Hybrid Demucs es necesario configurar previamente el proyecto con la frecuencia adecuada.

La entrada del modelo también mantiene una forma fija de doce canales y 8192 muestras. Esta forma coincide con la utilizada durante el trazado de TorchScript y con la configuración final del complemento. Como consecuencia, el modelo exportado no está pensado para recibir ventanas de distinta longitud o un número diferente de canales sin realizar una nueva adaptación y exportación.

La interfaz desarrollada es únicamente informativa y no permite modificar los parámetros del procesamiento. El usuario no puede cambiar desde el complemento el dispositivo empleado, el tamaño de la ventana, el solapamiento, la frecuencia de muestreo ni la ubicación del modelo. Cualquier modificación de estos valores requiere cambiar la configuración del proyecto o el código y volver a generar el complemento.

La evaluación de la separación fue principalmente funcional y auditiva. Se comprobó que `mixture.wav` podía procesarse en tiempo real, que las doce pistas recibían señal y que las salidas se distribuían correctamente. Sin embargo, no se grabaron las salidas del VST3 ni se realizó una comparación numérica muestra a muestra con los archivos obtenidos mediante la prueba offline. Tampoco se calcularon métricas objetivas de separación, como SDR o SI-SDR. Por este motivo, los resultados demuestran el funcionamiento de la integración, pero no permiten cuantificar con precisión la calidad de la separación ni las posibles diferencias respecto a la ejecución original del modelo.

Por último, los tiempos presentados proceden de cinco inferencias consecutivas realizadas durante una prueba concreta. Estas mediciones fueron suficientes para comprobar que el procesamiento quedaba por debajo de los 85.33 ms disponibles, pero no forman un estudio estadístico extenso. El margen temporal podría variar al utilizar otros equipos, ejecutar más complementos al mismo tiempo o aumentar la carga general del sistema.

Estas limitaciones no impiden cumplir el objetivo principal del trabajo, ya que se ha obtenido un prototipo capaz de ejecutar Hybrid Demucs dentro de REAPER y generar sus doce salidas en tiempo real. No obstante, delimitan las condiciones en las que se ha validado el sistema y señalan los aspectos que deberían mejorarse para convertirlo en una herramienta más flexible y preparada para su uso fuera del entorno de desarrollo.

7. CONCLUSIONES Y LÍNEAS FUTURAS

El trabajo realizado ha permitido transformar el modelo Hybrid Demucs heredado en un sistema capaz de utilizarse directamente dentro de REAPER mediante un complemento VST3. El resultado final recibe el audio desde la DAW, organiza las señales multicanal, ejecuta el modelo mediante LibTorch y devuelve las doce salidas a sus pistas correspondientes. Además, en la configuración utilizada durante las pruebas, el procesamiento pudo mantenerse en tiempo real y sin interrupciones audibles en el equipo de pruebas.

En este capítulo se revisa el grado de cumplimiento de los objetivos planteados al comienzo del proyecto y se reúnen las principales conclusiones obtenidas durante el desarrollo. Finalmente, se presentan varias líneas de trabajo que permitirían mejorar el prototipo actual, facilitar su utilización en otros equipos y ampliar la evaluación de su funcionamiento.

7.1. Cumplimiento de los objetivos

Al comienzo del trabajo se estableció como objetivo general desarrollar y evaluar un complemento VST3 multicanal capaz de ejecutar dentro de REAPER el modelo Hybrid Demucs. A partir de los resultados obtenidos, puede considerarse que este objetivo se ha cumplido dentro de las condiciones definidas para el prototipo. El complemento se desarrolló en C++ mediante JUCE, el modelo se integró utilizando LibTorch y las doce señales de salida pudieron recuperarse correctamente en REAPER. Además, con la configuración final, el procesamiento se mantuvo de forma continua y dentro del tiempo disponible por bloque.

El primer objetivo específico consistía en analizar el funcionamiento del modelo heredado. Para ello, se revisaron su configuración, el checkpoint utilizado, el número de canales, la frecuencia de muestreo y la forma de las señales de entrada y salida. Este análisis permitió comprobar que el sistema trabajaba con doce canales a 48 kHz y sirvió como base para preparar posteriormente una entrada con dimensiones [1, 12, 8192]. También se estudió la forma en la que el modelo procesaba los segmentos temporales y devolvía las señales separadas.

El segundo objetivo se centraba en conocer cómo funcionan los complementos de audio y cómo entrega REAPER los bloques de muestras. Durante las primeras pruebas se comprobó que la DAW llama periódicamente a `processBlock()` y que el número de muestras debe consultarse en cada llamada. También se calculó el tiempo correspondiente a cada bloque para poder compararlo con el tiempo empleado por la inferencia. Estos

conocimientos permitieron adaptar el funcionamiento del modelo al flujo continuo de audio utilizado por REAPER.

Los objetivos tercero y cuarto estaban relacionados con la creación del complemento y su configuración multicanal. Se desarrolló la estructura del VST3 mediante JUCE, se configuró el proyecto en Xcode y se obtuvo una compilación que pudo instalarse y ser reconocida por REAPER como un efecto válido. A continuación, se prepararon los buses de entrada y salida para trabajar con doce canales. El enrutamiento creado mediante el ReaScript permitió enviar el audio al bus de procesamiento y recuperar cada una de las doce salidas en una pista independiente, manteniendo su orden durante todo el recorrido.

El quinto objetivo planteaba preparar el modelo para utilizarlo desde C++ sin depender de la aplicación externa en Python. Para conseguirlo, Hybrid Demucs se exportó a TorchScript en dos versiones, una para CPU y otra para MPS. Antes de incorporarlas al complemento, ambas se cargaron y probaron de forma independiente. Estas pruebas confirmaron que los archivos exportados aceptaban una entrada de tipo float32 con dimensiones [1, 12, 8192] y devolvían una salida con la misma forma. Posteriormente, LibTorch permitió cargar el archivo destinado a MPS directamente desde el complemento.

Los objetivos sexto y séptimo abordaban la adaptación de los datos y la diferencia entre el tamaño de los bloques de REAPER y la ventana utilizada por el modelo. Las muestras recibidas mediante los búferes de JUCE se organizaron en una región de memoria continua y se transformaron en un tensor compatible con LibTorch. Después de la inferencia, se realizó el proceso contrario para devolver las señales a los canales de REAPER. Para mantener el contexto temporal se implementó una ventana histórica deslizante de 8192 muestras por canal, que se actualiza con cada nuevo bloque. La devolución del audio se completó mediante un solapamiento de 512 muestras para suavizar la unión entre fragmentos consecutivos.

El octavo objetivo requería mantener el procesamiento de forma continua y en tiempo real. La primera configuración, basada en una ventana de 16384 muestras, no alcanzó este objetivo porque el tiempo de inferencia superaba el tiempo disponible por bloque. Tras reducir la ventana a 8192 muestras, los tiempos registrados mediante MPS quedaron entre 53.70 y 64.79 ms. Estos valores se situaron por debajo de los 85,33 ms correspondientes a un bloque de 4096 muestras a 48 kHz. Durante las pruebas no se observaron paradas ni cortes audibles, por lo que este objetivo se cumplió para la configuración y el equipo utilizados.

El noveno objetivo consistía en evaluar el complemento dentro de REAPER. Las pruebas realizadas permitieron comprobar la carga del VST3 y del modelo, la estabilidad durante la reproducción, los tiempos de inferencia, la continuidad del audio y la presencia de señal en las doce salidas. La reproducción pudo iniciarse y detenerse con normalidad y no se observó una acumulación progresiva del retraso. No obstante, esta evaluación tuvo el alcance propio de un prototipo: el tiempo total de latencia no se midió mediante un procedimiento

específico y la calidad de la separación se valoró principalmente mediante escucha y comprobación funcional, sin utilizar métricas objetivas.

El décimo objetivo proponía comparar la solución desarrollada con el prototipo anterior. La principal mejora conseguida fue trasladar la ejecución del modelo al interior de REAPER. En la solución anterior, el procesamiento dependía del entorno externo de Python, mientras que el nuevo complemento permite cargar el modelo mediante LibTorch, procesar el audio durante la reproducción y devolver las doce salidas directamente a las pistas de la DAW. La prueba offline incluida en el repositorio permite consultar el resultado esperado para cada salida. Sin embargo, no se realizó una comparación numérica muestra a muestra entre esos archivos y las señales producidas en tiempo real. Por tanto, este objetivo se cumplió principalmente desde el punto de vista de la integración y del modo de utilización del sistema, pero no mediante una comparación cuantitativa de la calidad de ambas ejecuciones.

En conjunto, los objetivos relacionados con el análisis, el desarrollo, la integración y el procesamiento multicanal se cumplieron completamente. Los objetivos de evaluación y comparación también pudieron abordarse, aunque con las limitaciones propias de las pruebas realizadas. El resultado final demuestra la viabilidad de ejecutar el modelo heredado como un complemento VST3 multicanal dentro de REAPER y mantener su funcionamiento en tiempo real en el entorno utilizado durante el proyecto.

7.2. Conclusiones principales

La principal conclusión de este trabajo es que resulta viable utilizar un modelo basado en Hybrid Demucs como núcleo de procesamiento de un complemento VST3 multicanal. El modelo, que inicialmente se ejecutaba desde un entorno externo de Python, pudo trasladarse a una solución integrada en REAPER sin modificar su arquitectura ni repetir su entrenamiento. Esto permite acercar una herramienta desarrollada en un entorno de investigación a una forma de uso más próxima al trabajo habitual con audio dentro de una DAW.

El desarrollo también ha mostrado que la integración de un modelo de aprendizaje profundo en un complemento de audio no depende únicamente de conseguir cargarlo y ejecutar una inferencia. Una parte importante del trabajo consistió en conectar dos formas diferentes de manejar el audio. Por un lado, REAPER entrega bloques consecutivos relativamente pequeños y por otro, Hybrid Demucs necesita una ventana temporal mayor y organizada como un tensor multicanal. La ventana histórica deslizante permitió resolver esta diferencia y mantener disponible el contexto necesario sin detener la recepción de nuevos bloques.

El rendimiento temporal fue uno de los factores que más condicionó el diseño final. La configuración inicial con una ventana de 16384 muestras no podía completar la inferencia dentro del tiempo disponible, mientras que la reducción a 8192 muestras permitió mantener una reproducción estable mediante MPS. De esta experiencia se concluye que el funcionamiento en tiempo real no depende únicamente de la potencia del modelo o del equipo, sino también de decisiones como el tamaño de la ventana, el bloque utilizado por la DAW y el dispositivo encargado de ejecutar la inferencia. La integración requiere encontrar un equilibrio entre el contexto temporal proporcionado al modelo y el tiempo disponible para procesarlo.

La utilización de MPS fue determinante para alcanzar el rendimiento necesario en el equipo empleado. El uso de la GPU mediante Apple Metal permitió situar la inferencia por debajo de la duración de los bloques de audio y mantener un margen suficiente durante las pruebas. Sin embargo, este resultado debe entenderse como una comprobación realizada en una configuración concreta y no como una garantía de rendimiento en cualquier ordenador.

Otro resultado importante fue la gestión de las transiciones entre bloques. Aunque antes de aplicar el solapamiento no se apreciaban defectos claros durante la escucha, la observación ampliada de la forma de onda mostraba pequeñas discontinuidades. El procesamiento mediante solapamiento-suma permitió corregirlas sin introducir efectos audibles. Esto muestra la importancia de combinar la escucha con la inspección de las señales, ya que algunos problemas pueden no resultar evidentes auditivamente.

La organización multicanal también quedó resuelta de forma satisfactoria. El complemento pudo recibir, procesar y devolver doce canales manteniendo su correspondencia, mientras que el ReaScript simplificó la creación de las pistas y del enrutamiento necesario en REAPER. Por tanto, el resultado no se limita a ejecutar el modelo, sino que ofrece una cadena completa en la que las salidas pueden utilizarse individualmente dentro del propio proyecto de audio.

TorchScript y LibTorch proporcionaron una propuesta para ejecutar Hybrid Demucs desde C++ sin mantener Python abierto durante la utilización del complemento. Esta solución redujo la distancia entre el modelo original y el entorno final. No obstante, la exportación dejó características como el número de canales y la longitud de la entrada, por lo que la versión obtenida está válida para la configuración con la que fue exportada.

En cuanto a la separación, las pruebas confirmaron que las doce salidas recibían señal y que el sistema funcionaba de acuerdo con el enrutamiento previsto. Sin embargo, al no haberse realizado una evaluación mediante métricas objetivas ni una comparación muestra a muestra, no puede extraerse una conclusión cuantitativa sobre la calidad de las señales producidas por el complemento. Lo que sí queda demostrado es que la exportación y la integración no impiden ejecutar el modelo ni recuperar sus doce salidas durante la reproducción.

En conjunto, el proyecto demuestra que es posible integrar un modelo de separación multicanal dentro de una DAW y ejecutarlo en tiempo real bajo unas condiciones controladas. El resultado obtenido constituye una base funcional sobre la que se pueden incorporar mejoras de configuración, portabilidad y evaluación, pero ya permite comprobar el valor de trasladar este tipo de modelos desde un entorno experimental a una herramienta que puede utilizarse directamente durante el trabajo con audio.

7.3. Líneas de trabajo futuras

El presente trabajo ha permitido obtener un prototipo funcional capaz de ejecutar Hybrid Demucs dentro de un complemento VST3 y realizar el procesamiento multicanal directamente en REAPER. No obstante, tanto el modelo utilizado como las plataformas en las que se ha compilado y evaluado el sistema ofrecen posibilidades de ampliación. Estas mejoras podrían abordarse con mayor profundidad en un futuro Trabajo Fin de Máster.

La primera línea de trabajo consistiría en desarrollar o adaptar modelos específicamente orientados a la reducción del microphone bleeding. En este TFG se ha utilizado un modelo Hybrid Demucs previamente entrenado, sin modificar su arquitectura ni repetir el proceso de entrenamiento. Un trabajo posterior podría comparar diferentes arquitecturas de separación, modificar el modelo actual o entrenar nuevas versiones utilizando grabaciones más variadas en cuanto a instrumentos, salas, posiciones de los micrófonos y niveles de interferencia entre canales.

La evaluación de estos modelos no debería limitarse a comprobar la presencia de señal en las salidas. También sería necesario analizar la reducción del contenido procedente de otras fuentes, la conservación del timbre del instrumento principal y la aparición de posibles artefactos. Para ello, podrían combinarse métricas objetivas de separación con pruebas de escucha. Al mismo tiempo, debería estudiarse el equilibrio entre la calidad obtenida y el coste computacional, ya que un modelo más preciso no resultaría adecuado para el complemento si su tiempo de inferencia impediera mantener el procesamiento continuo.

Una segunda línea estaría relacionada con la ampliación del sistema a otros sistemas operativos. La versión desarrollada se ha compilado y probado en macOS sobre un equipo con procesador Apple Silicon, utilizando MPS para acelerar la inferencia. Como continuación, podrían prepararse proyectos de compilación específicos para Windows y Linux, manteniendo una base común de código mediante JUCE.

Esta adaptación requeriría configurar las versiones correspondientes de LibTorch, las rutas de cabeceras y bibliotecas, el enlazado de las dependencias dinámicas y los directorios de instalación del formato VST3 en cada sistema. También sería necesario adaptar la

selección del dispositivo de ejecución. Mientras que macOS utiliza MPS, las versiones para Windows y Linux podrían ejecutarse mediante CPU o aprovechar CUDA cuando se dispusiera de una GPU NVIDIA compatible.

Una vez generadas las distintas versiones, deberían realizarse pruebas equivalentes en cada plataforma para comprobar la carga del modelo, la configuración multicanal, la correspondencia de las doce entradas y salidas, los tiempos de inferencia, la latencia y la continuidad del audio. Estas pruebas permitirían determinar hasta qué punto el comportamiento obtenido en macOS puede reproducirse con otros equipos, sistemas operativos y configuraciones de hardware.

Ambas líneas podrían integrarse en un futuro Trabajo Fin de Máster dedicado al desarrollo de una segunda versión del sistema. Este trabajo combinaría la investigación de modelos de debleeding con mejores prestaciones y la creación de un complemento multiplataforma para macOS, Windows y Linux. El resultado permitiría superar el carácter experimental y dependiente de una única plataforma del prototipo actual, acercándolo a una herramienta más general, evaluable y distribuible en distintos entornos de producción musical.

ANEXOS

Anexo A. Configuración del entorno y dependencias

Este anexo recoge los pasos necesarios para preparar el entorno empleado durante el desarrollo, instalar las dependencias utilizadas y organizar los archivos principales del proyecto. Esta configuración permite ejecutar los scripts de Python, generar los módulos TorchScript y compilar el complemento VST3 mediante JUCE, Xcode y LibTorch.

CONFIGURACIÓN DEL ENTORNO

Antes de ejecutar los scripts relacionados con Hybrid Demucs se debe crear un entorno específico de Python:

- Instalar Conda mediante Miniconda o Anaconda.
- Descargar o clonar el repositorio público TFG-Hybrid-Demucs-VST3, presentado en el capítulo 5 [29]:

```
git clone <https://github.com/esm00046/TFG-Hybrid-Demucs-VST3>
```
- Acceder a la carpeta *Mabejar99_FinalProjectTFM*, donde se encuentra el archivo *environment-cuda.yml* proporcionado con el modelo.
- Abrir *environment-cuda.yml* y eliminar la siguiente dependencia:

```
cupy-cuda118=11.8.0
```

Esta modificación es necesaria porque CUDA está destinada a GPU de NVIDIA y no puede utilizarse en el MacBook con procesador Apple M4. No se modificaron el resto de las dependencias ni el nombre del entorno. La aceleración de la versión final se realizó mediante MPS, el backend de PyTorch para dispositivos Apple.

- Abrir un terminal y acceder a la carpeta del modelo:

```
cd <ruta_del_proyecto>/Mabejar99_FinalProjectTFM
```
- Crear el entorno utilizando el archivo modificado:

```
conda env create -f environment-cuda.yml
```
- Activar el entorno una vez terminada la instalación:

```
conda activate demucs
```


Con estos pasos se instalan las dependencias especificadas en el archivo y queda disponible el entorno demucs, utilizado para consultar el modelo original y ejecutar los scripts de exportación y validación.

INSTALACIÓN Y CONFIGURACIÓN DE LAS DEPENDENCIAS

Además del entorno de Python, el desarrollo requiere Xcode, JUCE y LibTorch:

- Instalar Xcode desde la App Store.
- Instalar sus herramientas de línea de comandos y aceptar la licencia:

```
xcode-select -install
```

```
sudo xcodebuild -license
```
- Para utilizar el cuaderno de inferencia desde Visual Studio Code, instalar IPython e ipywidgets si no se encuentran disponibles:

```
pip install ipython ipywidgets
```
- Descargar JUCE y ejecutar Projucer, que se utiliza para administrar y generar el proyecto del complemento.
- Descargar una distribución de LibTorch para macOS ARM64 compatible con la versión de PyTorch instalada. La carpeta descargada debe guardarse en una ubicación fija.

En caso de que macOS bloquee las bibliotecas dinámicas de LibTorch por haber sido descargadas de Internet, puede eliminarse el atributo de cuarentena mediante:

```
xattr -cr <ruta_de_libtorch>
```

Para enlazar LibTorch con el complemento se abre el archivo:

Plugins/Demucs12/Demucs12.jucer

Dentro de Projucer se selecciona *Exporters > Xcode (macOS)* y se introducen los siguientes valores:

- En *Header Search Paths*, marcando las rutas como no recursivas:

```
<ruta_de_libtorch>/include
```

```
<ruta_de_libtorch>/include/torch/csrc/api/include
```
- En *Extra Library Search Paths*:

```
<ruta_de_libtorch>/lib
```
- En *External Libraries to Link*:

```
torch
```

```
torch_cpu
```

```
c10
```
- En las opciones del compilador y del enlazador:

```
macOS Deployment Target: 11.0
C++ Language Standard: C++17
Extra Compiler Flags: -fno-modules -std=gnu++17 -mmacosx-
version-min=11.0
Extra Linker Flags: -Wl,-rpath,<ruta_de_libtorch>/lib -
mmacosx-version-min=11.0
```

Esta configuración debe guardarse en Projucer. Si las rutas se introducen únicamente en Xcode, pueden desaparecer al volver a generar el proyecto.

La instalación de PyTorch y la disponibilidad de MPS pueden comprobarse desde el entorno demucs mediante:

```
python -c "import torch; print(torch.__version__);
print(torch.backends.mps.is_available())"
```

En el equipo utilizado, la comprobación devolvía *true*, confirmando que PyTorch podía acceder a la GPU mediante MPS.

ORGANIZACIÓN DE LOS ARCHIVOS

El repositorio TFG-Hybrid-Demucs-VST3, disponible en el enlace indicado en el capítulo 5 [29], se organizó en varias carpetas para separar el modelo, el complemento, las pruebas y la documentación:

- *Mabejar99_FinalProjectTFM*: contiene el modelo Hybrid Demucs, su configuración y los scripts de Python.
- *Plugins/Demucs12*: contiene el proyecto JUCE del complemento VST3.
- *cpp_offline_tests*: contiene el proyecto empleado para comprobar el modelo desde C++ fuera de REAPER.
- *scripts*: contiene el ReaScript Demucs12_Bus.lua.
- *docs*: contiene la memoria.

Dentro de *Mabejar99_FinalProjectTFM* se encuentran los elementos necesarios para crear y validar los módulos TorchScript:

- *conf.yaml*, con la configuración del modelo.
- *demucs/*, con la implementación de Hybrid Demucs.
- *checkpoints/best_epoch=155.ckpt*, con los parámetros entrenados.
- *audio/*, con las señales utilizadas durante las pruebas.
- *scripts/*, con los programas de exportación y validación.
- *exported_models/*, donde se almacenan los módulos TorchScript para CPU y MPS.

La organización general del proyecto y el contenido de la carpeta correspondiente al modelo se muestran en la Figura 25.

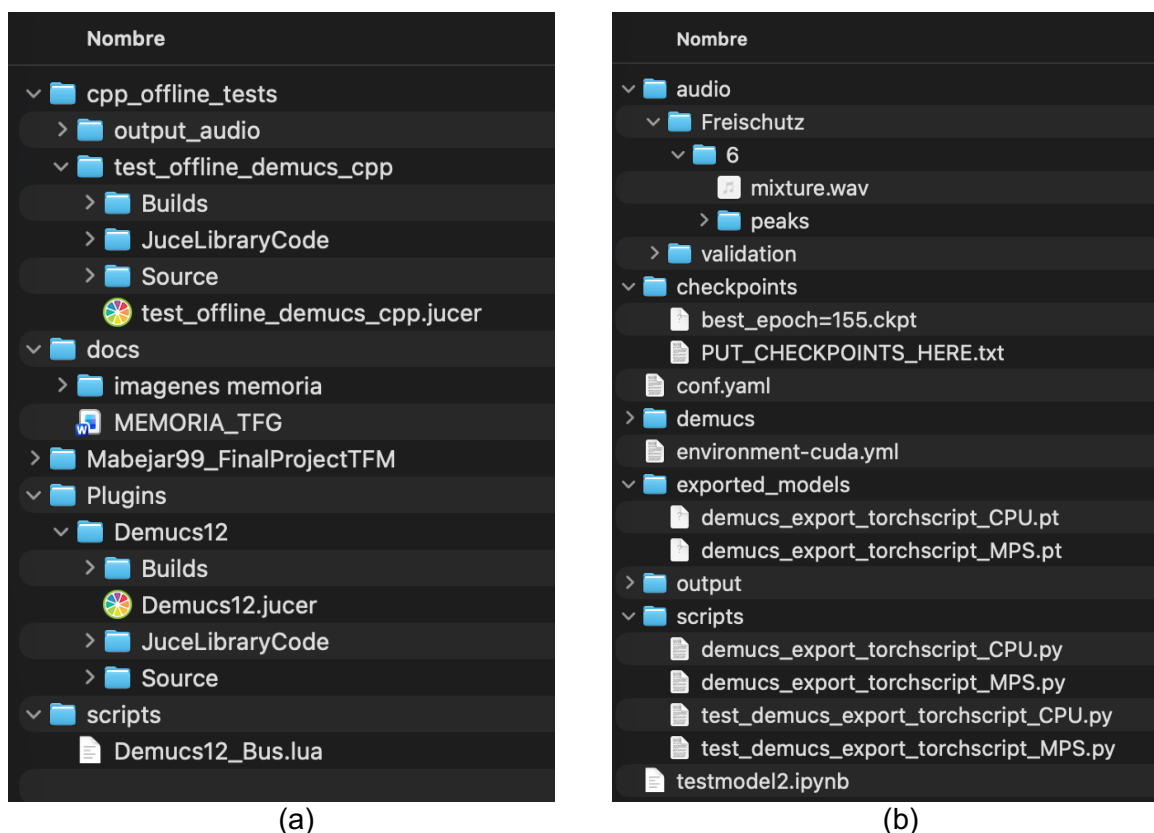


Figura 26 Organización de los archivos del proyecto: (a) directorios principales; (b) contenido de la carpeta correspondiente al modelo, los audios, el checkpoint y los módulos exportados.

Fuente: elaboración propia.

Una vez preparado el entorno, configuradas las dependencias y colocados los archivos en sus directorios correspondientes, el proyecto puede abrirse desde Projucer y compilarse en Xcode. La instalación del VST3 y su configuración multicanal dentro de REAPER se explican en el Anexo B.

Anexo B. Configuración multicanal y enrutamiento en REAPER

Este anexo recoge los pasos necesarios para instalar el complemento, configurar REAPER a la frecuencia utilizada por Hybrid Demucs y preparar el enrutamiento de sus doce canales de entrada y salida. Se parte de la configuración del entorno y de las dependencias descrita en el Anexo A.

INSTALACIÓN Y RECONOCIMIENTO DEL COMPLEMENTO

Durante el desarrollo se configuró REAPER para que buscara el complemento directamente en la carpeta donde Xcode generaba cada nueva compilación. De esta forma, no era necesario copiar manualmente *Demucs12.vst3* a la carpeta general de complementos después de cada modificación.

Para realizar esta configuración se siguieron los siguientes pasos:

- Abrir REAPER > *Settings* > *Plug-ins* > *VST*.
- Añadir en el campo VST plug-in paths la ruta correspondiente a la carpeta de compilación del proyecto. Para una compilación en modo Debug, su organización general era:

```
<ruta_del_repositorio>/Plugins/Demucs12/Builds/MacOSX/build/Debug
```

- Seleccionar *Re-scan* > *Clear cache and re-scan*.
- Abrir el buscador de efectos y comprobar que aparecía VST3: *Demucs12*.

Al generar una nueva versión desde Xcode, el archivo existente en esta carpeta se actualizaba automáticamente. Si REAPER continuaba utilizando una compilación anterior, se cerraba la aplicación y se repetía el borrado de la caché y el escaneo.

La Figura 26 muestra la ruta de compilación añadida a REAPER y la opción utilizada para borrar la caché y volver a buscar el complemento.

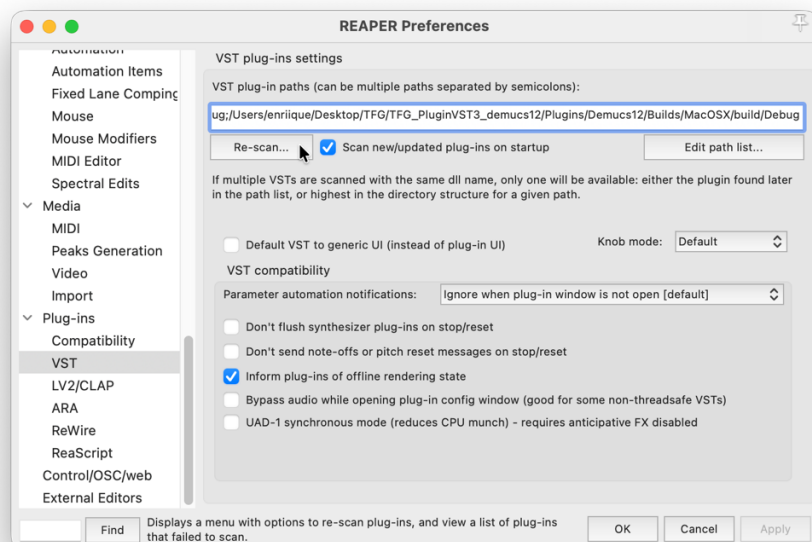


Figura 27 Configuración de REAPER para buscar el complemento directamente en la carpeta de compilación generada por Xcode. Fuente: elaboración propia.

CONFIGURACIÓN DEL DISPOSITIVO Y DEL PROYECTO

La versión final de Hybrid Demucs trabaja a una frecuencia de muestreo de 48 kHz. Por tanto, REAPER debe configurarse con los siguientes valores:

- Sistema de audio: CoreAudio.
- Dispositivo de salida: altavoces integrados del Mac o un dispositivo compatible con 48 kHz.
- Request sample rate: 48000 Hz.
- Request block size: 4096 muestras.

La frecuencia del proyecto también debe establecerse en 48000 Hz desde *File > Project Settings*. Con esta configuración, cada bloque de 4096 muestras representa aproximadamente 85.3 ms de audio.

El complemento no realiza remuestreo interno. Si el proyecto utiliza una frecuencia distinta de 48 kHz, la interfaz muestra una advertencia y el procesamiento no se ejecuta con la configuración prevista.

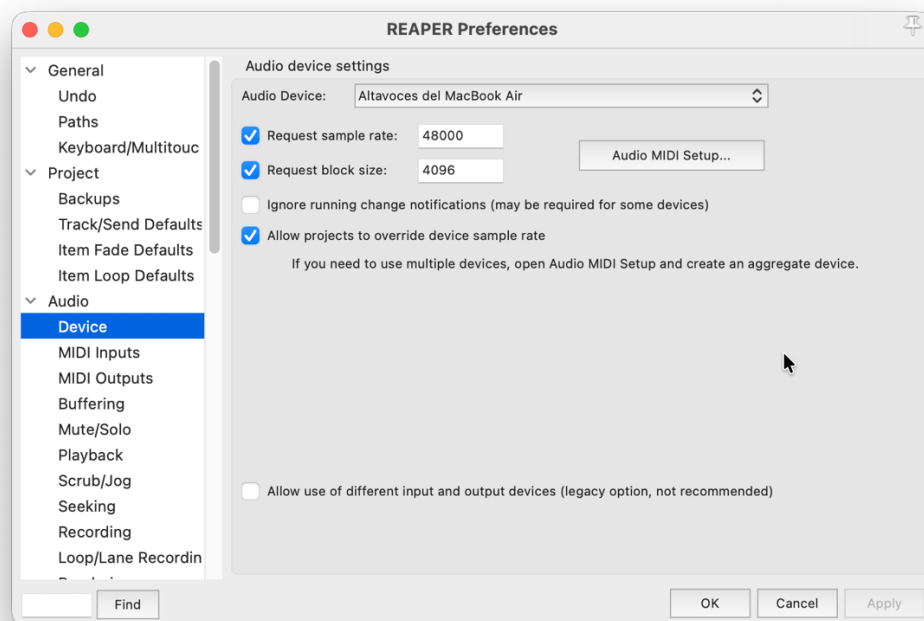


Figura 28 Configuración del dispositivo de audio de REAPER a 48 kHz y con un tamaño de bloque de 4096 muestras. Fuente: elaboración propia.

CONFIGURACIÓN MULTICANAL Y ENRUTAMIENTO

El procesamiento se organizó mediante una pista de origen que contenía la mezcla multicanal, un bus de procesamiento configurado con doce canales y doce pistas de salida independientes. El complemento *Demucs12* se insertó en el bus para recibir conjuntamente los doce canales y devolver las señales procesadas conservando su orden.

Para evitar la creación manual de todas las pistas y conexiones se utilizó el ReaScript *Demucs12_Bus.lua*, disponible en la carpeta scripts del repositorio. El archivo se encuentra disponible en la carpeta scripts del repositorio. Los pasos necesarios para cargarlo y ejecutarlo se indican a continuación:

- Abrir *Actions > Show action list*.
- Seleccionar *New action > Load ReaScript*.
- Elegir el archivo *Demucs12_Bus.lua*.
- Seleccionar la pista que contiene el archivo multicanal de entrada.
- Ejecutar el script desde la lista de acciones.

Después de su ejecución, el proyecto queda formado por la pista de origen, la pista *Demucs12 Bus* y las doce pistas de destino denominadas *Separated 1* a *Separated 12*. Cada salida del complemento se envía a la pista correspondiente, manteniendo la relación entre los canales de entrada y salida. El resultado final de esta configuración se muestra en la Figura 19 del apartado 5.2.8.

Anexo C. Guía de los scripts y archivos complementarios

Este anexo identifica los principales archivos de código desarrollados durante el proyecto y recoge las indicaciones básicas para ejecutar los programas auxiliares. La preparación del entorno y de las dependencias se explica en el Anexo A, mientras que la instalación del complemento y la utilización del ReaScript dentro de REAPER se describen en el Anexo B.

ARCHIVOS PRINCIPALES

Para evitar aumentar innecesariamente la extensión de la memoria, no se reproduce el código completo. Todos los archivos mencionados se encuentran disponibles en el repositorio público del proyecto. El código se encuentra distribuido en las siguientes carpetas:

- *Mabejar99_FinalProjectTFM/scripts*: contiene los programas utilizados para exportar Hybrid Demucs mediante TorchScript.
- *Mabejar99_FinalProjectTFM/exported_models*: almacena los módulos .pt generados para CPU y MPS.
- *cpp_offline_tests*: contiene la aplicación desarrollada para ejecutar el modelo desde C++ fuera de REAPER.
- *Plugins/Demucs12*: contiene el proyecto JUCE del complemento VST3.
- *scripts/Demucs12_Bus.lua*: corresponde al ReaScript utilizado para crear automáticamente el bus, insertar el complemento y preparar las pistas de salida.

Dentro del proyecto del complemento, el procesamiento principal se encuentra en *PluginProcessor.h* y *PluginProcessor.cpp*, mientras que *PluginEditor.h* y *PluginEditor.cpp* contienen la interfaz gráfica. La función de estos archivos se explica en el apartado 5.1.4.

GENERACIÓN DE LOS MÓDULOS TORCHSCRIPT

Antes de ejecutar los programas de exportación debe haberse preparado el entorno demucs siguiendo las indicaciones del Anexo A. También deben encontrarse en sus carpetas correspondientes el archivo de configuración *conf.yaml* y el checkpoint *best_epoch=155.ckpt*.

- Para generar los módulos TorchScript se abre un terminal, se accede a la carpeta del modelo y se activa el entorno:

```
cd <ruta_del_repositorio>/Mabejar99_FinalProjectTFM
conda activate demucs
```
- La versión destinada a CPU se genera mediante:

```
python scripts/demucs_export_torchscript_CPU.py
```
- Al finalizar el proceso se crea el archivo:
exported_models/demucs_export_torchscript_CPU.pt
- En un equipo Apple compatible con MPS puede generarse también la versión acelerada mediante:

```
python scripts/demucs_export_torchscript_MPS.py
```
- Este segundo programa comprueba previamente la disponibilidad del backend MPS y, si puede utilizarse, crea:
exported_models/demucs_export_torchscript_MPS.pt

Los dos archivos .pt deben conservarse dentro de *exported_models*, ya que son los módulos que posteriormente puede cargar el complemento mediante LibTorch. La versión

para MPS se utiliza en el equipo de desarrollo, mientras que la versión para CPU permite disponer de una alternativa cuando la aceleración mediante Apple Metal no está disponible.

EJECUCIÓN DE LA PRUEBA OFFLINE DESDE C++

Antes de integrar el modelo en el complemento se preparó una aplicación independiente mediante JUCE y LibTorch. El proyecto se encuentra en `cpp_offline_tests` y su código principal está almacenado en `cpp_offline_tests/test_offline_demucs_cpp/Source/Main.cpp`. Antes de compilarlo deben revisarse las rutas definidas al comienzo del archivo. Estas rutas indican:

- El módulo TorchScript que debe cargarse.
- El archivo WAV multicanal que se desea procesar.
- La carpeta en la que se guardarán los resultados.

Su organización general es la siguiente:

- `<ruta_del_repositorio>/Mabejar99_FinalProjectTFM/exported_models/`
- `<ruta_del_repositorio>/Mabejar99_FinalProjectTFM/audio/`
- `<ruta_del_repositorio>/cpp_offline_tests/output_audio/`

El archivo de entrada debe trabajar a 48 kHz, ya que esta prueba no realiza remuestreo. Una vez actualizadas las rutas, se abre el proyecto JUCE incluido en `cpp_offline_tests`, se genera el proyecto de Xcode y se compila la aplicación.

Al ejecutar el programa, el modelo procesa el archivo seleccionado y almacena sus doce señales de salida en `cpp_offline_tests/output_audio`, utilizando nombres comprendidos entre `source_0.wav` y `source_11.wav`.

Esta prueba es una herramienta auxiliar destinada a comprobar el modelo desde C++ y no es necesaria para utilizar normalmente el complemento. La compilación del VST3 se realiza desde `Plugins/Demucs12`, siguiendo la configuración descrita en el Anexo A. Posteriormente, su instalación y el uso del ReaScript `Demucs12_Bus.lua` se llevan a cabo mediante los pasos recogidos en el Anexo B.

Anexo D. Resultados y mediciones adicionales

Este anexo recoge evidencias complementarias obtenidas durante las pruebas finales del complemento Demucs12 dentro de REAPER. En el capítulo de resultados se han incluido

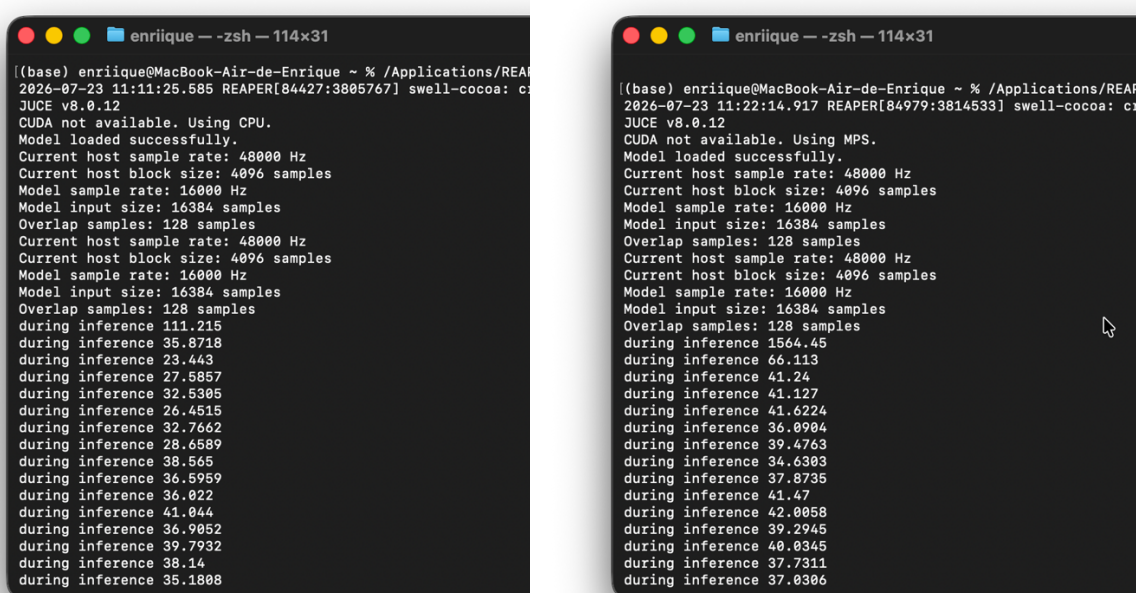
únicamente las mediciones más representativas, mientras que aquí se reúnen capturas y comprobaciones adicionales que permiten documentar el comportamiento temporal del sistema, la diferencia entre ejecución en CPU y mediante MPS, y la validación práctica del enrutamiento multicanal.

Las pruebas se realizaron con la configuración final del prototipo: frecuencia de muestreo de 48 kHz, bloques de 4096 muestras entregados por REAPER, ventana histórica de 8192 muestras, solapamiento de 512 muestras y procesamiento interno con doce canales. Esta configuración coincide con la utilizada en el complemento final y con los parámetros descritos en los capítulos de implementación y resultados.

MEDICIONES TEMPORALES EN CPU Y MPS

Durante el desarrollo se comprobó el tiempo de inferencia del modelo tanto en CPU como mediante Metal Performance Shaders. La versión de CPU permitió verificar que el módulo TorchScript podía ejecutarse correctamente sin depender de la aceleración gráfica, mientras que la versión MPS se utilizó como configuración principal por ofrecer un tiempo de procesamiento menor.

La Figura 28 muestra una comparación visual de los registros obtenidos en consola durante ambas ejecuciones. En los dos casos se observa la misma configuración general del sistema, pero cambia el dispositivo empleado para ejecutar el modelo. Esta comparación permitió confirmar que la aceleración mediante MPS era necesaria para mantener el procesamiento dentro del margen temporal disponible por bloque.



The image displays two side-by-side terminal windows from a macOS environment, showing the output of a script that tests model inference on both CPU and MPS. Both windows show identical system configuration details: sample rate of 48000 Hz, block size of 4096 samples, model sample rate of 16000 Hz, and input size of 16384 samples. The left window, representing CPU inference, shows a list of 20 inference times ranging from approximately 23.443 to 39.7932 milliseconds. The right window, representing MPS inference, shows a list of 20 inference times ranging from approximately 36.0994 to 66.113 milliseconds. The MPS times are generally higher than the CPU times, indicating that in this specific context, the CPU was faster than the MPS.

```
enrique — zsh — 114x31
((base) enrique@MacBook-Air-de-Enrique ~ % /Applications/REAPER[84427:3805767] swell-cocoa: c
2026-07-23 11:11:25.585 REAPER[84427:3805767] swell-cocoa: c
JUICE v8.0.12
CUDA not available. Using CPU.
Model loaded successfully.
Current host sample rate: 48000 Hz
Current host block size: 4096 samples
Model sample rate: 16000 Hz
Model input size: 16384 samples
Overlap samples: 128 samples
Current host sample rate: 48000 Hz
Current host block size: 4096 samples
Model sample rate: 16000 Hz
Model input size: 16384 samples
Overlap samples: 128 samples
during inference 111.215
during inference 35.8718
during inference 23.443
during inference 27.5857
during inference 32.5305
during inference 26.4515
during inference 32.7662
during inference 28.6589
during inference 38.565
during inference 36.5959
during inference 36.022
during inference 41.044
during inference 36.9052
during inference 39.7932
during inference 38.14
during inference 35.1808

enrique — zsh — 114x31
((base) enrique@MacBook-Air-de-Enrique ~ % /Applications/REAPER[84979:3814533] swell-cocoa: c
2026-07-23 11:22:14.917 REAPER[84979:3814533] swell-cocoa: c
JUICE v8.0.12
CUDA not available. Using MPS.
Model loaded successfully.
Current host sample rate: 48000 Hz
Current host block size: 4096 samples
Model sample rate: 16000 Hz
Model input size: 16384 samples
Overlap samples: 128 samples
Current host sample rate: 48000 Hz
Current host block size: 4096 samples
Model sample rate: 16000 Hz
Model input size: 16384 samples
Overlap samples: 128 samples
during inference 1564.45
during inference 66.113
during inference 41.24
during inference 41.127
during inference 41.6224
during inference 36.0994
during inference 39.4763
during inference 34.6303
during inference 37.8735
during inference 41.47
during inference 42.0058
during inference 39.2945
during inference 40.0345
during inference 37.7311
during inference 37.0306
```

Figura 29 Comparación de los tiempos de inferencia registrados durante la ejecución del complemento en CPU y mediante MPS. En ambos casos se utiliza una frecuencia de muestreo de 48 kHz, bloques de 4096 muestras, una ventana histórica de 8192 muestras y solapamiento-suma de 512 muestras. Fuente: elaboración propia.

Para interpretar estos resultados debe tenerse en cuenta que un bloque de 4096 muestras a 48 kHz representa aproximadamente 85.3 ms de audio. Por tanto, el tiempo de inferencia debe mantenerse por debajo de este valor para que el complemento pueda entregar el resultado antes de que REAPER solicite el siguiente bloque. Las mediciones mediante MPS se situaron aproximadamente entre 58 y 61 ms, por lo que dejaron un margen suficiente respecto al tiempo disponible por bloque. En cambio, la ejecución en CPU presentó un tiempo superior, por lo que se conservó como alternativa de validación, pero no como configuración principal del prototipo.

Estas mediciones justifican la decisión de utilizar MPS en el equipo de pruebas. Aunque la versión CPU permite comprobar la portabilidad básica del modelo TorchScript, la aceleración mediante la GPU integrada del MacBook resulta más adecuada para sostener el funcionamiento continuado dentro de REAPER.

COMPROBACIÓN DEL TAMAÑO DE VENTANA

También se realizaron pruebas modificando la longitud de la ventana de entrada del modelo. El objetivo era valorar el equilibrio entre el contexto temporal disponible para Hybrid Demucs y el coste computacional asociado a cada inferencia.

Inicialmente se comprobó el funcionamiento con ventanas mayores, ya que una mayor cantidad de muestras proporciona más contexto al modelo. Sin embargo, al aumentar la ventana también crece el tiempo necesario para ejecutar la inferencia. En las pruebas realizadas, una ventana de 16384 muestras ofrecía más contexto, pero el tiempo de procesamiento quedaba por encima del margen disponible por bloque, generando parones durante la reproducción.

Por este motivo, se adoptó como configuración final una ventana de 8192 muestras. Esta longitud representa un compromiso entre contexto temporal y tiempo de inferencia. Con esta configuración, el complemento pudo procesar bloques de 4096 muestras a 48 kHz manteniéndose por debajo de la duración temporal de cada bloque cuando se utilizaba MPS.

VALIDACIÓN DEL ENRUTAMIENTO MULTICANAL

Además de las mediciones temporales, se comprobó que el sistema pudiera utilizarse de forma práctica dentro de REAPER. Para ello, se ejecutó el ReaScript desarrollado, encargado de crear automáticamente el bus multicanal, insertar el complemento y generar las pistas de salida independientes.

La prueba confirmó que, al seleccionar una pista multicanal de doce canales, el script creaba correctamente el bus Demucs12 y las doce pistas separadas de salida. Cada canal procesado por el complemento quedaba redirigido a una pista independiente, permitiendo escuchar, supervisar o tratar cada señal por separado dentro de la DAW.

Esta comprobación resulta importante porque el objetivo del proyecto no era únicamente ejecutar el modelo desde C++, sino integrarlo dentro de un flujo de trabajo realista. La creación automática del bus y de las pistas evita que el usuario tenga que configurar manualmente el enrutamiento de doce canales, reduciendo la complejidad de uso del prototipo.

ALCANCE DE LAS MEDICIONES

Las mediciones incluidas en este anexo deben interpretarse como una validación experimental del prototipo en el equipo utilizado durante el desarrollo. No constituyen un estudio estadístico amplio ni una caracterización universal del rendimiento del modelo. Los tiempos pueden variar en función del equipo, la carga del sistema, la configuración de REAPER, el tamaño de bloque empleado y el número de procesos activos durante la reproducción.

Del mismo modo, la comparación entre CPU y MPS no pretende evaluar de forma general todos los posibles dispositivos de ejecución. Su finalidad es justificar la elección realizada en este TFG: emplear MPS como backend principal para acelerar la inferencia en un MacBook con procesador Apple M4, donde no se dispone de CUDA.

En conjunto, las pruebas recogidas en este anexo complementan los resultados principales de la memoria. Permiten documentar la diferencia entre CPU y MPS, justificar la selección de la ventana de 8192 muestras, confirmar la utilidad del solapamiento-suma y mostrar que el sistema puede organizar sus doce salidas dentro de REAPER mediante un flujo de trabajo automatizado.

BIBLIOGRAFÍA

- [1] Rosiński, A. (2020). Microphone techniques in stereo and surround recording. Jagiellonian University Press. <https://wuj.pl/en/book/microphone-techniques-in-stereo-and-surround-recording>
- [2] Cockos Incorporated. (2026). REAPER user guide. <https://www.reaper.fm/userguide.php>
- [3] CARABIAS-ORTI, Julio J.; COBOS, Máximo; VERA-CANDEAS, Pedro; RODRÍGUEZ-SERRANO, Francisco J. *Nonnegative signal factorization with learnt instrument models for sound source separation in close-microphone recordings*. *EURASIP Journal on Advances in Signal Processing*. 2013, art. 184. DOI: 10.1186/1687-6180-2013-184.
- [4] DROSSOS, Konstantinos; MIMILAKIS, Stylianos Ioannis; FLOROS, Andreas; VIRTANEN, Tuomas; SCHULLER, Gerald. *Close Miking Empirical Practice Verification: A Source Separation Approach*. En: *142nd Audio Engineering Society Convention*. Berlín, 20–23 de mayo de 2017.
- [5] DPA MICROPHONES. *What is “3:1 Rule”? DPA Microphones*. Consultado el 17 de agosto de 2026, de <https://www.dpamicrophones.com/dict/3-1-rule/>.
- [6] CANO, Estefanía; FITZGERALD, Derry; LIUTKUS, Antoine; PLUMBLEY, Mark D.; STÖTER, Fabian-Robert. «Musical Source Separation: An Introduction». *IEEE Signal Processing Magazine*. 2019, vol. 36, n.º 1, pp. 31–40. DOI: 10.1109/MSP.2018.2874719.
- [7] DÉFOSSEZ, Alexandre; USUNIER, Nicolas; BOTTOU, Léon; BACH, Francis. «Music Source Separation in the Waveform Domain». *arXiv:1911.13254*, 2019. [Documento original](#).
- [8] DÉFOSSEZ, Alexandre. «Hybrid Spectrogram and Waveform Source Separation». En: *Proceedings of the ISMIR 2021 Workshop on Music Source Separation*. 2021. [Documento original](#).
- [9] BÉJAR MARTOS, María Dolores. *Separation of Stems from Close-Microphone Mixtures for Spatialization of Audio Studio Recordings*. Trabajo Fin de Máster. Jaén: Universidad de Jaén, 2025. Repositorio institucional de la Universidad de Jaén.
- [10] STEINBERG MEDIA TECHNOLOGIES GMBH. [VST 3 Developer Portal](#). Documentación oficial del formato VST3 y de su integración en estaciones de trabajo de audio digital.
- [11] JUCE. [Tutorial: Create a Basic Audio/MIDI Plugin, Part 1](#). Documentación oficial para el desarrollo de complementos VST3 mediante JUCE.
- [12] PYTORCH CONTRIBUTORS. [Installing C++ Distributions of PyTorch](#). Documentación oficial de LibTorch y de la API de PyTorch para C++.
- [13] ZÖLZER, Udo. *Digital Audio Signal Processing*. 3.ª ed. Wiley, 2022.
- [14] OPPENHEIM, Alan V.; SCHAFER, Ronald W. *Discrete-Time Signal Processing*. 3.ª ed. Pearson, 2010.
- [15] JUCE. “juce::AudioBuffer Class Template Reference”. JUCE API Documentation [en línea]. Disponible en: https://docs.juce.com/master/classjuce_1_1AudioBuffer.html

- [16] JUCE. "juce::AudioProcessor Class Reference". JUCE API Documentation [en línea]. Disponible en: https://docs.juce.com/master/classjuce_1_1AudioProcessor.html
- [17] STEINBERG MEDIA TECHNOLOGIES GMBH. VST 3 Developer Portal [online]. 2026. Disp. desde: https://steinbergmedia.github.io/vst3_dev_portal/pages/index.html
- [18] COCKOS INCORPORATED. REAPER: Audio Production Without Limits [online]. [visitado 2026-08-11]. Disp. desde: <https://www.reaper.fm/>
- [19] PYTORCH. *TorchScript — PyTorch 2.6 documentation* [online]. [visitado 2026-08-11]. Disp. desde: <https://docs.pytorch.org/docs/2.6/jit.html>
- [20] PYTORCH. *Installing C++ Distributions of PyTorch* [online]. [visitado 2026-08-11]. Disp. desde: <https://docs.pytorch.org/cppdocs/installing.html>
- [21] STOLLER, Daniel; EWERT, Sebastian; DIXON, Simon. «Wave-U-Net: A Multi-Scale Neural Network for End-to-End Audio Source Separation». En: Proceedings of the 19th International Society for Music Information Retrieval Conference. París, 2018, pp. 334–340. [en línea]. [visitado 2026-08-12]. Disponible en: <https://arxiv.org/abs/1806.03185>
- [22] STÖTER, Fabian-Robert; UHLICH, Stefan; LIUTKUS, Antoine; MITSUFUJI, Yuki. «Open-Unmix: A Reference Implementation for Music Source Separation». Journal of Open Source Software. 2019, vol. 4, n.º 41, art. 1667. DOI: 10.21105/joss.01667. Disponible en: <https://doi.org/10.21105/joss.01667>
- [23] IZOTOPE, INC. RX 12 Advanced [online]. 2026. [visitado 2026-08-12]. Disponible en: <https://www.izotope.com/products/rx-advanced>
- [24] IZOTOPE, INC. RX Advanced Release Notes [online]. 2026. [visitado 2026-08-12]. Disponible en: <https://www.izotope.com/pages/release-notes/rx-advanced>
- [25] STEINBERG MEDIA TECHNOLOGIES GMBH. SpectraLayers Pro 12.0.30: Operation Manual [online]. 2025. [visitado 2026-08-12]. Disponible en: https://download.steinberg.net/downloads_software/SpectraLayers_12/help/Pro/index.html
- [26] META RESEARCH. Demucs: Music Source Separation [código fuente, online]. GitHub. [visitado 2026-08-12]. Disponible en: <https://github.com/facebookresearch/demucs>
- [27] HENNEQUIN, Romain; KHLIF, Anis; VOITURET, Félix; MOUSSALLAM, Manuel. «Spleeter: A Fast and Efficient Music Source Separation Tool with Pre-Trained Models». Journal of Open Source Software. 2020, vol. 5, n.º 50, art. 2154. DOI: 10.21105/joss.02154.
- [28] Meta Research. (2020). Denoiser: Real time speech enhancement in the waveform domain [Código fuente]. GitHub. <https://github.com/facebookresearch/denoiser>
- [29] Martínez, E. S. (2026). TFG-Hybrid-Demucs-VST3 [Código fuente]. GitHub. <https://github.com/esm00046/TFG-Hybrid-Demucs-VST3>.